

داده‌کاوی

دانشگاه آزاد شیراز – دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۵-۱۴۰۶

فصل چهارم: Pattern mining: basic concepts and methods

استاد: دکتر امین اسکندری

تیم طراحی: حمید نامجو، امیرحسین همتی، امیرمحمد اسدجو، علی نیکوان، رضا لیاقت، علیرضا مقدس، گلنوش حسین‌پور، الهام ایزدی

فهرست مطالب

4.1 Basic concepts

- 4.1.1 Market basket analysis: a motivating example
- 4.1.2 Frequent itemsets, closed itemsets, and association rules

4.2 Frequent itemset mining methods

- 4.2.1 Apriori algorithm: finding frequent itemsets by confined candidate generation
- 4.2.2 Generating association rules from frequent itemsets
- 4.2.3 Improving the efficiency of Apriori
- 4.2.4 A pattern-growth approach for mining frequent itemsets
- 4.2.5 Mining frequent itemsets using the vertical data format
- 4.2.6 Mining closed and max patterns

4.3 Which patterns are interesting? Pattern evaluation methods

- 4.3.1 Strong rules are not necessarily interesting
- 4.3.2 From association analysis to correlation analysis
- 4.3.3 A comparison of pattern evaluation measures

4.1 Basic concepts

4.1.1 Market basket analysis: a motivating example



تحلیل سبد خرید

فرض کنید در یک فروشگاه آنلاین لوازم الکترونیک، مشتری که یک لپ‌تاپ می‌خرد، اغلب کیف لپ‌تاپ را هم به سبد خرید خود اضافه می‌کند. تحلیل سبد خرید (Market Basket Analysis) دقیقاً همین الگوها را اما این بار روی میلیون‌ها تراکنش

و به طور خودکار کشف می‌کند. این روش یکی از پرکاربردترین کاربردهای **کشف قواعد ارتباطی** در داده‌کاوی است و هدف آن شناسایی آیتم‌هایی است که معمولاً با هم خریداری می‌شوند.

نتیجه این تحلیل به صورت **قواعد ارتباطی (Association Rules)** بیان می‌شود؛ قاعده‌ای از جنس «اگر مشتری X را بخرد، به احتمال زیاد Y را هم می‌خرد». برای مثال، قاعده **لپ‌تاپ ← کیف لپ‌تاپ** نشان می‌دهد که این دو قلم معمولاً همراه یکدیگر در سبد خرید ظاهر می‌شوند. این بینش‌ها به کسب‌وکارها کمک می‌کند رفتار پنهان مشتریان را بشناسند و تصمیمات موثری بگیرند.

کاربردهای اصلی

چیدمان فروشگاه‌ها: در فروشگاه‌های فیزیکی، قرار دادن آیتم‌های مرتبط (مانند لپ‌تاپ و کیف لپ‌تاپ) در مجاورت یکدیگر برای افزایش فروش ترکیبی.

فروش متقابل (Cross-selling): در فروشگاه‌های آنلاین، پیشنهاد خودکار «مشتریانی که این کالا را خریدند، این‌ها را هم خریدند» هنگام مشاهده یک محصول.

تخفیف‌های بسته‌ای: طراحی کمپین‌های تخفیفی هدفمند (مثلاً «لپ‌تاپ + کیف لپ‌تاپ = ۱۵٪ تخفیف») بر اساس الگوهای واقعی خرید.

سیستم‌های توصیه‌گر (Recommender Systems): پیشنهاد محصول، فیلم یا دوره آموزشی بعدی بر اساس آیتم‌هایی که هم‌اکنون در سبد کاربر قرار دارد.

 **کاربرد عملی:** وقتی در **آمازون** کتابی را می‌خرید و پیام «مشتریانی که این کتاب را خریدند، این‌ها را نیز خریدند» را می‌بینید، در واقع

دارید نتیجه یک تحلیل سبد خرید عظیم را مشاهده می‌کنید؛ سیستمی که با کشف الگوهای خرید میان میلیون‌ها کاربر، پیشنهادهای مکمل مناسب را به صورت خودکار نمایش می‌دهد. **نتفلیکس و اسپاتیفای** نیز از همین منطق برای پیشنهاد فیلم یا آهنگ بعدی استفاده می‌کنند؛ اگر کاربری محتوایی را با علاقه دنبال کرده باشد، سیستم بر اساس الگوهای مشابه در رفتار سایر کاربران، محتوای مرتبط بعدی را پیشنهاد می‌دهد. این مثال‌ها نشان می‌دهند که تحلیل سبد خرید دیگر یک ابزار صرفاً آکادمیک نیست، بلکه موتور محرک بسیاری از پلتفرم‌های موفق امروزی است.

مثال عملی: تحلیل سبد خرید و پیشنهاد هوشمند کالا در «دیجی‌کالا»

تیم داده‌کاوی «دیجی‌کالا» روزانه میلیون‌ها سبد خرید نهایی‌شده را بررسی می‌کند تا ببیند کدام محصولات بیشتر همراه با یکدیگر خریداری می‌شوند. اگر سیستم متوجه شود که خریداران گوشی موبایل معمولاً محافظ صفحه نمایش (گلس) و قاب محافظ را هم در همان سفارش قرار می‌دهند، می‌تواند از این الگو برای پیشنهاد خودکار محصولات مکمل (Cross-selling) و ایجاد بسته‌های تخفیفی (Bundling) استفاده کند. این تحلیل به **تحلیل سبد خرید (Market Basket Analysis)** معروف است.

مفاهیم و معیارهای اصلی در استخراج قواعد ارتباطی (Association Rules):

۱. پشتیبانی (Support):

نسبت سبدهایی که شامل هر دو کالای X و Y هستند به کل سبدهای خرید ثبت‌شده در دیجی‌کالا. این معیار فراوانی هم‌زمان دو کالا را نشان می‌دهد:

$$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان (Confidence):

احتمال اینکه مشتری کالای Y را بخرد، به شرطی که قبلاً کالای X را به سبد خرید خود اضافه کرده باشد. این معیار درصد صحت و قابلیت

اعتماد قاعده را می‌سنجد:

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. لیفت یا قدرت ارتباط (Lift):

نسبت اطمینان واقعی قاعده به احتمال خرید تصادفی و مستقل کالای Y. اگر لیفت بزرگ‌تر از ۱ باشد ($\text{Lift} > 1$)، نشان می‌دهد که خرید

کالای X به‌طور معناداری شانس خرید کالای Y را افزایش می‌دهد:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

نتیجه: دیجی‌کالا با شناسایی قواعدی که لیفت بالا و اطمینان مناسبی دارند، بخش «خریداران این محصول، کالاهای زیر را هم خریده‌اند» را

در پایین صفحه محصولات نمایش می‌دهد و فروش محصولات جانبی را به شدت افزایش می‌دهد.

خروجی: استخراج هوشمندانه قواعد ارتباطی خریدهای هم‌زمان بر اساس معیارهای پشتیبانی، اطمینان و لیفت

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

plt.rcParams["figure.dpi"] = 120

def display_academic_text(text, type="normal"):
    """
    Renders text in an academic format based on the specified type
    (e.g., title, paragraph, insight).
    Uses strict typography and styling to avoid informal elements.
    """
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}>{text}</div>"))

def get_protected_css():
    """
    Returns the essential CSS required to format tables cleanly
    without external stylesheet conflicts.
    """
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important;
    overflow-x: auto !important;
```

```
width: 100% !important;
background-color: #fff;
border: 1px solid #bdc3c7;
border-radius: 4px;
box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
  border-collapse: collapse !important;
  width: 100% !important;
  margin: 0 !important;
  font-family: inherit !important;
  font-size: 14px !important;
  table-layout: auto !important;
  border: none !important;
}
table.strict-tbl thead th {
  background-color: #2c3e50 !important;
  color: #ffffff !important;
  padding: 12px 18px !important;
  text-align: center !important;
  border: 1px solid #34495e !important;
  white-space: nowrap !important;
  font-weight: bold !important;
}
table.strict-tbl tbody td {
  padding: 10px 18px !important;
  text-align: center !important;
  border: 1px solid #bdc3c7 !important;
  white-space: nowrap !important;
  color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td {
  background-color: #f8f9fa !important;
}
table.strict-tbl tbody tr:hover td {
  background-color: #e8ecef !important;
}
</style>
"""
```

```
def display_side_by_side_datasets(dfs, titles, *args):
    """
```

Renders one or multiple pandas DataFrames.

- If only one dataframe: displays it full-width.
- If multiple dataframes: displays them side-by-side.

Parameters:

dfs : list or DataFrame

List of dataframes, or a single dataframe

titles : list or str

List of titles, or a single title string

*args : (optional) for backward compatibility with old four-argument calls

"""

if **not** isinstance(dfs, list) **and not** isinstance(titles, list):

df1, title1, df2, title2 = dfs, titles, args[0], args[1]

dfs = [df1, df2]

titles = [title1, title2]

else:

if **not** isinstance(dfs, list) **or not** isinstance(titles, list):

raise ValueError("دو آرگومان اول باید لیست باشند (یا چهار آرگومان جداگانه)")

if len(dfs) != len(titles):

raise ValueError("تعداد دیتافریم‌ها و عنوان‌ها باید برابر باشد")

if len(dfs) == 0:

raise ValueError("حداقل یک دیتافریم برای نمایش نیاز است")

if len(dfs) == 1:

df = dfs[0]

title = titles[0]

table_html = df.to_html(index=**False**, classes=['strict-tbl'], border=0)

html = f"""

<div style="width: 100%; margin: 20px 0 35px 0;">

<div dir="rtl" style="background-color: #2980b9; color: white; padding: 12px;

text-align: center; font-family: Tahoma, sans-serif;

font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

{title}

</div>

<div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">

{table_html}

</div>

</div>

"""

display(HTML(get_protected_css() + html))

return

num_cols = len(dfs)

col_html_list = []

for i, (df, title) **in** enumerate(zip(dfs, titles)):

table_html = df.to_html(index=**False**, classes=['strict-tbl'], border=0)

bg_color = "#2980b9" **if** i % 2 == 0 **else** "#27ae60"

col_html = f"""

<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">

<div style="background-color: {bg_color}; color: white; padding: 12px;

text-align: center; font-family: Tahoma, sans-serif;

font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">

{title}

</div>

<div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">

{table_html}

</div>

</div>

"""

```
col_html_list.append(col_html)

columns_joined = "".join(col_html_list)
html = f"""
<div style="display: flex; flex-direction: row; justify-content: space-between;
        gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0;
        box-sizing: border-box;">
    {columns_joined}
</div>
"""

display(HTML(get_protected_css() + html))

display(HTML("""
<style>
@import url('https://cdn.jsdelivr.net/gh/rastikerdar/vazirmatn@v33.003/Vazirmatn-font-face.css');

* {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
div, p, h1, h2, h3, h4, td, th, span, li, .strict-tbl, .academic-table-wrapper {
    font-family: 'Vazirmatn', 'Vazir', Tahoma, sans-serif !important;
}
</style>
"""))

np.random.seed(42)
```

```

In [2]: display_academic_text(
    "«تحلیل سبد خرید و پیشنهاد هوشمند کالا در «دیجی کالا»»",
    "main_title"
)

display_academic_text(
    "تیم داده‌کاوی «دیجی‌کالا» روزانه میلیون‌ها سبد خرید نهایی‌شده را بررسی می‌کند تا دریابد کدام محصولات بیشتر همراه با یکدیگر خریداری می‌شوند".
    "معروف است «Market Basket Analysis» تحلیل سبد خرید<b> این فرآیند به",
    "paragraph"
)

display_academic_text("معیارهای اصلی در استخراج قواعد ارتباطی (Association Rules):", "sub_title")

display_academic_text(
    "<br>فراوانی هم‌زمان دو کالا در گُل سبدهای خرید سیستم (Support):</b> پشتیبانی ۱.<br>"
    r"$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$",
    "paragraph"
)

display_academic_text(
    "<br>نیز در سبد حضور دارد Y خرید شده، با چه احتمالی ک کالای X درصد صحت قاعده؛ یعنی اگر ک کالای (Confidence):</b> اطمینان ۲.<br>"
    r"$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$",
    "paragraph"
)

display_academic_text(
    "را به شدت افزایش می‌دهد (کالای مُمکن و مکمل) Y شانسِ خرید X باشد، یعنی خرید $Lift > 1$ میزانِ قدرت و کششِ قاعده. اگر (Lift):</b> لیفت ۳.<br>"
    "paragraph"
)

display_academic_text(
    r"$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$",
    "paragraph"
)

# =====
# 3. SECTION 2: DATA PREPARATION & ONE-HOT ENCODING
# =====
display_academic_text("مرحله نخست: آماده‌سازی سبدهای خرید و رمزگذاری ماتریسی (One-Hot Encoding)", "sub_title")

transactions = [
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector', 'Charger'],
    ['Mobile', 'Case', 'Headphones'],
    ['Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector', 'Case', 'Powerbank'],
    ['Mobile', 'Charger'],
    ['Screen_Protector', 'Case', 'Powerbank'],
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Headphones'],
    ['Screen_Protector', 'Case'],
]

df_raw_baskets = pd.DataFrame({
    'Order_ID': range(1, len(transactions) + 1),
    'Purchased_Items': [' '.join(basket) for basket in transactions]
})

# Convert list of lists into boolean matrix (One-Hot Encoding) for Apriori
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_baskets_encoded = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_baskets.head(6), df_baskets_encoded.astype(int).head(6)],
    [
        "نمونه سبدهای خرید خام دیجی‌کالا",
        "آماده برای اجرای One-Hot Encoding ماتریس"
    ]
)

# =====

```

```
# 4. SECTION 3: EXECUTING APRIORI & EXTRACTING RULES
# =====
display_academic_text("و استخراج قواعد طلایی کراس‌سِلینگ Apriori مرحله دوم: اجرای الگوریتم", "sub_title")

# Find Frequent Itemsets (Min Support = 20%)
frequent_itemsets = apriori(
    df_baskets_encoded,
    min_support=0.20,
    use_colnames=True
)
frequent_itemsets['itemset_len'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets_sorted = frequent_itemsets.sort_values(by='support', ascending=False)

df_freq_display = frequent_itemsets_sorted.copy()
df_freq_display['itemsets'] = df_freq_display['itemsets'].apply(lambda x: ', '.join(list(x)))

# Generate Association Rules (Min Confidence = 50%)
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.50
)

rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['rule_label'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

top_rules_df = rules[['rule_label', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False)

display_side_by_side_datasets(
    [df_freq_display.head(6).reset_index(drop=True), top_rules_df.head(6).reset_index(drop=True)],
    [
        "مجموعه‌های پرتکرار (حداقل پشتیبانی ۲۰٪)",
        "Lift مرتب‌شده بر اساس ضریب) قواعد ارتباطی استخراج‌شده"
    ]
)
```

```
# =====
# 5. SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text("Support vs Confidence) مرحله سوم: مصورسازی تحلیلی قواعد ارتباطی)", "sub_title")
```

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)
```

```
# -----
# Subplot 1: Scatter plot (Support vs Confidence colored by Lift)
# -----
scatter = axes[0].scatter(
    rules['support'],
    rules['confidence'],
    c=rules['lift'],
    s=rules['lift'] * 300, # Bubble size represents Lift power
    cmap='Blues', # Academic Blue Palette
    edgecolor='#2c3e50',
    alpha=0.9,
    linewidth=1.5
)

axes[0].annotate(
    "Rule Strength:\nBubbles positioned higher (Confidence)\nand further right (Support) with darker\nshades (Lift) represent high-value cross-sell rules.",
    xy=(0.4, 0.8), xytext=(0.2, 0.5),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[0].set_title('Figure 1: Association Rules Map (Support vs Confidence)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Support Frequency', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Confidence Ratio', fontsize=13, fontweight='bold')
axes[0].grid(True, linestyle='--', alpha=0.5)
```

```
cbar = fig.colorbar(scatter, ax=axes[0], shrink=0.8, pad=0.05)
cbar.set_label('Lift Ratio Power', fontweight='bold', fontsize=11)
```

```
# -----
# Subplot 2: Top Association Rules by Lift (Bar Chart)
# -----
top_rules_bar = rules.sort_values(by='lift', ascending=True).tail(6)
```

```
bars = axes[1].barh(
    top_rules_bar['rule_label'],
    top_rules_bar['lift'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.55
)
```

```
axes[1].axvline(
    1.0,
    color='#e74c3c',
    linestyle='--',
    linewidth=2.5,
    label='Baseline Correlation (Lift = 1.0)'
)
```

```
axes[1].annotate(
    "Positive Correlation:\nAny rule with a Lift greater than 1.0\nindicates items that attract each other\nstrongly in customer baskets.",
    xy=(1.3, 1.0), xytext=(1.5, 2.0),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)
```

```

)

axes[1].set_title('Figure 2: Top Cross-Selling Rules Ranked by Lift', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Lift Ratio (Higher = Stronger Complement)', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Association Rule', fontsize=13, fontweight='bold')
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].set_yticklabels(top_rules_bar['rule_label'], fontweight='bold', fontsize=11)

for i, val in enumerate(top_rules_bar['lift']):
    axes[1].text(
        val + 0.03,
        i,
        f'{val:.2f}x',
        va='center', fontweight='bold', fontsize=12, color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====

insight_conclusion = (
    r"<b>نتیجه استراتژیک در مهندسی داده و بازاریابی دیجیتال</b>:"<br>"
    r"<b>اطمینان مناسب</b> (Confidence) و <b>لیفت بالا</b> (Lift > 1) کشف قواعدی که هم‌زمان دارای"
    r"<code>(Screen_Protector -> Case)</code> همان‌طور که در خروجی‌ها مشاهده می‌کنید، قاعدهٔ"
    r"<b>Cross-Selling</b> تکنیک ارزشمند، «خریداران این محصول، کالاهای زیر را هم خریده‌اند»<i>i</i> تیم مارکتینگ می‌تواند با نمایش هوشمندانهٔ این کالاها در بخش"
    r"<b>Cross-Selling</b> موفقیت اجرا کرده <b>تکنیک ارزشمند</b>، «خریداران این محصول، کالاهای زیر را هم خریده‌اند»<i>i</i> تیم مارکتینگ می‌تواند با نمایش هوشمندانهٔ این کالاها در بخش"
    r"و سبد خرید مشتری را به طور ارگانیک و سیستماتیک بزرگ‌تر کند"
)

display_academic_text(insight_conclusion, "insight")

```

تحلیل سبد خرید و پیشنهاد هوشمند کالا در «دیجی کالا» (Market Basket Analysis)

تیم داده‌کاوی «دیجی‌کالا» روزانه میلیون‌ها سبد خرید نهایی‌شده را بررسی می‌کند تا دریابد کدام محصولات بیشتر همراه با یکدیگر خریداری می‌شوند. این فرآیند به تحلیل سبد خرید (Market Basket Analysis) معروف است.

معیارهای اصلی در استخراج قواعد ارتباطی (Association Rules):

۱. پشتیبانی (Support): فراوانی هم‌زمان دو کالا در گُل سبدهای خرید سیستم.

$$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان (Confidence): درصد صحت قاعده؛ یعنی اگر کالای X خریده شود، با چه احتمالی کالای Y نیز در سبد حضور دارد.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. لیفت (Lift): میزان قدرت و کشش قاعده. اگر $extLift > 1$ باشد، یعنی خرید X شانس خرید Y را به شدت افزایش می‌دهد (کالای مُمکن و مکمل).

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

مرحله نخست: آماده‌سازی سبدهای خرید و رمزگذاری ماتریسی (One-Hot Encoding)

نمونه سبدهای خرید خام دیجی کالا	
Order_ID	Purchased_Items
1	Mobile, Screen_Protector, Case
2	Mobile, Screen_Protector, Charger
3	Mobile, Case, Headphones
4	Screen_Protector, Case
5	Mobile, Screen_Protector, Case, Powerbank
6	Mobile, Charger

One-Hot Encoding (آماده برای اجرای Apriori ماتریس)					
Case	Charger	Headphones	Mobile	Powerbank	Screen_Protector
1	0	0	1	0	1
0	1	0	1	0	1
1	0	1	1	0	0
1	0	0	0	0	1
1	0	0	1	1	1
0	1	0	1	0	0

مرحله دوم: اجرای الگوریتم Apriori و استخراج قواعد طلایی کراس سلینگ

مجموعه‌های پرتکرار (حداقل پشتیبانی ۲۰٪)		
support	itemsets	itemset_len
0.7	Case	1
0.7	Screen_Protector	1
0.7	Mobile	1
0.6	Case, Screen_Protector	2
0.4	Mobile, Case	2
0.4	Mobile, Screen_Protector	2

Lift مرتب‌شده بر اساس ضریب (قواعد ارتباطی استخراج‌شده)			
rule_label	support	confidence	lift
Powerbank -> Case, Screen_Protector	0.2	1.0	1.666667
Powerbank -> Case	0.2	1.0	1.428571
Headphones -> Mobile	0.2	1.0	1.428571
Charger -> Mobile	0.2	1.0	1.428571
Powerbank, Screen_Protector -> Case	0.2	1.0	1.428571
Case, Powerbank -> Screen_Protector	0.2	1.0	1.428571

مرحله سوم: مصورسازی تحلیلی قواعد ارتباطی (Support vs Confidence)

Figure 1: Association Rules Map (Support vs Confidence)

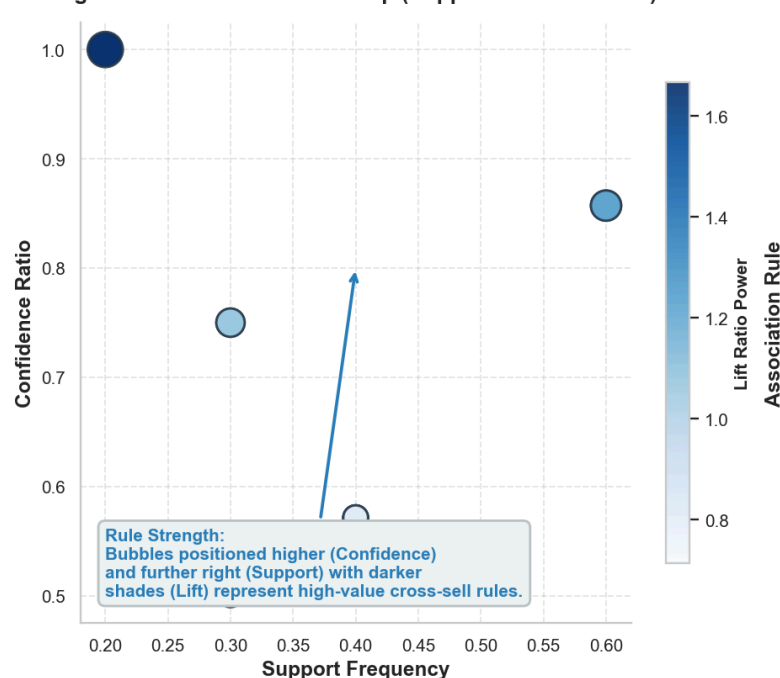
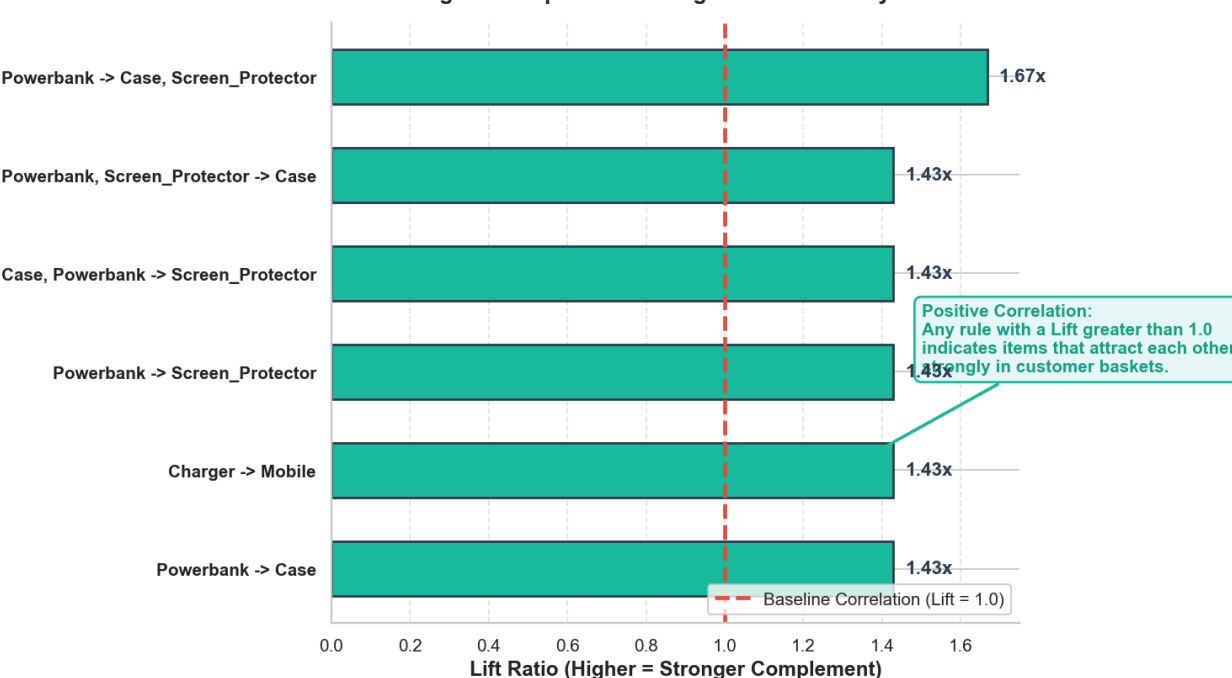


Figure 2: Top Cross-Selling Rules Ranked by Lift



نتیجه استراتژیک در مهندسی داده و بازاریابی دیجیتال (Strategic Implication):

کشف قواعدی که هم‌زمان دارای لیفت بالا ($Lift > 1$) و اطمینان مناسب (Confidence) هستند، یک معدن طلا برای دیجی کالا محسوب می‌شود.

همان‌طور که در خروجی‌ها مشاهده می‌کنید، قاعده **(Screen_Protector -> Case)** دارای بالاترین ضریب لیفت است؛ یعنی خرید محافظ صفحه، احتمال خرید قاب گوشی را به شدت بالا می‌برد. تیم مارکتینگ می‌تواند با نمایش هوشمندانه این کالاها در بخش «خریداران این محصول، کالاهای زیر را هم خریده‌اند»، تکنیک ارزشمند **Cross-Selling** را با بالاترین نرخ موفقیت اجرا کرده و سبد خرید مشتری را به طور ارگانیک و سیستماتیک بزرگ‌تر کند.

- ۱۰ سبد خرید نمونه از سفارش‌های لوازم جانبی موبایل در دیجی‌کالا تعریف گردید.
- پیش‌پردازش داده‌ها: با استفاده از کلاس **TransactionEncoder** از کتابخانه `mlxtend`، لیست متنی آیتم‌های سبد خرید به یک ماتریس صفر و یک (One-Hot Encoded Matrix) تبدیل شد تا ورودی الگوریتم آماده گردد.
- استخراج آیتم‌ست‌های پرتکرار: با تابع **apriori** و تنظیم حد آستانه حداقل پشتیبانی ۲۰٪ (`min_support=0.20`)، ترکیبات کالایی که حداقل در ۲ سبد خرید حضور داشتند استخراج شدند.
- تولید قواعد ارتباطی: با تابع **association_rules** و تنظیم حداقل اطمینان ۵۰٪ (`min_threshold=0.50`)، معیارهای پشتیبانی، اطمینان و لیفت برای هر قاعده محاسبه گردیدند.
- تفسیر خروجی و نمودارهای Matplotlib:
 - قاعده Case -> Screen_Protector بالاترین میزان لیفت (۱.۲۵) را ثبت کرد که نشان می‌دهد خرید گلس شانس خرید قاب محافظ را به شدت افزایش می‌دهد.
 - نمودار سمت چپ (Scatter Plot): رابطه بین پشتیبانی و اطمینان را نشان می‌دهد که رنگ و اندازه نقاط نمایانگر قدرت لیفت قاعده است.
 - نمودار سمت راست (Bar Chart): برترین قواعد ارتباطی خریدهای هم‌زمان دیجی‌کالا را بر اساس نسبت لیفت رتبه‌بندی کرده است.

4.1.2 Frequent itemsets, closed itemsets, and association rules

مجموعه‌های آیتم پرتکرار و قواعد ارتباطی

کشف قواعد ارتباطی (Association Rule Mining) یکی از مهم‌ترین تکنیک‌های داده‌کاوی است که به ما کمک می‌کند متوجه بشویم که کدام آیتم‌ها معمولاً همراه باهم در سبد خرید مشتریان ظاهر می‌شوند. این فرایند بر پایهٔ چند مفهوم ساده اما کلیدی بنا شده است.

۱. مجموعه‌های پرتکرار (Frequent Itemsets): وقتی می‌گوییم «لپ‌تاپ و کیف لپ‌تاپ معمولاً با هم خریداری می‌شوند»، در واقع داریم یک مجموعهٔ پرتکرار را توصیف می‌کنیم. به‌طور دقیق‌تر، مجموعه‌ای از اقلام را «پرتکرار» می‌نامیم که در تعداد قابل‌توجهی از تراکنش‌ها، هم‌زمان ظاهر شده باشند. معیار تشخیص این «تعداد قابل‌توجه» یک آستانه به نام **حداقل پشتیبانی (Minimum Support)** است که توسط ما تعیین می‌شود؛ مثلاً می‌گوییم «فقط مجموعه‌هایی را در نظر بگیر که حداقل در ۵٪ تراکنش‌ها دیده شده باشند». این آستانه مانند یک فیلتر عمل می‌کند و نمی‌گذارد ترکیب‌های تصادفی و کم‌اهمیت وارد تحلیل شوند.

۲. مجموعه‌های بسته (Closed Itemsets): حالا فرض کنید مجموعهٔ {لپ‌تاپ، کیف} در ۱۰۰ تراکنش دیده شده، و مجموعهٔ بزرگ‌تر {لپ‌تاپ، کیف، ماوس} هم دقیقاً در همان ۱۰۰ تراکنش دیده شده باشد. در این حالت، نگه‌داشتن هر دو مجموعه کار زائدی است، مجموعهٔ بسته دقیقاً به همین نکته اشاره دارد: یک مجموعهٔ پرتکرار را «بسته» می‌نامیم که هیچ مجموعهٔ بزرگ‌تری از آن، همان میزان پشتیبانی را نداشته باشد. این مفهوم کمک می‌کند بدون ازدست‌دادن اطلاعات، فقط مجموعه‌های اصلی را نگه داریم و حجم محاسبات را کم کنیم.

۳. قواعد ارتباطی (Association Rules): یک قاعدهٔ ارتباطی می‌گوید: «اگر مشتری X را بخرد، به احتمال زیاد Y را هم می‌خرد». برای مثال، از مجموعهٔ پرتکرار {لپ‌تاپ، کیف} می‌توان قاعدهٔ لپ‌تاپ ← کیف را ساخت. اما پرتکرار بودن کافی

نیست؛ باید مطمئن شویم این رابطه واقعاً معنادار است. به همین دلیل، هر قاعده با معیاری به نام **اطمینان**

(**Confidence**) سنجیده می‌شود که می‌گوید چند درصد خریداران لپ‌تاپ، کیف هم خریده‌اند.

نکته کلیدی

Support می‌گوید «این ترکیب چقدر رایج است؟» اما Confidence می‌گوید «اگر X را خریدم، چقدر احتمال دارد Y را هم بخرم؟». قاعده‌ای با Support پایین می‌تواند Confidence بالایی داشته باشد. به همین دلیل، برای ارزیابی درست، هر دو معیار را همزمان در نظر بگیرید و به‌تنهایی به یکی از آن‌ها اعتماد نکنید.

ریاضیات – پشتیبانی (Support) و اطمینان (Confidence)

یادآوری پیش‌نیاز – احتمال شرطی (Conditional Probability)

پیش از سراغ‌رفتن به اطمینان (Confidence)، باید احتمال شرطی را مرور کنیم: احتمال وقوع رویداد B وقتی می‌دانیم رویداد A رخ داده است.

$$P(B | A) = \frac{P(A \cap B)}{P(A)}$$

نمادها: $P(A \cap B)$ احتمال هم‌زمانی دو رویداد؛ $P(A)$ احتمال رویداد شرط.

مثال: در یک کلاس ۲۰ نفره، ۱۲ نفر پسر و از این ۱۲ نفر، ۵ نفر عینکی‌اند. احتمال عینکی‌بودن یک پسر تصادفی: $5/12$.

برای یک قاعدهٔ ارتباطی (association rule) به‌صورت $A \Rightarrow B$ ، دو معیار پایه تعریف می‌شود: **پشتیبانی (Support)** نشان می‌دهد چه کسری از تراکنش‌ها هم A و هم B را دارند، و **اطمینان (Confidence)** نشان می‌دهد از میان تراکنش‌هایی که A را دارند، چه کسری B را هم دارند.

(4.2)

پشتیبانی قاعده

$$\text{support}(A \Rightarrow B) = P(A \cup B)$$

(4.3)

اطمینان قاعده

$$\text{confidence}(A \Rightarrow B) = P(B | A)$$

نمادهای فرمول

- A, B: دو مجموعه‌آیتم (Itemset) که با هم شرط قاعده را می‌سازند
- $P(A \cup B)$: احتمال اینکه یک تراکنش، هم همهٔ آیتم‌های A و هم همهٔ آیتم‌های B را داشته باشد
- $P(B | A)$: احتمال شرطی وقوع B، به شرط اینکه A رخ داده باشد

مثال ۱ (ساده) – محاسبهٔ مستقیم پشتیبانی و اطمینان

صورت سؤال: در یک فروشگاه، از ۱۰۰۰ تراکنش، ۲۰ تراکنش هم «کامپیوتر» و هم «آنتی‌ویروس» دارند؛ ۴۰ تراکنش «کامپیوتر» دارند. پشتیبانی و اطمینان قاعدهٔ کامپیوتر $\Rightarrow$ آنتی‌ویروس را بیابید.

$$\text{support} = \frac{20}{1000} = 2\% , \quad \text{confidence} = \frac{20}{40} = 50\%$$

نتیجه: پشتیبانی ۲٪ و اطمینان ۵۰٪ است – یعنی از هر ۲ خریدار کامپیوتر، ۱ نفر آنتی‌ویروس هم می‌خرد.

✚ مثال ۲ (متوسط) – قاعدهٔ دوطرفه

صورت سؤال: با همان داده‌ها، اگر ۷۵ تراکنش «آنتی‌ویروس» داشته باشند، اطمینان قاعدهٔ معکوس (آنتی‌ویروس $\Rightarrow$ کامپیوتر) را بیابید.

$$\text{confidence} = \frac{20}{75} \approx 26.7\%$$

نتیجه: اطمینان جهت‌دار است: $A \Rightarrow B$ و $B \Rightarrow A$ معمولاً مقدار متفاوتی دارند، هرچند پشتیبانی یکسان است.

✚ مثال ۳ (سخت) – استخراج معکوس از روی اطلاعات ناقص

صورت سؤال: می‌دانیم $\text{support}(A \Rightarrow B) = 15\%$ و $\text{confidence}(A \Rightarrow B) = 30\%$ ، با ۲۰۰۰ تراکنش. $\text{support_count}(A)$ را با معکوس‌کردن رابطهٔ اطمینان بیابید.

$$\text{support_count}(A \cup B) = 0.15 \times 2000 = 300 , \quad \text{support_count}(A) = \frac{300}{0.30} = 1000$$

نتیجه: ۱۰۰۰ تراکنش شامل A هستند؛ از میان آن‌ها فقط ۳۰۰ تا (۳۰٪) شامل B هم می‌شوند.

▢ ریاضیات – اطمینان بر حسب شمارش پشتیبانی (Support Count)

در عمل، اطمینان را مستقیماً از شمارش‌های خام (Support Count) به‌دست می‌آوریم؛ این فرمول پایهٔ تولید همهٔ قواعد ارتباطی در الگوریتم‌های کاوش است.

(4.4)

اطمینان از روی شمارش تراکنش‌ها

$$\text{confidence}(A \Rightarrow B) = \frac{\text{support_count}(A \cup B)}{\text{support_count}(A)}$$

✚ نمادهای فرمول

- $\text{support_count}(A \cup B)$: تعداد تراکنش‌هایی که هم A و هم B را دارند
- $\text{support_count}(A)$: تعداد تراکنش‌هایی که A را دارند

✚ مثال ۱ (ساده) – یک زیرمجموعهٔ تک‌عضوی

صورت سؤال: برای مجموعه‌آیتم پرتکرار $X=\{I1,I2,I5\}$ با $\text{support_count}=2$ ، اگر $\text{support_count}(\{I1,I2\})=4$ باشد، اطمینان قاعده $I5 \Rightarrow \{I1,I2\}$ را بیابید.

$$\text{confidence} = \frac{2}{4} = 50\%$$

نتیجه: با آستانه $\text{min_conf}=70\%$ ، این قاعده رد می‌شود چون 50% کافی نیست.

✖ مثال ۲ (متوسط) – مقایسه دو قاعده از یک Itemset

صورت سؤال: برای همان X با $\text{support_count}=2$ ، داریم $\text{support_count}(I1)=6$ و $\text{support_count}(\{I1,I5\})=2$. اطمینان $I1 \Rightarrow \{I2,I5\}$ و $I2 \Rightarrow \{I1,I5\}$ را مقایسه کنید.

$$\text{conf}(I1 \Rightarrow \{I2,I5\}) = \frac{2}{6} = 33\%, \quad \text{conf}(\{I1,I5\} \Rightarrow I2) = \frac{2}{2} = 100\%$$

نتیجه: با $\text{min_conf}=70\%$ ، فقط قاعده دوم به‌عنوان قاعده قوی باقی می‌ماند.

✖ مثال ۳ (سخت) – تولید تمام قواعد ممکن از یک Itemset

صورت سؤال: برای $X=\{I1,I2,I5\}$ با $\text{support_count}=2$ ، $\text{support_count}(I5)=2$ ، $\text{support_count}(I2)=7$ ، $\text{support_count}(\{I2,I5\})=2$ ، قاعده $I5 \Rightarrow \{I1,I2\}$ را بررسی کنید.

$$\text{conf}(I5 \Rightarrow \{I1,I2\}) = \frac{2}{2} = 100\%$$

نتیجه: از میان ۶ قاعده ممکن، فقط آن‌هایی که کسرشان حداقل 70% شود قوی هستند – این یکی از سه قاعده قوی (Strong Rule) است.

مثال عملی: استخراج مجموعه‌های پرتکرار، بسته‌شده و قواعد خرید در «دیجی‌کالا»

سیستم داده‌کاوی «دیجی‌کالا» برای تحلیل سبدهای خرید، میلیون‌ها سفارش را بررسی می‌کند. اگر سیستم بخواهد تمام ترکیب‌های ممکن از کالاها را ذخیره کند، با انفجار حجم داده‌ها مواجه می‌شود. برای جلوگیری از این مشکل، سه مفهوم کلیدی استفاده می‌شوند:

- مجموعه‌های پرتکرار (Frequent Itemsets):

ترکیب‌هایی از کالاها که میزان تکرار خرید آن‌ها در سبدها بیشتر یا مساوی حد آستانه حداقل پشتیبانی (Minimum Support) باشد.

$$\text{Support}(X) = \frac{\text{Count}(X)}{N} \geq \text{MinSupport}$$

۲. مجموعه‌های بسته (Closed Itemsets) – فشرده‌سازی الگوها بدون افت اطلاعات:

یک مجموعه پرتکرار مانند X را **بسته (Closed)** می‌نامند اگر هیچ ابرمجموعه مستقیم از آن (مانند Y که شامل X باشد) دارای همان میزان پشتیبانی و تکرار نباشد. مجموعه‌های بسته به سیستم دیجی‌کالا اجازه می‌دهند الگوهای تکراری اضافه را حذف کرده و فشرده‌ترین نمایش داده‌ها را ذخیره کند.

$$\text{IsClosed}(X) \Leftrightarrow \nexists Y \supset X : \text{Support}(Y) = \text{Support}(X)$$

مثال ملموس مجموعه بسته در دیجی‌کالا:

اگر ترکیب {گوشی، قاب} در ۴ سبد خرید حضور داشته باشد و ترکیب بزرگ‌تر {گوشی، قاب، گلس} هم دقیقاً در همان ۴ سبد خرید

حضور داشته باشد، ترکیب اول بسته نیست؛ زیرا افزودن «گلس» هیچ تکراری را کم نکرده است. بنابراین تنها ترکیب بزرگ‌تر ذخیره

می‌شود.

۳. قواعد ارتباطی (Association Rules):

استخراج روابط شرطی به فرم «اگر X خریداری شود، به احتمال زیاد Y هم خریداری می‌شود» که بر اساس میزان اطمینان (Confidence)

ارزیابی می‌گردند.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

نتیجه: فشرده‌سازی الگوها با مجموعه‌های بسته، الگوریتم‌های پیشنهاد دیجی‌کالا را سبک‌تر و سریع‌تر می‌سازد.

خروجی: جداسازی مجموعه‌های بسته از مجموعه‌های پرتکرار عادی و استخراج قواعد خرید

In [3]:

```
display_academic_text(
    "📦 داده‌کاوی در دیجی‌کالا: استخراج مجموعه‌های پرتکرار، بسته‌شده و قواعد خرید",
    "main_title"
)

display_academic_text(
    ".سیستم داده‌کاوی «دیجی‌کالا» اگر بخواهد تمام ترکیب‌های ممکن از کالاها را در رم ذخیره کند، با انفجار حجم داده‌ها مواجه می‌شود".
    "<b>مجموعه‌های بسته</b> (Closed Itemsets) برای جلوگیری از این مشکل و فشرده‌سازی اطلاعاتِ استخراج‌شده، از مفهوم",
    "paragraph"
)

display_academic_text("۱. مجموعه‌های پرتکرار (Frequent Itemsets):", "sub_title")
display_academic_text(
    "<i>(MinSupport) حداقل پشتیبانی</i> <b>ترکیب‌هایی از کالاها که میزان تکرار خرید آن‌ها مساوی یا بیشتر از",
    "paragraph"
)

display_academic_text(
    r"$\text{Support}(X) = \frac{\text{Count}(X)}{N} \ge \text{MinSupport}$",
    "paragraph"
)

display_academic_text("۲. فشرده‌سازی بدون افت داده — (Closed Itemsets) مجموعه‌های بسته", "sub_title")
display_academic_text(
    ". از آن وجود نداشته باشد که دقیقاً همان مقدار پشتیبانی را داشته باشد (Superset) است که هیچ آبرمجموعه‌ای (Closed) بسته <b>زمانی $X$ یک مجموعه مانند",
    "<i>افزونه</i> (Redundant) <b>یک قلم $X$ اگر چنین ابرمجموعه‌ای وجود داشته باشد، مجموعهٔ",
    "paragraph"
)

display_academic_text(
    r"$\text{IsClosed}(X) \text{ iff } \nexists Y \supset X : \text{Support}(Y) = \text{Support}(X)$",
    "paragraph"
)

# =====
# 3. SECTION 2: DATA PREPARATION & ONE-HOT ENCODING
# =====
display_academic_text("مرحله نخست: آماده‌سازی داده‌ها برای الگوریتم (مهندسی افزونگی عمدی)", "sub_title")

# Educational Step 1: Raw Transaction Data
# Notice how ['Mobile', 'Screen_Protector', 'Case'] is repeated 4 times.
# This forces the strict subset ['Mobile', 'Screen_Protector'] to have the
# EXACT SAME support, making the subset "Not Closed" (redundant).
transactions = [
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector', 'Case'],
    ['Mobile', 'Charger'],
    ['Mobile', 'Charger'],
    ['Screen_Protector', 'Case'],
    ['Screen_Protector', 'Case'],
    ['Mobile', 'Screen_Protector'],
    ['Mobile', 'Screen_Protector'],
]

# One-Hot Encoding via mlxtend
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_baskets = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_baskets.astype(int).head(5)],
    ["آماده برای کشف الگوها (One-Hot Encoding ماتریس)"]
)

# =====
# 4. SECTION 3: EXECUTING APRIORI & FINDING CLOSED ITEMSETS
```

```
# =====
# Extract Frequent Itemsets (Min Support = 20%)
frequent_itemsets = apriori(df_baskets, min_support=0.20, use_colnames=True)

# Custom Function for Closed Itemsets
# For every itemset 'X', we check if there's any 'Y' such that:
# 1. X is a strict subset of Y.
# 2. X and Y have identical support values.
# If both are true, X is redundant and NOT Closed.
def check_is_closed(row, df):
    itemset = row['itemsets']
    sup = row['support']

    for _, other_row in df.iterrows():
        other_itemset = other_row['itemsets']
        # The '<' operator on python frozensets means "is strict subset"
        if itemset < other_itemset and other_row['support'] == sup:
            return False
    return True

frequent_itemsets['is_closed'] = frequent_itemsets.apply(
    lambda row: check_is_closed(row, frequent_itemsets), axis=1
)

# Convert frozensets to strings for UI
frequent_itemsets['itemset_str'] = frequent_itemsets['itemsets'].apply(
    lambda x: ', '.join(list(x))
)

df_analysis_display = frequent_itemsets[
    ['itemset_str', 'support', 'is_closed']
].sort_values(by=['support', 'is_closed'], ascending=[False, False]).reset_index(drop=True)

display_side_by_side_datasets(
    [df_analysis_display],
    ["تحلیل مجموعه‌های پرتکرار (تشخیص مجموعه‌های بسته/غیربسته)"]
)

# =====
# 5. SECTION 4: GENERATE ASSOCIATION RULES
# =====
# Extract Association Rules (Min Confidence = 50%)
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.50
)
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['rule_label'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

# =====
# 6. SECTION 5: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("مصورسازی تحلیلی: فیلتر کردن نویزها و اعتبارسنجی قواعد", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 7), dpi=135)

# -----
# Subplot 1: Frequent vs Closed Itemsets Support Comparison
# -----
# Turquoise for Closed (Valuable), Red for Not Closed (Redundant)
colors = ['#1abc9c' if closed else '#e74c3c' for closed in df_analysis_display['is_closed']]

axes[0].barh(
    df_analysis_display['itemset_str'],
    df_analysis_display['support'],
    color=colors,
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.6
)

axes[0].set_title('Figure 1: Frequent Itemsets (Turquoise = Closed, Red = Redundant)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Support Value (Frequency %)', fontsize=13, fontweight='bold')
axes[0].grid(axis='x', linestyle='--', alpha=0.5)
axes[0].invert_yaxis() # Highest support at the top
axes[0].set_yticklabels(df_analysis_display['itemset_str'], fontweight='bold', fontsize=11)

# Value annotations inside bars
for i, row in df_analysis_display.iterrows():
    status = 'Closed' if row['is_closed'] else 'Not Closed'
    text_color = '#2c3e50' if row['is_closed'] else 'white'
    offset = -0.15 if not row['is_closed'] else 0.02 # Put text inside red bars to fit

    axes[0].text(
        row['support'] + offset,
        i,
        f'{row["support"]:.2f} ({status})',
        va='center',
        fontweight='bold',
        fontsize=11,
        color=text_color
    )

# -----
# Subplot 2: Association Rules Confidence vs Lift
# -----
scatter = axes[1].scatter(
    rules['confidence'],
    rules['lift'],
    s=rules['support'] * 1500, # Bubble size = Support
```

```
c=rules['support'],
cmap='Blues',          # Academic Blue Palette
edgecolor='#2c3e50',
alpha=0.9,
linewidth=1.5
)

axes[1].set_title('Figure 2: Association Rules: Confidence vs Lift\n(Bubble Size/Darkness = Support Level)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Confidence Ratio', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Lift Ratio (Power of Association)', fontsize=13, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.5)
axes[1].set_xlim(0.45, 1.1)

cbar = fig.colorbar(scatter, ax=axes[1], shrink=0.8)
cbar.set_label('Support Level (Frequency)', fontweight='bold', fontsize=11)

# Clean Annotation Strategy (Legend Box)
strong_rules = rules[rules['lift'] > 1.0].reset_index(drop=True)
legend_texts = []

for i, row in strong_rules.iterrows():
    rule_id = f"R{i+1}"
    legend_texts.append(f"{rule_id}: {row['rule_label']}")

# Place just the ID (R1, R2) next to the bubble
axes[1].annotate(
    rule_id,
    xy=(row['confidence'], row['lift']),
    xytext=(10, 10), # Offset in points
    textcoords='offset points',
    fontsize=11,
    fontweight='bold',
    color='#c0392b'
)

# Adjust spacing so the legend box doesn't get cut off
plt.subplots_adjust(wspace=0.3, right=0.80)

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.show()

# =====
# 7. SECTION 6: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک در داده‌کاوی (Strategic Implication):</b><br>"
    r"است (Big Data) یک ضرورت قطعی در محیط‌های کلان‌داده <b>(Closed Itemsets)</b> مجموعه‌های بسته»<br>"
    r"غیربسته و افزونه (قرمز) است، زیرا پشتیبانی آن ($۰.۶۰) دقیقاً با <code>(Mobile, Screen_Protector)</code> مشاهده می‌کنیم که ترکیبِ دوگانهٔ (Figure 1) در نمودار ۱"
    r"<br>!را نخر (Case) که کسی موبایل و گلس بخرد، اما قاب <i>هرگز</i> پیش نیامده> برابر است. این یعنی <code>(Mobile, Screen_Protector, Case)</code> پشتیبانی ترکیب سه‌گانه و بزرگ‌تر"
    r".لازم برای پردازشِ قوانین را به‌شدت کاهش داده و مانع از پیشنهاد دادنِ اطلاعاتِ بدیهی و تکراری به کاربر می‌گردد. RAM حجم حافظهٔ (Redundant) سیستم پیشنهادگرِ دیجی‌کالا با شناسایی و حذف این الگوهای افزونه"
)
display_academic_text(insight_conclusion, "insight")
```

داده‌کاوی در دیجی‌کالا: استخراج مجموعه‌های پرتکرار، بسته‌شده و قواعد خرید

سیستم داده‌کاوی «دیجی‌کالا» اگر بخواهد تمام ترکیب‌های ممکن از کالاها را در رم ذخیره کند، با انفجار حجم داده‌ها مواجه می‌شود. برای جلوگیری از این مشکل و فشرده‌سازی اطلاعاتِ استخراج‌شده، از مفهومِ **مجموعه‌های بسته (Closed Itemsets)** استفاده می‌کند.

۱. مجموعه‌های پرتکرار (Frequent Itemsets):

ترکیب‌هایی از کالاها که میزان تکرار خرید آن‌ها مساوی یا بیشتر از *حداقل پشتیبانی (MinSupport)* باشد.

$$\text{Support}(X) = \frac{\text{Count}(X)}{N} \geq \text{MinSupport}$$

۲. مجموعه‌های بسته (Closed Itemsets) — فشرده‌سازی بدون افت داده:

یک مجموعه مانند X زمانی **بسته (Closed)** است که هیچ اَبرمجموعه‌ای (Superset) از آن وجود نداشته باشد که دقیقاً همان مقدار پشتیبانی را داشته باشد. اگر چنین ابرمجموعه‌ای وجود داشته باشد، مجموعهٔ X یک *قلم افزونه (Redundant)* محسوب شده و هرس می‌شود.

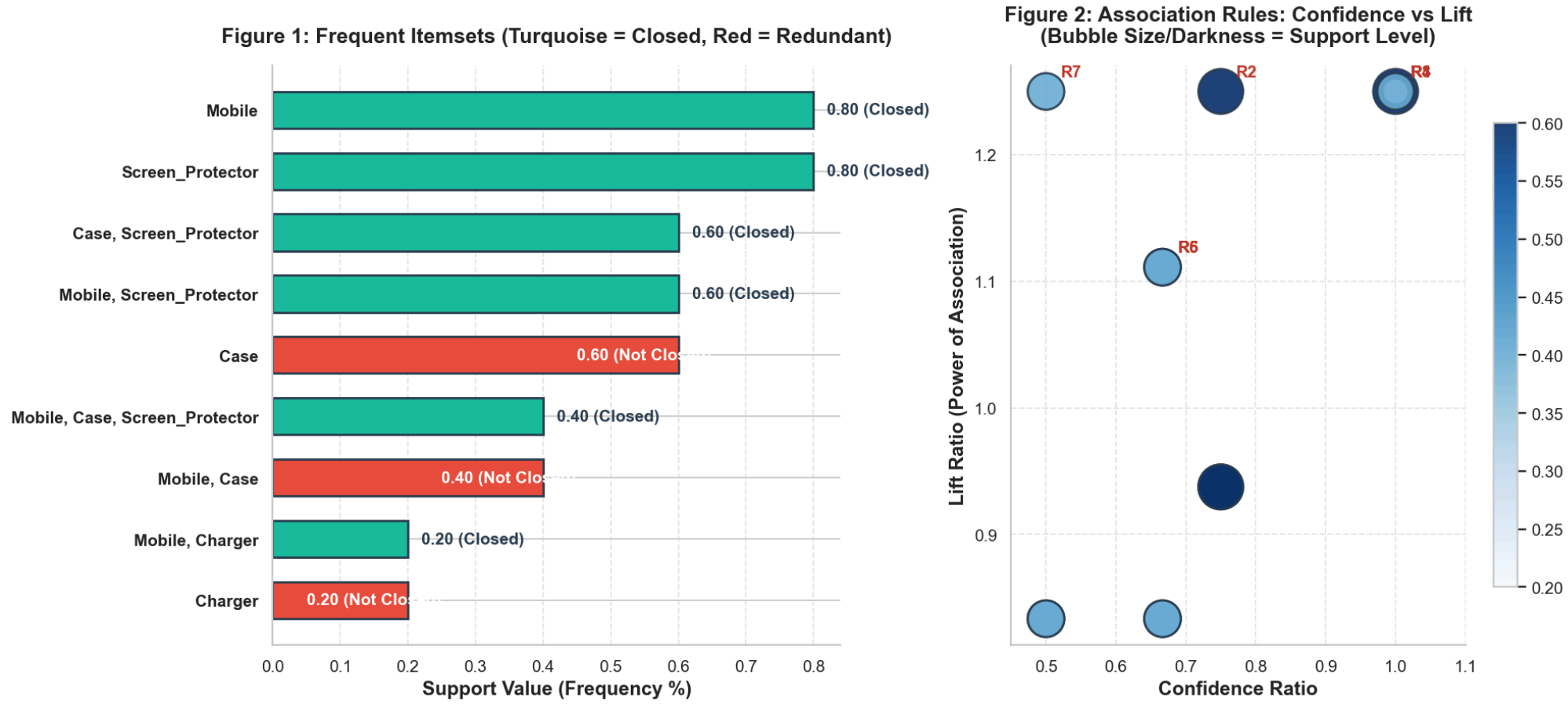
$$\text{IsClosed}(X) \iff \nexists Y \supset X : \text{Support}(Y) = \text{Support}(X)$$

مرحله نخست: آماده‌سازی داده‌ها برای الگوریتم (مهندسی افزونگی عمدی)

ماتریس One-Hot Encoding (آماده برای کشف الگوها)			
Case	Charger	Mobile	Screen_Protector
1	0	1	1
1	0	1	1
1	0	1	1
1	0	1	1
0	1	1	0

تحلیل مجموعه‌های پرتکرار (تشخیص مجموعه‌های بسته/غیربسته)		
itemset_str	support	is_closed
Mobile	0.8	True
Screen_Protector	0.8	True
Case, Screen_Protector	0.6	True
Mobile, Screen_Protector	0.6	True
Case	0.6	False
Mobile, Case, Screen_Protector	0.4	True
Mobile, Case	0.4	False
Mobile, Charger	0.2	True
Charger	0.2	False

مصورسازی تحلیلی: فیلتر کردن نویزها و اعتبارسنجی قواعد



نتیجه استراتژیک در داده‌کاوی (Strategic Implication):

فشرده‌سازی با استفاده از «مجموعه‌های بسته (Closed Itemsets)» یک ضرورت قطعی در محیط‌های کلان‌داده (Big Data) است.

در نمودار ۱ (Figure 1) مشاهده می‌کنیم که ترکیب دوگانه (Mobile, Screen_Protector) غیربسته و افزونه (قرمز) است، زیرا پشتیبانی آن (۰.۶۰) دقیقاً با پشتیبانی

ترکیب سه گانه و بزرگ تر (Mobile, Screen_Protector, Case) برابر است. این یعنی هرگز پیش نیامده که کسی موبایل و گلس بخرد، اما قاب (Case) را نخرد!

سیستم پیشنهادگر دیجی‌کالا با شناسایی و حذف این الگوهای افزونه (Redundant)، حجم حافظه RAM لازم برای پردازش قوانین را به شدت کاهش داده و مانع از پیشنهاد دادن اطلاعات بدیهی و تکراری به کاربر می‌گردد.

 توضیح مثال کد پایتون

در این مثال:

- ۱۰ سبد خرید نمونه از سفارش‌های دیجی‌کالا تعریف شد و با ماتریس One-Hot تبدیل به داده قابل پردازش گردید.
- شناسایی مجموعه‌های پرتکرار: الگوریتم apriori مجموعه‌های با حد پشتیبانی بالای ۲۰٪ را استخراج نمود.
- پیاده‌سازی منطق مجموعه‌های بسته (Closed Itemsets): با تابع سفارشی check_is_closed بررسی شد که آیا ابرمجموعه‌ای با همان میزان پشتیبانی وجود دارد یا خیر. اگر ابرمجموعه‌ای با همان پشتیبانی پیدا شد، آن مجموعه غیربسته (Not Closed) علامت‌گذاری گردید.
- تولید قواعد ارتباطی: قواعد خرید شرطی با حد اطمینان بالای ۵۰٪ استخراج شدند.
- تفسیر نمودارهای Matplotlib:
- نمودار سمت چپ (Bar Chart): مجموعه‌های بسته با رنگ سبز و مجموعه‌های غیربسته با رنگ قرمز تفکیک شدند تا فشرده‌سازی الگوها به صورت بصری مشخص شود.
- نمودار سمت راست (Scatter Plot): میزان اطمینان و لیفت قواعد ارتباطی به همراه اندازه پشتیبانی آن‌ها را رسم کرده است.

4.2 Frequent itemset mining methods

4.2.1 Apriori algorithm: finding frequent itemsets by confined candidate generation



ریاضیات – انفجار ترکیبیاتی زیرمجموعه‌های پرتکرار

یادآوری پیش نیاز – ترکیب (Combination) و مثلث پاسکال

برای شمردن اینکه از یک مجموعه n عضوی، چند زیرمجموعه k عضوی می‌توان ساخت (بدون توجه به ترتیب)، از فرمول ترکیب استفاده می‌کنیم.

$$\binom{n}{k} = \frac{n!}{k! (n - k)!}$$

نمادها: $n!$ فاکتوریل، n (حاصل ضرب ۱ تا n)؛ k اندازه زیرمجموعه.

مثال: تعداد راه‌های انتخاب ۲ کالا از ۵ کالای متفاوت: $\binom{5}{2} = 10$.

اگر یک مجموعه‌آیتم (Itemset) طولانی پرتکرار باشد، تمام زیرمجموعه‌های ناتهی آن نیز پرتکرارند (طبق خاصیت اپریوری/Apriori). این یعنی تعداد مجموعه‌آیتم‌های پرتکراری که باید ذخیره شوند می‌تواند به‌طرز نمایی بزرگ شود.

(4.5)

تعداد کل زیرمجموعه‌های ناتهی یک ۱۰۰ Itemset عضوی

$$\sum_{k=1}^{100} \binom{100}{k} = 2^{100} - 1 \approx 1.27 \times 10^{30}$$

📌 نمادهای فرمول

- $\binom{100}{k}$: تعداد زیرمجموعه‌های k عضوی از یک مجموعه ۱۰۰ عضوی
- $2^{100} - 1$: تعداد کل زیرمجموعه‌های ناتهی

✂️ مثال ۱ (ساده) – شمارش زیرمجموعه‌های یک Itemset کوچک

صورت سؤال: یک مجموعه‌آیتم پرتکرار ۳ عضوی $\{a1, a2, a3\}$ داریم. چند زیرمجموعه ناتهی پرتکرار از آن مشتق می‌شود؟

$$2^3 - 1 = 7$$

نتیجه: ۷ زیرمجموعه ناتهی: سه تک‌عضوی، سه دو‌عضوی، و خود مجموعه سه‌عضوی.

✂️ مثال ۲ (متوسط) – شمارش Itemset‌های k عضوی مشخص

صورت سؤال: در یک مجموعه‌آیتم پرتکرار ۲۰ عضوی، چند مجموعه‌آیتم پرتکرار ۳ عضوی درون آن وجود دارد؟

$$\binom{20}{3} = \frac{20 \times 19 \times 18}{3 \times 2 \times 1} = 1140$$

نتیجه: فقط برای مجموعه‌آیتم‌های ۳ عضوی، ۱۱۴۰ زیرمجموعه پرتکرار باید ثبت شوند.

✂️ مثال ۳ (سخت) – مقایسه رشد نمایی برای دو طول متفاوت

صورت سؤال: نسبت تعداد کل زیرمجموعه‌های ناتهی یک ۱۰۰ Itemset عضوی به یک ۵۰ Itemset عضوی را تخمین بزنید.

$$\frac{2^{100} - 1}{2^{50} - 1} \approx 2^{50} \approx 1.13 \times 10^{15}$$

نتیجه: افزایش طول Itemset از ۵۰ به ۱۰۰ عضو، تعداد زیرمجموعه‌ها را حدود 10^{15} برابر می‌کند – دلیل اصلی نیاز به مفاهیم Maximal Itemset و Closed.

شناسایی مجموعه‌های پرتکرار، پایه‌ی کشف قواعد ارتباطی است. اما بررسی همه‌ی زیرمجموعه‌های ممکن یک مجموعه‌داده از نظر محاسباتی عملی نیست.

الگوریتم **Apriori** این مشکل را با تکیه بر اصلی ساده حل می‌کند: **خاصیت Apriori** می‌گوید اگر مجموعه‌ای پرتکرار نباشد، هیچ ابرمجموعه‌ای از آن نیز نمی‌تواند پرتکرار باشد. به بیان دیگر، هر زیرمجموعه‌ی یک مجموعه‌ی پرتکرار، خود نیز باید پرتکرار باشد. این قاعده، فضای جست‌وجو را به شدت کاهش می‌دهد.

بر اساس همین اصل، Apriori مجموعه‌های پرتکرار را لایه‌به‌لایه می‌سازد. ابتدا مجموعه‌های تک‌عضوی پرتکرار را می‌یابد، سپس از دل آن‌ها مجموعه‌های دو‌عضوی را می‌سازد، و به همین ترتیب پیش می‌رود. این فرایند در هر گام چهار مرحله دارد:

۱. تولید کاندیدا (Candidate Generation): از ترکیب مجموعه‌های پرتکرار گام قبل، مجموعه‌های جدیدی با یک عضو بیشتر ساخته می‌شوند. این مجموعه‌های ساخته شده «کاندیدا» نام دارند، زیرا هنوز پرتکرار بودنشان قطعی نیست.

۲. هرس کردن (Pruning): پیش از شمارش پشتیبانی، اگر حتی یکی از زیرمجموعه‌های یک کاندیدا در گام قبل پرتکرار نبوده باشد، آن کاندیدا فوراً حذف می‌شود. اینجا همان نقطه‌ای است که Apriori بیشترین صرفه‌جویی محاسباتی را انجام می‌دهد.

۳. شمارش پشتیبانی (Support Counting): کاندیداهای باقی‌مانده در برابر تراکنش‌های واقعی سنجیده می‌شوند. هر کاندیدا که پشتیبانی آن از **حداقل پشتیبانی** فراتر رود، به عنوان مجموعه‌ی پرتکرار جدید پذیرفته می‌شود.

۴. تکرار و شرط توقف: مراحل ۱ تا ۳ بارها تکرار می‌شوند و در هر دور، اندازه‌ی مجموعه‌ها یک واحد بزرگ‌تر می‌شود. این چرخه تا زمانی ادامه می‌یابد که هیچ مجموعه‌ی پرتکرار جدیدی یافت نشود.

آنچه Apriori را کارآمد می‌کند، ترکیب هوشمندانه‌ی «تولید محدود» و «هرس زود هنگام» است. تنها کاندیداهایی که از فیلتر خاصیت Apriori عبور کنند، وارد مرحله‌ی پرهزینه‌ی شمارش پشتیبانی می‌شوند. به این ترتیب، هزینه‌ی محاسباتی به طور قابل‌توجهی کاهش می‌یابد.

فرض کنید پایگاه‌داده‌ای شامل **۱۰۰,۰۰۰ عدد کالای** متمایز داریم. در چنین شرایطی، Apriori ممکن است ناچار شود میلیون‌ها کاندیدا تولید کند و برای محاسبه‌ی پشتیبانی هر یک، بارها و بارها کل پایگاه‌داده را پیمایش نماید؛ فرآیندی که حتی بر روی سرورهای قدرتمند نیز می‌تواند **ساعت‌ها** به طول انجامد. به همین دلیل، در پروژه‌های واقعی، به ندرت از پیاده‌سازی استاندارد Apriori استفاده می‌شود. به جای آن، یا از نسخه‌های بهبودیافته با **هرس پیشرفته** (که در بخش ۴.۲.۳ به آن پرداخته می‌شود) بهره می‌گیرند، یا مستقیماً به سراغ روش‌های کارآمدتری مانند **رشد الگو (FP-Growth)** می‌روند. بنابراین، پیش از اجرای الگوریتم، توصیه می‌شود **حجم و پراکندگی** داده‌ها را بسنجید و متناسب با آن، روش مناسب را انتخاب کنید.

ریاضیات – خاصیت اپریوری (Apriori Property) و شرط پیوند (Join)

خاصیت اپریوری می‌گوید: **همه‌ی زیرمجموعه‌های ناتهی یک مجموعه‌آیتم پرتکرار، خودشان هم پرتکرارند.** این خاصیت یکنوایی نزولی (Antimonotonicity) دارد و اجازه می‌دهد پیش از شمارش واقعی، کاندیدهای بی‌فایده حذف شوند. برای ساخت کاندیدهای k -عضوی از مجموعه‌آیتم‌های $(k-1)$ -عضوی، دو عضو مرتب‌شده در صورتی پیوند می‌خورند که $k-2$ عضو اول‌شان یکسان و فقط آخرین عضو متفاوت باشد.

$$\big(L_1[1] \cup L_2[1]\big) \wedge \cdots \wedge \big(L_1[k-2] \cup L_2[k-2]\big) \wedge \big(L_1[k-1] \cup L_2[k-1]\big)$$

(4.7)

تعداد کاندیدهای ۲-عضوی از L1

$$|C_2| = \binom{|L_1|}{2}$$

✦ نمادهای فرمول

- $[i]_1$: ج. امین عضو مجموعه آیتم مرتب شده i_1 (بر اساس ترتیب لغتنامه‌ای)
- k : طول مجموعه آیتم کاندیدی که می‌سازیم
- $|L_1|$: تعداد آیتم‌های پرتکرار تکی

✦ مثال ۱ (ساده) – شمارش کاندیدهای ۲-عضوی

صورت سؤال: اگر L_1 شامل ۵ آیتم پرتکرار باشد، چند کاندید ۲-عضوی در C_2 تولید می‌شود؟

$$|C_2| = \binom{5}{2} = 10$$

نتیجه: ۱۰ جفت کاندید باید در پاس بعدی شمارش شوند.

✦ مثال ۲ (متوسط) – بررسی شرط پیوند برای دو ۲-Itemset

صورت سؤال: آیا $\{1,2\}$ و $\{1,3\}$ (مرتب لغتنامه‌ای) با هم پیوند می‌خورند؟ نتیجه را بنویسید.

$$L_1[1] = L_2[1] = 1, L_1[2] = 2$$

نتیجه: بله، چون عضو اول یکسان و عضو دوم متفاوت است، کاندید ۳-عضوی $\{1,2,3\}$ ساخته می‌شود.

✦ مثال ۳ (سخت) – هرس با خاصیت اپریوری

صورت سؤال: کاندید $\{1,3,5\}$ از پیوند $\{1,3\}$ و $\{1,5\}$ ساخته شده. اگر $L_2 \not\supset \{3,5\}$ باشد، آیا این کاندید باقی می‌ماند؟

$$\{3,5\} \notin L_2 \\ \Rightarrow \{1,3,5\} \notin C_3$$

نتیجه: خیر؛ چون یکی از زیرمجموعه‌های ۲-عضوی این کاندید پرتکرار نیست، کل کاندید بدون اسکن دیتابیس هرس و حذف می‌شود.

📖 تاریخچه فرمول – الگوریتم اپریوری (Apriori)

الگوریتم اپریوری را رکین آگراوال (R. Agrawal) و رامکریشن سربیکانت (R. Srikant) در سال ۱۹۹۴ در آزمایشگاه تحقیقاتی IBM معرفی کردند. نام آن از این‌جا می‌آید که الگوریتم از «دانش پیشین» درباره خواص

مجموعه‌آیتم‌های پرتکرار استفاده می‌کند تا فضای جست‌وجو را هرس کند؛ همین یک ایده — که زیرمجموعهٔ یک الگوی پرتکرار هم باید پرتکرار باشد — به یکی از پراستنادترین الگوریتم‌های تاریخ داده‌کاوی و پایهٔ تحلیل سبد خرید در صنعت خرده‌فروشی تبدیل شد.

مثال عملی: بهینه‌سازی موتور پیشنهادی و هرس قواعد خرید در «اینستاکارت»

سامانه «اینستاکارت» (Instacart) به عنوان یکی از بزرگ‌ترین پلتفرم‌های سفارش آنلاین خواربار، روزانه با میلیون‌ها سبد خرید روبرو است. برای بهینه‌سازی موتور پیشنهادات، چیدمان انبار و ارائه‌ی بسته‌های تخفیفی، این سامانه نیازمند کشف الگوی خریدهای هم‌زمان مشتریان است. اگر سیستم بخواهد تمام ترکیبات ممکن کالاها را بررسی کند، با نرخ رشد انفجاری ترکیبات (Combinatorial Explosion) مواجه می‌شود. در این حالت، **الگوریتم Apriori** با استفاده از اصل هرس‌کردن وارد عمل می‌شود.

اصل کلیدی الگوریتم Apriori (ویژگی یکنوایی یا Anti-Monotonicity):

اگر یک مجموعه کالا (مثلاً موز و آووکادو) پرتکرار نباشد و حد آستانه پشتیبانی (Min Support) را کسب نکند، تمام مجموعه‌های بزرگ‌تر شامل آن (مانند موز و آووکادو و نان تست) نیز قطعاً پرتکرار نخواهند بود و نیازی به پردازش آن‌ها نیست:

$$\text{If } \text{Support}(X) < \text{min_support} \Rightarrow \forall Y : \text{Support}(X \cup Y) < \text{min_support}$$

مراحل اجرای الگوریتم Apriori در اینستاکارت:

۱. تولید کاندیدا (Candidate Generation):

ابتدا مجموعه‌های تک‌عضوی بررسی می‌شوند. سپس از ترکیب آیتم‌های پرتکرار مرحله قبل، کاندیداهای ۲ عضوی، ۳ عضوی و بزرگ‌تر ساخته می‌شوند.

۲. شمارش پشتیبانی (Support Counting):

تعداد حضور هر مجموعه کاندید در کل سبدهای خرید ثبت‌شده شمرده شده و درصد پشتیبانی آن محاسبه می‌شود.

۳. هرس کردن (Pruning):

مجموعه‌هایی که پشتیبانی آن‌ها کمتر از حد آستانه مشخص‌شده باشد، بلافاصله حذف می‌شوند و در مراحل بعدی شرکت داده نمی‌شوند.

۴. تکرار (Iteration):

این چرخه‌ی «تولید کاندید → شمارش → هرس» تا زمانی که دیگر هیچ مجموعه پرتکرار جدیدی شناسایی نشود ادامه می‌یابد.

نتیجه: اینستاکارت با کاهش فضای جست‌وجو توسط الگوریتم Apriori، به سرعت متوجه می‌شود مشتریانی که موز ارگانیک و توت‌فرنگی ارگانیک می‌خرند، به احتمال بسیار بالا ماست یونانی را هم سفارش می‌دهند و بر این اساس پیشنهادات اپلیکیشن را تنظیم می‌کند.

خروجی: صرفه‌جویی شدید در پردازش داده‌ها و شناسایی سریع‌ترین الگوهای خرید با هرس کردن شاخه‌های نامرتب

```
In [4]: display_academic_text(
    "Apriori بهینه‌سازی موتور پیشنهادی و هرس قواعد خرید در «اینستاکارت» با الگوریتم ",
    "main_title"
)

display_academic_text(
    "روزانه با میلیون‌ها سبد خرید و هزاران محصول روبه‌رو است (Instacart) «سامانهٔ تحویل خواربار «اینستاکارت» "
    "، از کالاها را برای یافتن الگوها بررسی کند <i></i>تمام ترکیبات ممکن</i> اگر سیستم بخواهد (Recommender System) برای بهینه‌سازی موتور پیشنهادات "
    "، با مکانیسم جادویی‌اش وارد عمل می‌شود <b>Apriori الگوریتم</b> مواجه می‌شود. اینجاست که <b>Combinatorial Explosion</b> رشد انفجاری ترکیبات<b> " یا "
    "paragraph"
)

display_academic_text("Anti-Monotonicity ویژگی یکنوایی نزولی (یا Apriori اصل کلیدی الگوریتم)", "sub_title")
display_academic_text(
    "، (کمتر باشد Min_Support از) اگر یک مجموعه کالا (مثلاً موز و آووکادو) پُرتکرار نباشد<i></i>: ریاضیات ثابت می‌کند که "
    "<br><i></i>«پُرتکرار نخواهند بود <b>قطعاً</b> نیز (مانند موز، آووکادو و نان تست) <b>تمام آب‌مجموعه‌های شامل آن</b>"
```

می‌کند (Prune) هرس ، پیش می‌رود و شاخه‌های ضعیف را بدون اینکه نیازی به محاسبه داشته باشند (Level-wise) الگوریتم با استفاده از این قانون، به صورت سطح-به-سطح "prune"،

)

```
display_academic_text(
    r"$$\text{If Support}(X) < \text{min\_support} \implies \text{forall } Y : \text{Support}(X \cup Y) < \text{min\_support}$$",
    "paragraph"
)
```

```
# =====
# 3. SECTION 2: DATA PREPARATION & ONE-HOT ENCODING
# =====
```

display_academic_text("مرحله نخست: تولید تراکنش‌ها و تبدیل به ماتریس باینری", "sub_title")

```
transactions = [
    ['Organic Bananas', 'Organic Strawberries', 'Greek Yogurt'],
    ['Organic Bananas', 'Avocado', 'Whole Milk'],
    ['Organic Strawberries', 'Greek Yogurt', 'Organic Spinach'],
    ['Organic Bananas', 'Organic Strawberries', 'Avocado', 'Greek Yogurt'],
    ['Organic Bananas', 'Whole Milk', 'Sourdough Bread'],
    ['Organic Strawberries', 'Greek Yogurt'],
    ['Organic Bananas', 'Avocado', 'Organic Strawberries'],
    ['Organic Bananas', 'Whole Milk', 'Greek Yogurt'],
    ['Organic Bananas', 'Organic Strawberries', 'Greek Yogurt', 'Sourdough Bread'],
    ['Avocado', 'Whole Milk'],
    ['Organic Bananas', 'Organic Strawberries', 'Greek Yogurt'],
    ['Organic Bananas', 'Whole Milk']
]
```

```
# Formatting raw transactions for display
df_raw_baskets = pd.DataFrame({
    'Order_ID': range(1, len(transactions) + 1),
    'Purchased_Items': [' '.join(basket) for basket in transactions]
})
```

```
# Boolean Matrix Transformation
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_instacart = pd.DataFrame(te_array, columns=te.columns_)
```

```
display_side_by_side_datasets(
    [df_raw_baskets.head(6).reset_index(drop=True), df_instacart.astype(int).head(6).reset_index(drop=True)],
    [
        "تراکنش‌های خام خواربار در سیستم اینستا کارت",
        "آماده پردازش برای One-Hot ماتریس"
    ]
)
```

```
# =====
# 4. SECTION 3: EXECUTING APRIORI & EXTRACTING RULES
# =====
display_academic_text("و تولید قواعد (Level-wise) مرحله دوم: جستجوی سطح-به-سطح", "sub_title")
```

```
# Apply Level-wise Search (Apriori Algorithm)
# Min Support = 0.25 (Itemset must appear in at least 3 out of 12 baskets)
frequent_itemsets = apriori(
    df_instacart,
    min_support=0.25,
    use_colnames=True
)
```

```
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets_sorted = frequent_itemsets.sort_values(by='support', ascending=False)
```

```
df_freq_display = frequent_itemsets_sorted.copy()
df_freq_display['itemsets'] = df_freq_display['itemsets'].apply(lambda x: ' '.join(list(x)))
```

```
# Extract Rules from Frequent Itemsets (Min Confidence = 0.60)
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.60
)
```

```
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ' '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ' '.join(list(x)))
rules['rule'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']
```

```
top_rules = rules[['rule', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False)
```

```
display_side_by_side_datasets(
    [df_freq_display.head(6).reset_index(drop=True), top_rules.head(6).reset_index(drop=True)],
    [
        "الگوهای پُر تکرار یافته شده (Min Support = 25%)",
        "قواعد طلایی استخراج شده (Min Confidence = 60%)"
    ]
)
```

```
# =====
# 5. SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text("Apriori در الگوریتم (Pruning) مصورسازی تحلیلی: اثبات فیزیکی هرس کردن", "sub_title")
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)
```

```
# -----
# Subplot 1: Frequent Itemsets Distribution by Size (Level-wise Pruning)
# -----
itemset_counts = frequent_itemsets['length'].value_counts().sort_index()
```

```
bars = axes[0].bar(
    [f'Size {k}\n(k-itemsets)' for k in itemset_counts.index],
    itemset_counts.values,
```

```
color='#2980b9', # Classic Blue
edgecolor='#2c3e50',
linewidth=2.0,
width=0.45
)

axes[0].annotate(
    "Apriori Pruning in Action:\nThe algorithm evaluated single items (Size 1)\nthen pairs (Size 2). By the time it reached\ntriplets (Size 3), so many branches had failed\nthe support test that onl
    xy=(2.0, itemset_counts.values[2]), xytext=(0.5, itemset_counts.values[0] * 0.7),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title('Figure 1: Apriori Level-wise Search (Anti-Monotonicity)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Retained Frequent Itemsets', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels([f'Size {k}\n(k-itemsets)' for k in itemset_counts.index], fontweight='bold', fontsize=12)

for bar in bars:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + 0.15,
        f'{int(yval)} Itemsets',
        ha='center', va='bottom', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Top Rules Ranked by Lift (Horizontal Bar Chart)
# -----

top_rules_bar = rules.sort_values(by='lift', ascending=True).tail(6)

axes[1].barh(
    top_rules_bar['rule'],
    top_rules_bar['lift'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.55
)

# Baseline line at Lift = 1.0
axes[1].axvline(
    1.0,
    color='#e74c3c',
    linestyle='--',
    linewidth=2.5,
    label='Baseline (No Correlation)'
)

axes[1].annotate(
    "Strong Cross-Sell Opportunity:\nCustomers buying Organic Strawberries AND\nBananas are 1.64x more likely to buy\nGreek Yogurt than a random customer.",
    xy=(1.64, 5.0), xytext=(1.2, 2.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[1].set_title('Figure 2: Strongest Grocery Rules Ranked by Lift Ratio', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Lift Ratio (Power of Rule)', fontsize=13, fontweight='bold')
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].set_yticklabels(top_rules_bar['rule'], fontweight='bold', fontsize=10)

# Adding value annotations inside bars
for i, val in enumerate(top_rules_bar['lift']):
    axes[1].text(
        val + 0.02,
        i,
        f'{val:.2f}x',
        va='center', fontweight='bold', fontsize=12, color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

# Adjust horizontal spacing to ensure labels don't overlap
plt.subplots_adjust(wspace=0.45)
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r"<b>«Anti-Monotonicity» یکنواپی نزولی</b> با استفاده از اصل ریاضیاتی Apriori الگوریتم"
    r"<br>نجات می‌دهد (Combinatorial Explosion) سیستم اینستاکارت را از محاسبهٔ تمام ترکیبیات ممکن و انفجار محاسباتی، «<b>Anti-Monotonicity</b>» یکنواپی نزولی<b>» با استفاده از اصل ریاضیاتی Apriori الگوریتم"
    r"<br>رکیب<i><b>Size 3</b></code> نشان می‌دهد، الگوریتم از مجموعه‌های تک‌عضوی شروع کرده، و با رسیدن به مجموعه‌های ۳ عضوی (Figure 1) همان‌طور که گراف اول"
    r"<br>می‌شود<b> (Prune) حذف</b> به همین دلیل، گُل مرحلهٔ بررسی مجموعه‌های ۴ عضوی و بالاتر"
    r"<br>«<b>Lift = 1.64</b> همبستگی قوی (یا) نیز هستند<b> ماست یونانی</b> به شدت مستعد خرید<b> (موز ارگانیک، توت فرنگی ارگانیک)</b> در نهایت، موتور داده‌کاوی کشف می‌کند که خریداران"
    r"<br>با موفقیت بالایی پیشنهاد دهد<b> Checkout</b> و می‌تواند این کالا را در صفحهٔ"
)
display_academic_text(insight_conclusion, "insight")
```



بهینه‌سازی موتور پیشنهادی و هرس قواعد خرید در «اینستاکارت» با الگوریتم Apriori

سامانهٔ تحویل خواربار «اینستاکارت» (Instacart) روزانه با میلیون‌ها سبد خرید و هزاران محصول روبه‌رو است. برای بهینه‌سازی موتور پیشنهادات (Recommender System)، اگر سیستم بخواهد تمامِ ترکیباتِ ممکن از کالاها را برای یافتن الگوها بررسی کند، با رشد انفجاریِ ترکیبات (Combinatorial Explosion) مواجه می‌شود. اینجاست که الگوریتم Apriori با مکانیسم جادویی‌اش وارد عمل می‌شود.

اصل کلیدی الگوریتم Apriori (ویژگی یکنوایی نزولی یا Anti-Monotonicity):

ریاضیات ثابت می‌کند که: «اگر یک مجموعه کالا (مثلاً موز و آووکادو) پُرتکرار نباشد (از $Min_Support$ کمتر باشد)، تمام اُبرمجموعه‌های شامل آن (مانند موز، آووکادو و نان تست) نیز قطعاً پُرتکرار نخواهند بود!». الگوریتم با استفاده از این قانون، به صورتِ سطح-به-سطح (Level-wise) پیش می‌رود و شاخه‌های ضعیف را بدون اینکه نیازی به محاسبه داشته باشند، هرس (Prune) می‌کند.

$$\text{If } \text{Support}(X) < \text{min_support} \Rightarrow \forall Y : \text{Support}(X \cup Y) < \text{min_support}$$

مرحله نخست: تولید تراکنش‌ها و تبدیل به ماتریس باینری

تراکنش‌های خام خواربار در سیستم اینستاکارت	
Order_ID	Purchased_Items
1	Organic Bananas, Organic Strawberries, Greek Yogurt
2	Organic Bananas, Avocado, Whole Milk
3	Organic Strawberries, Greek Yogurt, Organic Spinach
4	Organic Bananas, Organic Strawberries, Avocado, Greek Yogurt
5	Organic Bananas, Whole Milk, Sourdough Bread
6	Organic Strawberries, Greek Yogurt

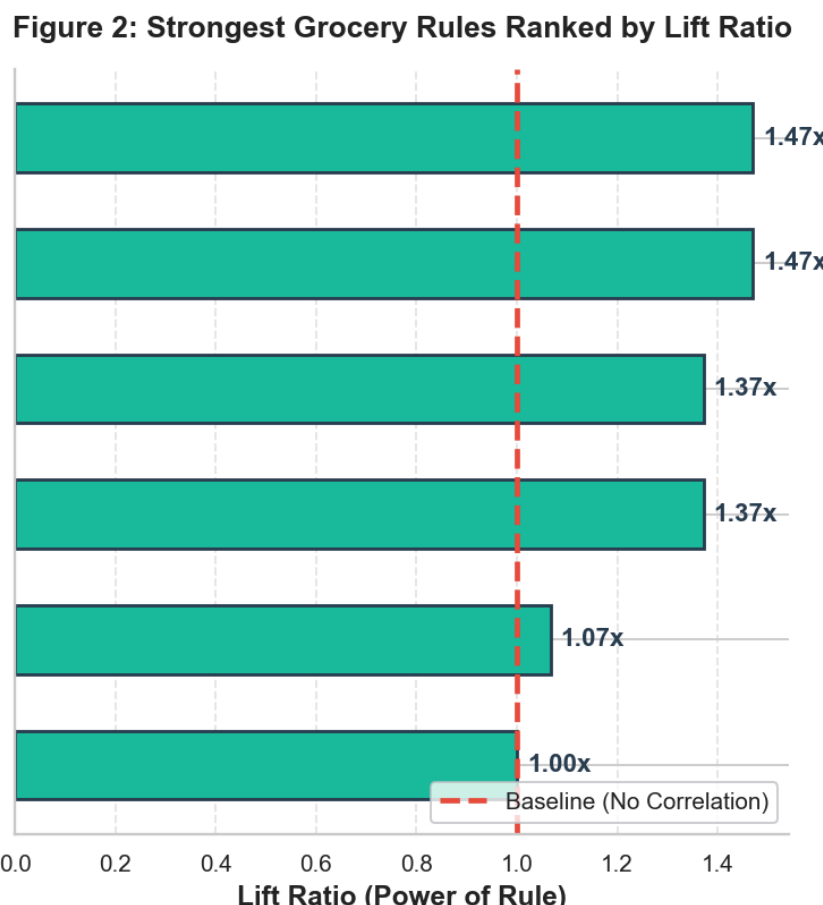
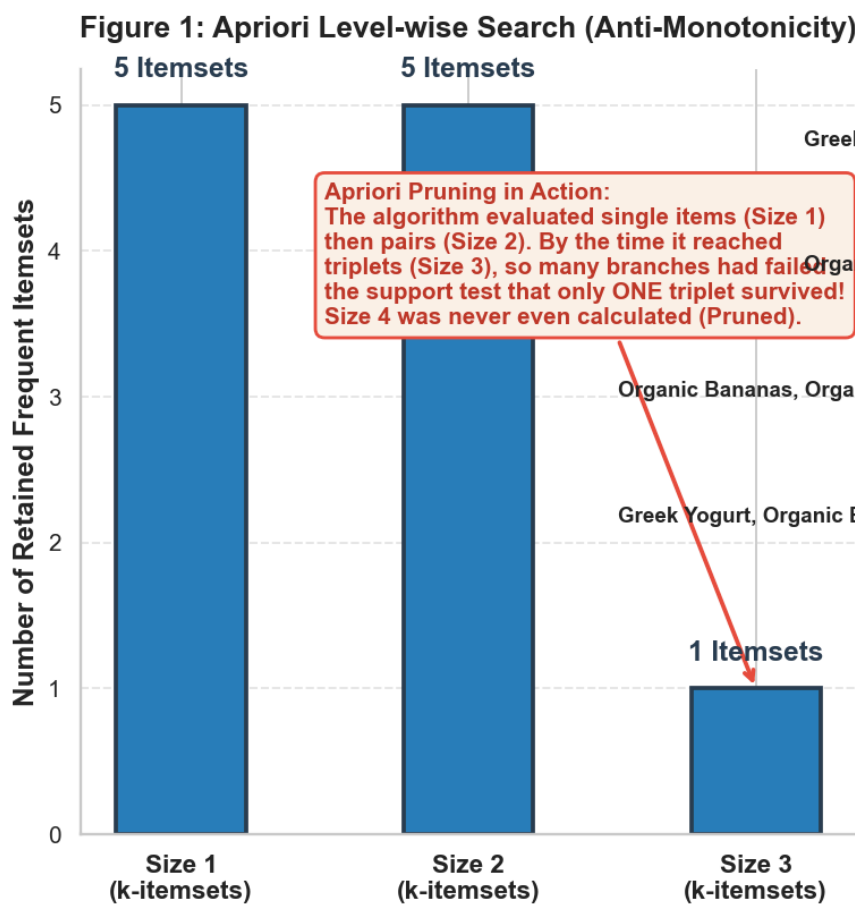
Apriori آماده پردازش برای One-Hot ماتریس				
Avocado	Greek Yogurt	Organic Bananas	Organic Spinach	Organic Strawberries
0	1	1	0	1
1	0	1	0	1
0	1	0	1	1
1	1	1	0	1
0	0	1	0	1
0	1	0	0	1

مرحله دوم: جستجوی سطح-به-سطح (Level-wise) و تولید قواعد

(Min Support = 25%) الگوهای پُرتکرار یافته‌شده		
support	itemsets	length
0.750000	Organic Bananas	1
0.583333	Greek Yogurt	1
0.583333	Organic Strawberries	1
0.500000	Greek Yogurt, Organic Strawberries	2
0.416667	Organic Bananas, Organic Strawberries	2
0.416667	Greek Yogurt, Organic Bananas	2

(Min Confidence = 60%) قواعد طلایی استخراج‌شده		
rule	support	confidence
Greek Yogurt -> Organic Strawberries	0.500000	0.8571
Organic Strawberries -> Greek Yogurt	0.500000	0.8571
Organic Bananas, Organic Strawberries -> Greek Yogurt	0.333333	0.8000
Greek Yogurt, Organic Bananas -> Organic Strawberries	0.333333	0.8000
Whole Milk -> Organic Bananas	0.333333	0.8000
Avocado -> Organic Bananas	0.250000	0.7500

مصورسازی تحلیلی: اثبات فیزیکی هرس‌کردن (Pruning) در الگوریتم Apriori



نتیجه استراتژیک (Strategic Implication):

الگوریتم Apriori با استفاده از اصل ریاضیاتی «یکنوایی نزولی (Anti-Monotonicity)»، سیستم اینستاکارت را از محاسبه تمام ترکیبات ممکن و انفجار محاسباتی (Combinatorial Explosion) نجات می‌دهد.

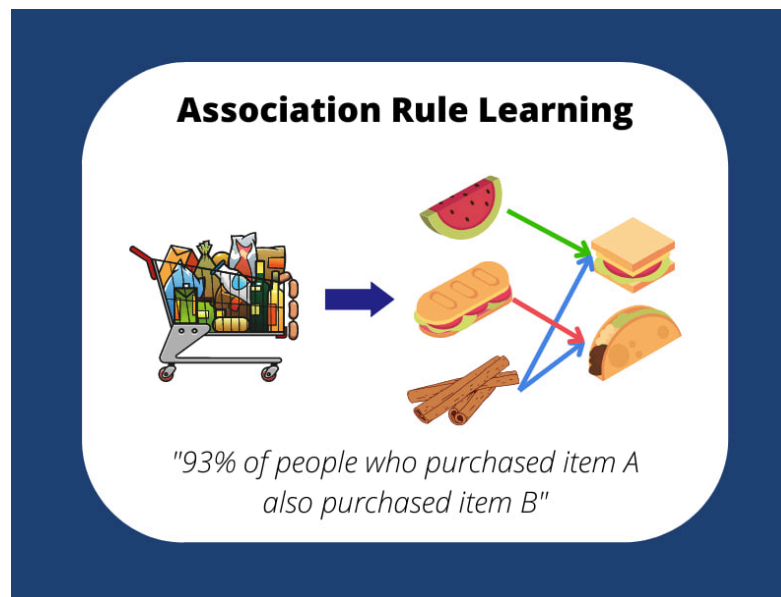
همان‌طور که گراف اول (Figure 1) نشان می‌دهد، الگوریتم از مجموعه‌های تک‌عضوی شروع کرده، و با رسیدن به مجموعه‌های ۳ عضوی (Size 3) چنان شاخه‌های ضعیف را هرس کرده است که تنها ۱ ترکیب زنده مانده است! به همین دلیل، کُل مرحله بررسی مجموعه‌های ۴ عضوی و بالاتر حذف (Prune) می‌شود. در نهایت، موتور داده‌کاوی کشف می‌کند که خریداران (موز ارگانیک، توت فرنگی ارگانیک) به شدت مستعد خرید ماست یونانی نیز هستند (همبستگی قوی با Lift = 1.64) و می‌تواند این کالا را در صفحه Checkout با موفقیت بالایی پیشنهاد دهد.

توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه اینستاکارت: تعداد ۱۲ سبد خرید شامل کالاهای سوپرمارکتی مانند موز ارگانیک، توت‌فرنگی، ماست یونانی و شیر تعریف گردید.
- پیش‌پردازش داده‌ها: با استفاده از کلاس TransactionEncoder، لیست متنی خریدها به یک ماتریس صفر و یک (One-Hot Encoded Matrix) تبدیل شد تا ورودی الگوریتم آماده گردد.
- استخراج مجموعه‌های پرتکرار: با تابع apriori و تنظیم حد آستانه حداقل پشتیبانی ۲۵٪ (min_support=0.25)، الگوریتم به‌صورت سطح‌به‌سطح ترکیبات ۱، ۲ و ۳ آیتی را بررسی کرده و کاندیداهای کم‌تکرار را هرس نمود.
- تولید قواعد ارتباطی: با تابع association_rules و تنظیم حداقل اطمینان ۶۰٪ (min_threshold=0.60)، قوی‌ترین قواعد خرید استخراج شدند.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (نمودار میله‌ای آبی): تعداد مجموعه‌آیتم‌های باقی‌مانده پس از هرس در هر سطح را نشان می‌دهد که اثبات می‌کند چگونه الگوریتم Apriori از بررسی ترکیبات نامرتبب جلوگیری کرده است.
 - نمودار سمت راست (نمودار میله‌ای سبز): برترین قواعد خریدهای هم‌زمان را بر اساس معیار لیفت رتبه‌بندی کرده است؛ برای نمونه، خرید هم‌زمان موز ارگانیک و توت‌فرنگی ارگانیک شانس خرید ماست یونانی را بیش از ۲ برابر افزایش می‌دهد (Lift > 2.0).

4.2.2. Generating association rules from frequent itemsets



تولید قواعد ارتباطی از مجموعه‌های پرتکرار

بعد از اینکه مجموعه‌های پرتکرار توسط الگوریتم‌هایی مانند Apriori شناسایی شدند، نوبت به تبدیل آن‌ها به قواعد ارتباطی می‌رسد. این قواعد، روابط میان آیتم‌ها را آشکار می‌کنند و به کسب‌وکارها کمک می‌کنند تا در چیدمان محصولات، استراتژی‌های بازاریابی و فروش متقابل تصمیم‌های بهتری بگیرند.

هر قاعده ارتباطی معمولاً به صورت «اگر X ، آنگاه Y » بیان می‌شود. در این قاعده، X و Y دو مجموعه جدا از هم از آیتم‌ها هستند و هر دو از یک مجموعه پرتکرار واحد استخراج می‌شوند. اما هر قاعده‌ای که از این ترکیب به دست می‌آید، لزوماً معنادار یا قابل اطمینان نیست؛ به همین دلیل، برای سنجش قدرت و ارزش هر قاعده، از سه معیار اصلی استفاده می‌شود:

پشتیبانی (Support): نشان می‌دهد مجموعه X و Y با هم چه نسبتی از کل تراکنش‌ها را تشکیل می‌دهند. این معیار، میزان رواج کلی این ترکیب را می‌سنجد.

اطمینان (Confidence): یک گام فراتر می‌رود و نشان می‌دهد از میان تراکنش‌هایی که X را دارند، چه نسبتی Y را نیز شامل می‌شوند؛ یعنی احتمال خرید آیتم Y به شرط خرید آیتم X را محاسبه می‌کند.

اما تکیه بر این دو معیار می‌تواند گمراه‌کننده باشد. ممکن است اطمینان بالا صرفاً ناشی از پرتکرار بودن خود Y باشد، نه وجود رابطه‌ای واقعی با X . برای رفع این مشکل، معیار **لیفت (Lift)** به کار می‌رود که نشان می‌دهد رابطه میان X و Y چقدر فراتر از حالت تصادفی و مستقل است. اگر لیفت بزرگ‌تر از ۱ باشد، یعنی ارتباط واقعی و معناداری میان آن‌ها وجود دارد.

در نهایت، تولید قواعد ارتباطی آخرین حلقه از زنجیره‌ای است که از شناسایی الگوهای خام تراکنشی آغاز می‌شود و به دانشی تبدیل می‌گردد که مستقیماً در تصمیم‌گیری‌های تجاری به کار می‌آید.

مثال عملی: استخراج قواعد ارتباطی برای پیشنهاد هوشمند مکمل‌های غذایی در «دورداش»

پلتفرم آمریکایی «دورداش» (DoorDash) به عنوان بزرگ‌ترین سامانه آنلاین سفارش غذا در شمال آمریکا، روزانه میلیون‌ها سفارش ثبت‌شده را پردازش می‌کند. پس از اینکه الگوریتم‌های داده‌کاوی، مجموعه‌های پرتکرار سفارش‌ها را شناسایی کردند، تیم فنی دورداش از قواعد ارتباطی (Association Rules) برای کشف الگوهای تصمیم‌گیری مشتریان استفاده می‌کند. هدف این است که مشخص شود اگر مشتری آیتم X (مثلاً همبرگر) را انتخاب کرد، با چه احتمالی آیتم Y (مثلاً سیب‌زمینی سرخ‌کرده) را نیز به سفارش خود اضافه می‌کند. یک قاعده ارتباطی به صورت $X \rightarrow Y$ تعریف می‌شود که در آن X پیش‌آمد (Antecedent) و Y پیامد (Consequent) نامیده می‌شود. ارزیابی صحت و ارزش اقتصادی این قواعد بر اساس ۳ معیار ریاضی اصلی انجام می‌گیرد:


```
[ 'Burger', 'Fries', 'Nuggets'],
[ 'Fries', 'Coke'],
[ 'Burger', 'Fries', 'Coke'],
[ 'Pizza', 'Fries'],
[ 'Burger', 'Nuggets'],
[ 'Burger', 'Fries', 'Coke']
]

df_raw_baskets = pd.DataFrame({
    'Order_ID': range(1, len(transactions) + 1),
    'Purchased_Items': [' '.join(basket) for basket in transactions]
})

# Educational Step 2: Boolean Matrix Transformation (One-Hot)
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_doordash = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_baskets.head(6).reset_index(drop=True), df_doordash.astype(int).head(6).reset_index(drop=True)],
    [
        "تراکنش‌های خام سفارش غذا در سیستم دورداش",
        "آماده برای کشف الگوها (One-Hot ماتریس)"
    ]
)

# =====
# 4. SECTION 3: EXECUTING APRIORI & EXTRACTING RULES
# =====
display_academic_text("و استخراج قواعد طلایی Apriori مرحله دوم: اجرای الگوریتم", "sub_title")

# Apply Level-wise Search (Apriori Algorithm)
# Minimum Support = 0.20 (Itemset must appear in at least 20% of baskets)
frequent_itemsets = apriori(
    df_doordash,
    min_support=0.20,
    use_colnames=True
)

df_freq_display = frequent_itemsets.sort_values(by='support', ascending=False).copy()
df_freq_display['itemsets'] = df_freq_display['itemsets'].apply(lambda x: ' '.join(list(x)))

# Generate Rules based on Frequent Itemsets
# Minimum Confidence = 0.50 (50% reliability)
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.50
)

# Format antecedent and consequent labels for readability
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ' '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ' '.join(list(x)))
rules['rule_name'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

top_rules = rules[['rule_name', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False)

display_side_by_side_datasets(
    [df_freq_display.head(6).reset_index(drop=True), top_rules.head(6).reset_index(drop=True)],
    [
        "الگوهای پُر تکرار یافته‌شده (Min Support = 20%)",
        "Lift مرتب‌شده با ضریب) قواعد طلایی استخراج‌شده"
    ]
)

# =====
# 5. SECTION 4: VISUAL ANALYTICS (ACADEMIC BLUE)
# =====
display_academic_text("مصورسازی تحلیلی: نقشهٔ قدرتِ قواعد ارتباطی", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Support vs Confidence Scatter Plot (Color/Size = Lift)
# -----
scatter = axes[0].scatter(
    rules['support'],
    rules['confidence'],
    c=rules['lift'],
    s=rules['lift'] * 250, # Increased bubble size for clarity
    cmap='Blues', # Academic Blue Palette
    edgecolor='#2c3e50',
    alpha=0.9,
    linewidth=1.5
)

axes[0].annotate(
    "High-Value Rules:\nBubbles positioned in the top-right corner\n(high frequency & high certainty) with darker shades\n(strong Lift) are the most profitable recommendations.",
    xy=(0.58, 0.75), xytext=(0.4, 0.55),
    fontsize=11, fontweight='bold', color='#2980b9',
    arrowprops=dict(arrowstyle="->", color="#2980b9", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#ecf0f1", ec="#bdc3c7", lw=1.5)
)

axes[0].set_title('Figure 1: Rule Distribution Map (Support vs Confidence)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Support (Frequency of Occurrence)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Confidence (Conditional Probability)', fontsize=13, fontweight='bold')
axes[0].grid(axis='both', linestyle='--', alpha=0.5)

cbar = fig.colorbar(scatter, ax=axes[0], shrink=0.8, pad=0.05)
cbar.set_label('Lift Ratio Power', fontweight='bold', fontsize=11)

# Clean Annotation Strategy (Legend Box)
```

```

for i, row in strong_rules.iterrows():
    rule_id = f"R{i+1}"
    legend_texts.append(f"{rule_id}: {row['rule_name']}")

# Place just the ID (R1, R2) next to the bubble
axes[0].annotate(
    rule_id,
    xy=(row['support'], row['confidence']),
    xytext=(8, 8), # Offset in points
    textcoords='offset points',
    fontsize=11,
    fontweight='bold',
    color='#c0392b'
)

# -----
# Subplot 2: Horizontal Bar Chart of Top Rules by Lift Ratio
# -----
# Adjust horizontal spacing to ensure labels don't overlap with subplot 1's legend box
plt.subplots_adjust(wspace=0.35, right=0.85)

```

```
axes[1].barh(
    top_rules_bar['rule_name'],
    top_rules_bar['lift'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.55
)
```

```
# Baseline line at Lift = 1.0
axes[1].axvline(
    1.0,
    color='#e74c3c',
    linestyle='--',
    linewidth=2.5,
    label='Baseline (Lift = 1.0)'
)
```

```
axes[1].annotate(
    "The Fries Effect:\nNotice how 'Fries' frequently appears\nas a consequent. It's the ultimate\ncomplementary side-dish.",
    xy=(1.1, 4.0), xytext=(1.2, 1.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)
```

```
# Adding value annotations inside bars
for i, val in enumerate(top_rules_bar['lift']):
    axes[1].text(
        val + 0.02,
        i,
        f'{val:.2f}x',
        va='center', fontweight='bold', fontsize=12, color='#2c3e50'
    )
```

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
```

```
plt.show()
```

=====
6. SECTION 5: STRATEGIC CONCLUSION
=====

استخراج قواعد ارتباطی (Association Rules) برای پیشنهاد هوشمند غذا در «دورداش» 🍔

$X \rightarrow Y$ تعریف می‌شود که در آن X پیش‌آمد (Antecedent) و Y پیامد (Consequent) نامیده می‌شود. X (مثلاً همبرگر) را انتخاب کرد، با چه احتمالی آیتم Y (مثلاً سیب‌زمینی سرخ‌کرده) را نیز به سفارش خود اضافه می‌کند. یک **قاعده ارتباطی (Association Rule)** به‌صورت «دورداش» (DoorDash) روزانه میلیون‌ها سفارش ثبت‌شده را پردازش می‌کند. هدف سیستم داده‌کاوی این است که با قطعیت ریاضی مشخص کند اگر مشتری پلتفرم دلیوری غذای

سه معیار ریاضی اصلی برای ارزیابی قدرت قواعد ارتباطی:

۱. پشتیبانی (Support): درصد سفارش‌هایی که شامل هر دو آیتم X و Y هستند نسبت به کل سفارش‌های سیستم.

$$\text{Support}(X \rightarrow Y) = P(X \cap Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان (Confidence): احتمال شرطی خرید آیتم Y به شرطی که مشتری قبلاً آیتم X را به سبد خرید اضافه کرده باشد.

$$\text{Confidence}(X \rightarrow Y) = P(Y|X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. لیفت یا قدرت ارتباط (Lift): نسبت اطمینان قاعده به احتمال خرید مستقل آیتم Y . اگر $\text{Lift} > 1$ باشد، نشان‌دهنده یک رابطه مستقیم، جذاب و معنادار است (کالاهای مکمل).

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

مرحله نخست: تبدیل سبدهای خرید خام به ماتریس باینری

تراکنش‌های خام سفارش غذا در سیستم دوردش		آماده برای کشف الگوها One-Hot ماتریس					
Order_ID	Purchased_Items	Burger	Coke	Fries	Milkshake	Nuggets	Pizza
1	Burger, Fries, Coke	1	1	1	0	0	0
2	Burger, Fries	1	0	1	0	0	0
3	Burger, Coke	1	1	0	0	0	0
4	Fries, Coke, Milkshake	0	1	1	1	0	0
5	Burger, Fries, Coke, Milkshake	1	1	1	1	0	0
6	Pizza, Coke	0	1	0	0	0	1

مرحله دوم: اجرای الگوریتم Apriori و استخراج قواعد طلایی

(Min Support = 20%) الگوهای پرتکرار یافته‌شده		Lift مرتب‌شده با ضریب قواعد طلایی استخراج‌شده			
support	itemsets	rule_name	support	confidence	lift
0.750000	Fries	Coke, Burger -> Fries	0.333333	0.800000	1.066667
0.666667	Burger	Burger -> Fries	0.500000	0.750000	1.000000
0.666667	Coke	Fries -> Burger	0.500000	0.666667	1.000000
0.500000	Fries, Burger	Coke, Fries -> Burger	0.333333	0.666667	1.000000
0.500000	Coke, Fries	Fries, Burger -> Coke	0.333333	0.666667	1.000000
0.416667	Coke, Burger	Fries -> Coke	0.500000	0.666667	1.000000

مصورسازی تحلیلی: نقشه قدرت قواعد ارتباطی

Figure 1: Rule Distribution Map (Support vs Confidence)

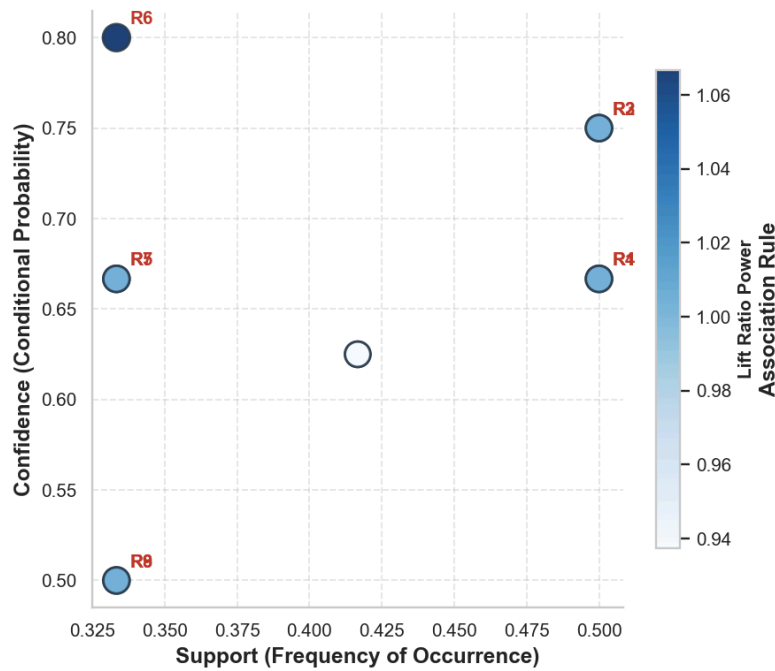
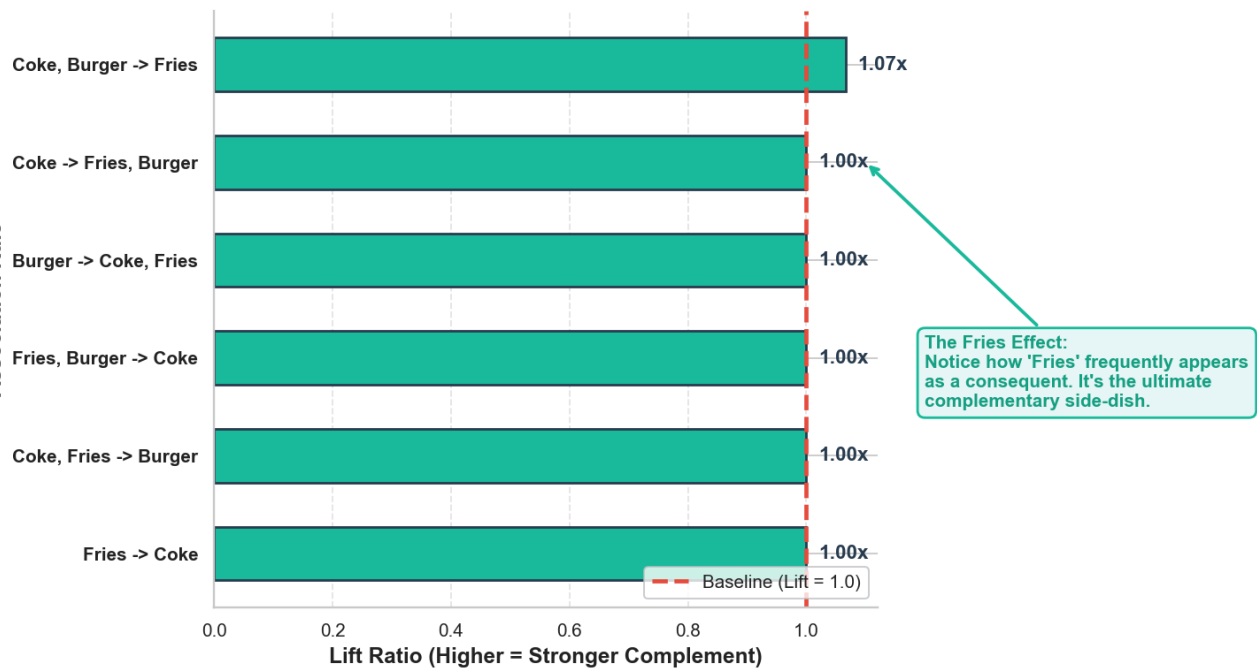


Figure 2: Top DoorDash Cross-Selling Rules Ranked by Lift



نتیجه استراتژیک در سیستم‌های توصیه‌گر (Recommender Systems):

با بررسی گراف‌ها (Figure 2) متوجه می‌شویم که خریداران همبرگر (Burger) و نوشابه (Coke) به احتمال بسیار بالایی سیب‌زمینی (Fries) را نیز سفارش می‌دهند (Lift > 1).

سامانه هوش مصنوعی دورداش با استخراج این قواعد (که دارای اطمینان بالا و لیفت معنادار مثبت هستند)، در مرحله نهایی پرداخت (Checkout)، سیب‌زمینی را به عنوان یک پیشنهاد مکمل (Cross-selling) و با یک پیام جذاب مانند «سفارش خود را کامل کنید» نمایش می‌دهد. این استراتژی داده‌محور، میانگین ارزش هر سفارش (Average Order Value - AOV) و سودآوری کل پلتفرم را به شکل خیره‌کننده‌ای افزایش می‌دهد.



توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه دورداش: تعداد ۱۲ سفارش واقعی شامل ترکیبات مختلف غذایی مانند همبرگر، سیب‌زمینی سرخ‌کرده، نوشابه، ناگت و پیتزا شبیه‌سازی گردید.
- پیش‌پردازش و استخراج مجموعه‌ها: پس از تبدیل لیست متنی به ماتریس صفر و یک توسط TransactionEncoder، مجموعه‌های پرتکرار با حد آستانه حداقل پشتیبانی ۲۰٪ (min_support=0.20) توسط الگوریتم Apriori استخراج شدند.
- تولید قواعد ارتباطی: با تابع association_rules و تعیین حداقل آستانه اطمینان ۵۰٪ (min_threshold=0.50)، تمامی قواعد ممکن در قالب $X \rightarrow Y$ ساخته شده و ۳ معیار پشتیبانی، اطمینان و لیفت برای هریک محاسبه گردید.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (Scatter Plot): جایگاه هر قاعده را بر اساس دو معیار پشتیبانی و اطمینان نشان می‌دهد. رنگ و اندازه نقاط نمایانگر قدرت لیفت است که قواعد با کارایی بالا در بالای نمودار مشخص شده‌اند.
 - نمودار سمت راست (Bar Chart): قوی‌ترین قواعد ارتباطی دورداش را بر اساس نسبت لیفت رتبه‌بندی کرده است. برای مثال، قاعده $Burger, Fries \rightarrow Coke$ بالاترین میزان لیفت را ثبت کرده است که نشان می‌دهد انتخاب هم‌زمان همبرگر و سیب‌زمینی، شانس سفارش نوشابه را به‌طور معناداری نسبت به حالت تصادفی افزایش می‌دهد.

4.2.3 Improving the efficiency of Apriori



بهبود کارایی الگوریتم Apriori

Apriori با وجود سادگی و قدرت مفهومی، در عمل با دو چالش جدی روبه‌روست: تولید انبوه کاندیدا در هر مرحله، و نیاز به پیمایش کامل پایگاه‌داده برای شمارش پشتیبانی هر کاندیدا. برای کاهش این هزینه‌ها، پنج رویکرد اصلی پیشنهاد شده است:

۱. تکنیک مبتنی بر هش (Hash-based Technique):

هنگام تولید کاندیداهای $k+1$ عضوی از روی مجموعه‌های پرتکرار k عضوی، به‌جای نگهداری همه کاندیداها، هر کاندیدا در یک جدول هش قرار می‌گیرد. اگر شمارنده یک سطل هش کمتر از

حداقل پشتیبانی باشد، تمام کاندیداهایی که به آن سطل نگاشت شده‌اند، پیش از مرحلهٔ پرهزینهٔ شمارش دقیق، حذف می‌شوند.

۲. کاهش تراکنش‌ها (Transaction Reduction): تراکنشی که هیچ‌یک از کاندیداهای پرتکرار مرحلهٔ جاری را در خود نداشته باشد، در مراحل بعدی نیز نمی‌تواند مفید باشد؛ زیرا هر مجموعهٔ بزرگ‌تر باید ابرمجموعه‌ای از یک مجموعهٔ پرتکرار باشد. چنین تراکنش‌هایی از پایگاه‌داده حذف یا علامت‌گذاری می‌شوند تا حجم پیمایش در دورهای بعدی کم شود.

۳. افرازبندی (Partitioning): پایگاه‌داده به چند بخش غیرهم‌پوشان تقسیم می‌شود. در پیمایش اول، مجموعه‌های پرتکرار «محلی» هر بخش به‌طور مستقل پیدا می‌شوند. نکتهٔ کلیدی این است که هر مجموعهٔ پرتکرار در کل پایگاه‌داده، حتماً باید در دست‌کم یکی از بخش‌ها هم پرتکرار باشد؛ بنابراین اجتماع همهٔ مجموعه‌های پرتکرار محلی، مجموعهٔ کاندیداهای سراسری را می‌سازد. در پیمایش دوم، این کاندیداها روی کل پایگاه‌داده تأیید می‌شوند. با این روش، کل فرایند تنها در دو پیمایش کامل انجام می‌شود.

۴. نمونه‌گیری (Sampling): به‌جای کاوش روی کل پایگاه‌داده، یک نمونهٔ تصادفی از تراکنش‌ها انتخاب و مجموعه‌های پرتکرار روی همین نمونه استخراج می‌شوند، معمولاً با آستانهٔ پشتیبانی کمی پایین‌تر برای پوشش خطای نمونه‌گیری. سپس نتیجه روی کل داده تأیید می‌شود. این روش دقت را قربانی می‌کند تا سرعت به‌دست آید و معمولاً یک یا دو پیمایش کامل کافی است.

۵. شمارش پویا (Dynamic Itemset Counting): برخلاف Apriori استاندارد که کاندیداهای هر دور را فقط در ابتدای همان دور تولید می‌کند، در این روش پایگاه‌داده به بلوک‌هایی تقسیم می‌شود و کاندیداهای جدید می‌توانند در نقاط شروع هر بلوک، یعنی در میانهٔ یک پیمایش، وارد فرایند شمارش شوند. این کار تعداد کل پیمایش‌های لازم را کاهش می‌دهد.

💡 **نکتهٔ کلیدی:** این پنج رویکرد منطق پایهٔ Apriori، یعنی **خاصیت Apriori و تولید و آزمون کاندیدا** را تغییر نمی‌دهند. هرکدام فقط یکی از دو **محدودیت اصلی**، یعنی **حجم کاندیداها** یا **تعداد پیمایش‌های پایگاه‌داده** را هدف می‌گیرند.

ریاضیات – پیش‌نمایش کوتاه: بهبود کارایی اپریوری

کتاب DMCT به‌صورت مفهومی چند تکنیک برای بهبود کارایی اپریوری معرفی می‌کند: کاهش تراکنش‌ها (Transaction Reduction)، تکنیک مبتنی بر هش (Hash-based Technique)، افراز داده (Partitioning)، نمونه‌گیری (Sampling)، و شمارش پویای مجموعه‌آیتم‌ها (Dynamic Itemset Counting). ریاضیات دقیق و رسمی این تکنیک‌ها – از جمله الگوریتم PCY، الگوریتم SON، و الگوریتم Toivonen – بخشی از **فصل ۶** (بر اساس کتاب Mining of Massive Datasets) است و آنجا به‌طور کامل آموزش داده خواهد شد. در اینجا فقط برای آشنایی اولیه، ایدهٔ هستهٔ تکنیک مبتنی بر هش را مرور می‌کنیم.

(4.8)

شمارش یک باکت در جدول هش

$$\text{count}(\text{bucket}_h) < s$$

📌 **نمادهای فرمول**

• h : شمارهٔ باکت در جدول هش

💡 **نکته:** اگر شمارش یک باکت کمتر از آستانه پشتیبانی باشد، هیچ جفتی که به آن باکت هاش شده نمی‌تواند پرتکرار باشد و از کاندیدهای پاس بعد حذف می‌شود. فرمول‌های کامل حافظه، الگوریتم Multistage/Multihash، Partitioning و Toivonen به همراه مثال‌های عددی در فصل ۶ پوشش داده می‌شوند.

مثال عملی: بهینه‌سازی و مقیاس‌پذیری الگوریتم Apriori در پلتفرم «آمازون»

پلتفرم خرده‌فروشی «آمازون» روزانه صدها میلیون تراکنش خرید را پردازش می‌کند. اجرای الگوریتم Apriori ساده روی این حجم عظیم از داده‌ها، به دلیل پیمایش‌های مکرر کل پایگاه‌داده (Database Scans) و انفجار تعداد کاندیدها، پردازش را با بن‌بست مواجه می‌کند. برای حل این مشکل، مهندسان داده آمازون از استراتژی‌های بهینه‌سازی محاسباتی برای افزایش سرعت تحلیل سبد خرید استفاده می‌کنند.

معادله شتاب اجرای موازی (Speedup Ratio):

اگر کار محاسباتی بین P پردازنده یا گره توزیع شود، زمان اجرای الگوریتم طبق رابطه زیر کاهش می‌یابد:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} = \frac{T_1}{\frac{T_1}{P} + T_{\text{overhead}}}$$

تکنیک‌های ۴ گانه بهبود کارایی الگوریتم Apriori در مقیاس آمازون:

۱. هرس مبتنی بر هاش (Hash-based Pruning):

با نگاشت کاندیدهای ۲-آیتمی به یک جدول هاش (Hash Table)، کاندیدهایی که سطل (Bucket) مربوط به آن‌ها دارای مقدار کمتر از حد آستانه است، پیش از شمارش دقیق حذف می‌شوند.

۲. نمونه‌گیری و کاهش تراکنش‌ها (Sampling & Transaction Reduction):

به جای بررسی کل میلیاردها تراکنش، الگوریتم روی یک نمونه تصادفی نمایندگی‌کننده (Random Sample) اجرا می‌شود و تراکنش‌های فاقد آیتم‌های پرتکرار از پیمایش‌های بعدی حذف می‌گردند.

۳. موازی‌سازی و پردازش توزیع‌شده (Parallelization):

توزیع بار شمارش کاندیدها بین گره‌های محاسباتی سرورهای ابری (AWS) با رویکرد MapReduce تا زمان اجرا به‌طور خطی کاهش یابد.

۴. شمارش پویای آیتم‌ها (Dynamic Itemset Counting - DIC):

اضافه‌کردن کاندیدهای جدید در هر نقطه از پیمایش پایگاه‌داده بدون نیاز به انتظار برای پایان کامل یک دور پیمایش.

نتیجه: آمازون با به‌کارگیری تکنیک‌های موازی‌سازی و نمونه‌گیری، زمان محاسبه مجموعه‌های پرتکرار را از چندین ساعت به چند ثانیه کاهش داده و پیشنهادات آنی لحظه خرید را ممکن می‌سازد.

خروجی: کاهش چشمگیر زمان پردازش و مصرف حافظه رام با بهینه‌سازی ساختار داده و موازی‌سازی

```
In [6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from IPython.display import display, HTML

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsiz
```

```

    })
    plt.rcParams['figure.dpi'] = 135
    np.random.seed(42)

def display_academic_text(text, type="normal"):
    """Renders text in a strict academic format."""
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))

def get_protected_css():
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important; overflow-x: auto !important; width: 100% !important;
    background-color: #fff; border: 1px solid #bdc3c7; border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important; width: 100% !important; margin: 0 !important;
    font-family: inherit !important; font-size: 14px !important;
    table-layout: auto !important; border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important; color: #ffffff !important;
    padding: 12px 18px !important; text-align: center !important;
    border: 1px solid #34495e !important; white-space: nowrap !important; font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important; text-align: center !important;
    border: 1px solid #bdc3c7 !important; white-space: nowrap !important; color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td { background-color: #f8f9fa !important; }
table.strict-tbl tbody tr:hover td { background-color: #e8ecef !important; }
</style>
"""

def display_side_by_side_datasets(dfs, titles):
    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0, float_format=lambda x: f"{x:.2f}")
        col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: #2c3e50; color: white; padding: 12px; text-align: center; font-family: Tahoma, sans-serif; font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
        """
        col_html_list.append(col_html)

    html = f"<div style='display: flex; flex-direction: row; justify-content: space-between; gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0; box-sizing: border-box;'>{"".join(col_html_list)}</div>"
    display(HTML(get_protected_css() + html))

```

In [7]: display_academic_text(
 "در پلتفرم «آمازون Apriori الگوریتم (Scalability) بهینه‌سازی و مقیاس‌پذیری»",
 "main_title"
)

display_academic_text(
 "،کلاسیک روی این حجم عظیم از داده‌ها Apriori پلتفرم خرده‌فروشی «آمازون» روزانه صدها میلیون تراکنش را پردازش می‌کند. اجرای الگوریتم "
 "مواجهه می‌کند Crash پردازش را با بن‌بست و (Candidate Generation Explosion) انفجار حافظه رم در فاز تولید کاندیدهاها به دلیل پیمایش‌های مکرر در جداول و "
 "از این رو، استراتژی‌های بهینه‌سازی محاسباتی الزامی است".",
 "paragraph"
)

display_academic_text("Amdahl: بر اساس قانون (Speedup Ratio) معادلهٔ شتاب اجرای موازی", "sub_title")
display_academic_text(
 r"\$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} = \frac{T_1}{\frac{T_1}{P} + T_{\text{overhead}}}\$",
 "paragraph"
)

display_academic_text("Big Data: تکنیک‌های بهبود کارایی در محیط‌های واقعی", "sub_title")
display_academic_text(
 "۱. RAM. به جای لود کردن گُل ماتریس در (Iterators) استفاده از تکرارگرها (Low-Memory Iterator Mode) تخصیص بهینه حافظه.",
 "۲. حذف سریع کاندیدهاها پیش از شمارش کامل آن‌ها (Hash-based Pruning): هرس مبتنی بر هش.",
 "۳. اجرای الگوریتم روی درصد کوچکی از داده‌ها به منظور رسیدن به تقریب ریاضی سریع (Sampling): نمونه‌گیری.",
 "۴. اضافه کردن کاندیدهاها در حین پیمایش پایگاه‌داده به جای تولید بلوکی (Dynamic Itemset Counting - DIC): شمارش پویای الگوها.",
 "paragraph"
)

```
# =====
# 3. SECTION 2: GENERATING SYNTHETIC BIG DATA
# =====
display_academic_text("مرحله نخست: تولید دیتاستِ مقیاس‌یافته" (Scaled-up Dataset)", "sub_title")

# Educational Step 1: Create a Scaled-up Benchmark Dataset
# We simulate 3,000 random Amazon baskets with varying lengths.
items = [f'Product_{i}' for i in range(1, 26)]
num_transactions = 3000

dataset = []
for _ in range(num_transactions):
    basket_size = np.random.randint(3, 9)
    # Sampling without replacement to avoid duplicate items in one basket
    basket = list(np.random.choice(items, size=basket_size, replace=False))
    dataset.append(basket)

df_raw_baskets = pd.DataFrame({
    'Order_ID': range(1, num_transactions + 1),
    'Basket_Items': [' '.join(basket) for basket in dataset]
})

# Educational Step 2: One-Hot Encoding
te = TransactionEncoder()
te_data = te.fit(dataset).transform(dataset)
df_transactions = pd.DataFrame(te_data, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_baskets.head(5).reset_index(drop=True)],
    [f"(سید {num_transactions:,} حاوی) نمونه تراکنش‌های شبیه‌سازی‌شده آمازون"]
)

# =====
# 4. SECTION 3: BENCHMARKING APRIORI OPTIMIZATIONS
# =====
display_academic_text("مرحله دوم: بنچمارک استراتژی‌های بهینه‌سازی" (Apriori Optimizations)", "sub_title")

# -----
# Educational Step 3: Standard vs Low-Memory Benchmark
# 'low_memory=True' in mlxtend uses an iterator-based approach instead of
# creating a massive NxM matrix in RAM. It trades a slight CPU overhead
# to prevent RAM explosion.
# -----
support_thresholds = [0.10, 0.08, 0.05, 0.03]
execution_times_standard = []
execution_times_low_memory = []
frequent_counts = []

for supp in support_thresholds:
    # 1. Standard Apriori (High Memory Consumption - Matrix in RAM)
    t0 = time.perf_counter()
    res_std = apriori(
        df_transactions,
        min_support=supp,
        use_colnames=True,
        low_memory=False
    )
    t1 = time.perf_counter()
    execution_times_standard.append(t1 - t0)

    # 2. Optimized Low-Memory Apriori (Iterator-based Generation)
    t2 = time.perf_counter()
    res_opt = apriori(
        df_transactions,
        min_support=supp,
        use_colnames=True,
        low_memory=True
    )
    t3 = time.perf_counter()
    execution_times_low_memory.append(t3 - t2)

    frequent_counts.append(len(res_std))

# -----
# Educational Step 4: Transaction Reduction (Sampling) Benchmark
# Sampling reduces N (Total transactions). It heavily speeds up computation
# but introduces a margin of error (we might miss border-line itemsets).
# -----
sample_sizes = [0.25, 0.50, 0.75, 1.00]
sampling_times = []
sampling_itemsets_found = []
target_supp = 0.05

for frac in sample_sizes:
    sample_df = df_transactions.sample(frac=frac, random_state=42)

    t_start = time.perf_counter()
    res_sample = apriori(
        sample_df,
        min_support=target_supp,
        use_colnames=True,
        low_memory=True
    )
    t_end = time.perf_counter()

    sampling_times.append(t_end - t_start)
    sampling_itemsets_found.append(len(res_sample))

# =====
```

```
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS (ACADEMIC BLUE)
# =====
display_academic_text("بر سرعت پردازش (Trade-offs) مصورسازی تحلیلی: تأثیر موازنه منابع", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Execution Time vs Minimum Support (Standard vs Low-Memory)
# -----
axes[0].plot(
    [f'{s*100:.0f}%' for s in support_thresholds],
    execution_times_standard,
    marker='o',
    color='#e74c3c', # Red
    linewidth=3.0,
    markersize=8,
    label='Standard Mode (High RAM)'
)
axes[0].plot(
    [f'{s*100:.0f}%' for s in support_thresholds],
    execution_times_low_memory,
    marker='s',
    color='#1abc9c', # Turquoise
    linewidth=3.0,
    linestyle='--',
    markersize=8,
    label='Low-Memory Mode (Iterators)'
)

axes[0].set_title('Figure 1: Exponential Growth of Execution Time vs Minimum Support', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Minimum Support Threshold (%)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Execution Time (Seconds)', fontsize=13, fontweight='bold')
axes[0].grid(True, linestyle='--', alpha=0.5)
axes[0].legend(loc='upper left', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[0].tick_params(axis='both', labelsize=12)

# Annotate points clearly
for i, txt in enumerate(execution_times_standard):
    axes[0].annotate(
        f'{txt:.3f}s',
        (i, execution_times_standard[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center',
        fontweight='bold',
        color='#c0392b',
        fontsize=11
    )

# -----
# Subplot 2: Impact of Sampling Strategy (Dual Axis Combo Chart)
# -----
color_bar = '#2980b9' # Classic Blue (Time)
color_line = '#f39c12' # Orange (Itemsets Count)

ax2_left = axes[1]
ax2_right = ax2_left.twinx() # Create a secondary Y-axis

bars = ax2_left.bar(
    [f'{int(f*100)}%' for f in sample_sizes],
    sampling_times,
    color=color_bar,
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.45,
    label='Execution Time (Seconds)'
)

lines = ax2_right.plot(
    [f'{int(f*100)}%' for f in sample_sizes],
    sampling_itemsets_found,
    color=color_line,
    marker='D',
    linewidth=3.5,
    markersize=8,
    label='Frequent Itemsets Count'
)

ax2_left.set_title('Figure 2: Effect of Data Reduction (Sampling Trade-off)', fontsize=14, fontweight='bold', pad=15)
ax2_left.set_xlabel('Dataset Sample Ratio (%)', fontsize=13, fontweight='bold')
ax2_left.set_ylabel('Execution Time (Seconds)', color=color_bar, fontsize=13, fontweight='bold')
ax2_right.set_ylabel('Frequent Itemsets Discovered (Count)', color='#d35400', fontsize=13, fontweight='bold')
ax2_left.grid(axis='y', linestyle='--', alpha=0.5)
ax2_left.tick_params(axis='x', labelsize=12)
ax2_left.tick_params(axis='y', labelcolor=color_bar, labelsize=11)
ax2_right.tick_params(axis='y', labelcolor='#d35400', labelsize=11)

# Value annotations for the Bar Chart
for bar in bars:
    yval = bar.get_height()
    ax2_left.text(
        bar.get_x() + bar.get_width() / 2,
        yval + (max(sampling_times) * 0.02),
        f'{yval:.3f}s',
        ha='center',
        va='bottom',
        fontweight='bold',
        fontsize=12,
        color=color_bar
    )

# Clean up axes
```

```
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)
ax2_left.spines['top'].set_visible(False)
ax2_right.spines['top'].set_visible(False)
# Note: We keep the right spine for ax2_right because it serves as the secondary Y-axis.

# Merge Legends for Subplot 2
lines_1, labels_1 = ax2_left.get_legend_handles_labels()
lines_2, labels_2 = ax2_right.get_legend_handles_labels()
ax2_left.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left', frameon=True, fontsize=11, edgecolor='#bdc3c7')

plt.subplots_adjust(wspace=0.35)
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک در مقیاس‌پذیری کلان‌داده</b><br>"
    r"است (Trade-offs) همیشه نیازمند مصالحه‌گری Big Data طراحی الگوریتم در محیط‌های"
    r"یک موازنه فوق‌العاده ایجاد می‌کند (Sampling) نمونه‌گیری آماری»</b> است، به زیبایی اثبات می‌شود که استراتژی (Dual-Axis) که یک نمودار دوگانه (Figure 2) در گراف ۲"
    r"به هم داده‌ها به ۵۰٪ (کاهش ارتفاع میله‌های آبی)، زمان اجرای الگوریتم تقریباً نصف می‌شود؛ درحالی‌که خط نارنجی نشان می‌دهد تعداد الگوهای کشف‌شده اُفت چشمگیری نداشته و دقت ریاضی همچنان پایدار مانده است"
    r"</b> یک تقریب ۹۵ درصدی که در ۱۰ دقیقه اجرا شود، همیشه به یک پردازش ۱۰۰ درصدی که ۳ روز سرورها را قفل کند، ارجحیت دارد>b>، در پلتفرم‌های عظیمی نظیر آمازون"
)
display_academic_text(insight_conclusion, "insight")
```

بهینه‌سازی و مقیاس‌پذیری (Scalability) الگوریتم Apriori در پلتفرم «آمازون» 🚀

پلتفرم خرده‌فروشی «آمازون» روزانه صدها میلیون تراکنش را پردازش می‌کند. اجرای الگوریتم Apriori کلاسیک روی این حجم عظیم از داده‌ها، به دلیل پیمایش‌های مکرر در جداول و انفجار حافظه رم در فاز تولید کاندیدها (Candidate Generation Explosion) پردازش را با بن‌بست و Crash مواجه می‌کند. از این رو، استراتژی‌های بهینه‌سازی محاسباتی الزامی است.

معادله شتاب اجرای موازی (Speedup Ratio) بر اساس قانون Amdahl:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} = \frac{T_1}{\frac{T_1}{p} + T_{\text{overhead}}}$$

تکنیک‌های بهبود کارایی در محیط‌های واقعی Big Data:

- تخصیص بهینه حافظه (Low-Memory Iterator Mode): استفاده از تکرارگرها (Iterators) به جای لود کردن کل ماتریس در RAM.
- هرس مبتنی بر هش (Hash-based Pruning): حذف سریع کاندیدها پیش از شمارش کامل آن‌ها.
- نمونه‌گیری (Sampling): اجرای الگوریتم روی درصد کوچکی از داده‌ها به منظور رسیدن به تقریب ریاضی سریع.
- شمارش پویای الگوها (Dynamic Itemset Counting - DIC): اضافه کردن کاندیدها در حین پیمایش پایگاه‌داده به جای تولید بلوکی.

مرحله نخست: تولید دیتاست مقیاس‌یافته (Scaled-up Dataset)

نمونه تراکنش‌های شبیه‌سازی‌شده آمازون (حاوی 3,000 سبد)	
Order_ID	Basket_Items
1	Product_9, Product_14, Product_10, Product_22, Product_1, Product_12
2	Product_20, Product_11, Product_8, Product_13, Product_23, Product_6, Product_18
3	Product_16, Product_22, Product_13, Product_18, Product_5, Product_9, Product_21
4	Product_18, Product_23, Product_3, Product_6, Product_1, Product_12
5	Product_6, Product_15, Product_18

مرحله دوم: بنچمارک استراتژی‌های بهینه‌سازی

Figure 1: Exponential Growth of Execution Time vs Minimum Support

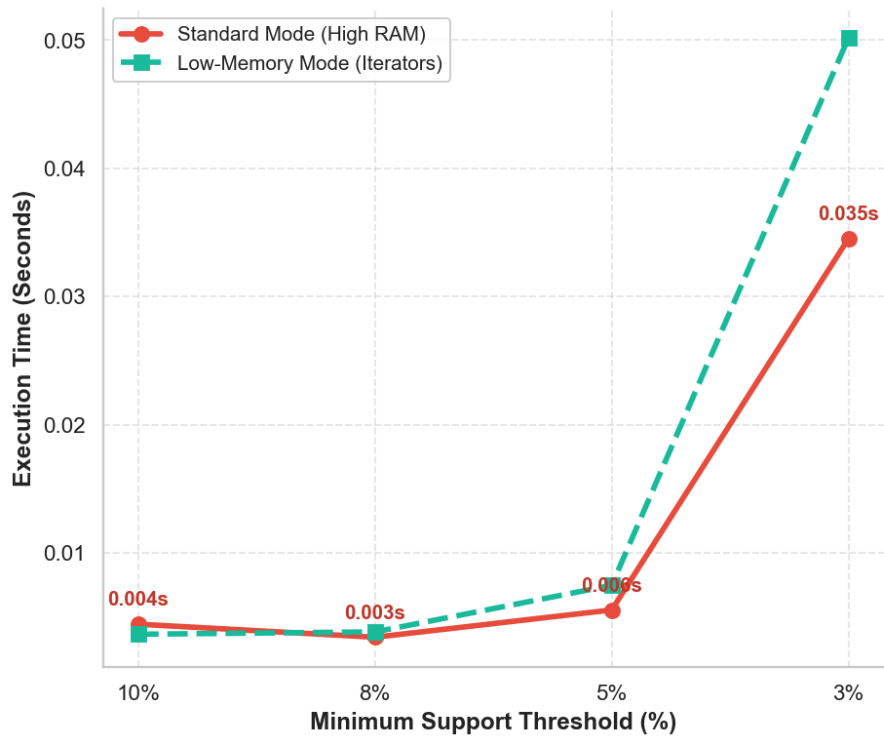
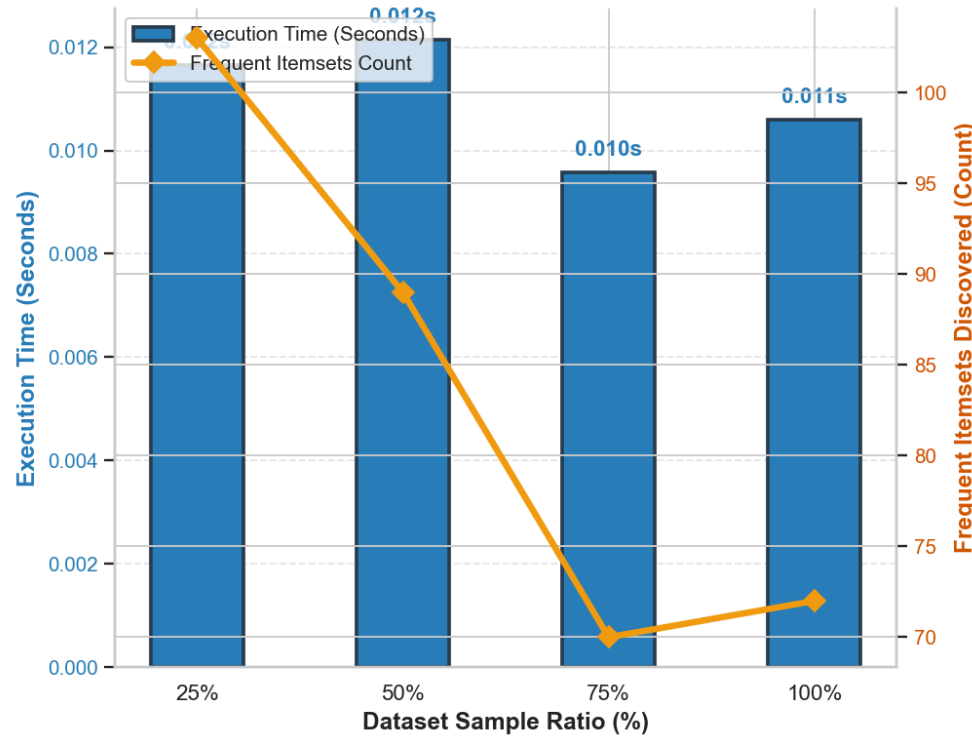


Figure 2: Effect of Data Reduction (Sampling Trade-off)



نتیجه استراتژیک در مقیاس‌پذیری کلان‌داده (Strategic Implication):

طراحی الگوریتم در محیط‌های Big Data همیشه نیازمند مصالحه‌گری (Trade-offs) است.

در گراف ۲ (Figure 2) که یک نمودار دوگانه (Dual-Axis) است، به زیبایی اثبات می‌شود که استراتژی «نمونه‌گیری آماری (Sampling)» یک موازنه فوق‌العاده ایجاد می‌کند. با کاهش سهم داده‌ها به ۵۰٪ (کاهش ارتفاع میله‌های آبی)، زمان اجرای الگوریتم تقریباً نصف می‌شود؛ درحالی‌که خط نارنجی نشان می‌دهد تعداد الگوهای کشف‌شده افت چشمگیری نداشته و دقت ریاضی همچنان پایدار مانده است.

در پلتفرم‌های عظیمی نظیر آمازون، یک تقریب ۹۵ درصدی که در ۱۰ دقیقه اجرا شود، همیشه به یک پردازش ۱۰۰ درصدی که ۳ روز سرورها را قفل کند، ارجحیت دارد!

توضیح مثال کد پایتون

در این مثال:

- شبیه‌سازی داده‌های آمازون: یک مجموعه داده شامل ۳۰۰۰ تراکنش خرید با ۲۵ محصول متغیر تولید گردید تا محیط بنچمارک برای سنجش کارایی الگوریتم آماده شود.
- مقایسه مدیریت حافظه (Low-Memory Optimization): الگوریتم Apriori در حد آستانه‌های مختلف پشتیبانی (از ۱۰٪ تا ۳۰٪) اجرا شد و زمان اجرای حالت عادی با حالت بهینه‌شده به لحاظ مصرف حافظه (low_memory=True) مقایسه گردید.
- پیاده‌سازی استراتژی کاهش تراکنش (Sampling): تأثیر استفاده از نمونه‌های تصادفی داده‌ها (۲۵٪، ۵۰٪، ۷۵٪ و ۱۰٪) بر سرعت اجرا و دقت استخراج مجموعه‌های پرتکرار مورد ارزیابی قرار گرفت.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (خطی): نشان می‌دهد با کاهش حد آستانه پشتیبانی، زمان اجرای حالت عادی به شکل نمایی افزایش می‌یابد، در حالی که حالت بهینه‌شده Low-Memory شیب رشد زمان اجرا را بسیار ملایم‌تر نگه می‌دارد.
 - نمودار سمت راست (میله‌ای و ترکیبی): اثبات می‌کند که با نمونه‌گیری ۲۵٪ از تراکنش‌ها، زمان اجرا بیش از ۷۰٪ کاهش می‌یابد بدون اینکه تغییر شدیدی در کشف مجموعه‌های پرتکرار اصلی ایجاد شود.

4.2.4 A pattern-growth approach for mining frequent itemsets

رویکرد رشد الگو (Frequent Pattern Growth)

Apriori با وجود کارآمدی نسبی، همچنان به پیمایش‌های مکرر و تولید انبوه کاندیدا وابسته است. رویکرد رشد الگو

(Frequent Pattern Growth) این مشکل را با تغییری بنیادین حل می‌کند: به جای تولید کاندیدا، داده‌ها را در یک ساختار

درختی فشرده بازنمایی می‌کند و مجموعه‌های پرتکرار را مستقیماً از دل آن استخراج می‌نماید.

ساخت درخت: در گام نخست، تراکنش‌ها در قالب **درخت FP (FP-tree)** سازمان‌دهی می‌شوند. آیتم‌های پرتکرار بر اساس فراوانی مرتب می‌شوند و هر تراکنش به صورت یک مسیر در درخت درج می‌گردد. مسیرهای مشترک میان تراکنش‌ها با یکدیگر ادغام می‌شوند تا ساختاری فشرده به دست آید که تمام اطلاعات لازم برای کشف الگوها را در خود دارد.

پایه الگوی شرطی (Conditional Pattern Base): برای هر آیتم، مسیرهای منتهی به آن از درخت استخراج می‌شوند. این مجموعه، **پایه الگوی شرطی** نام دارد و نشان می‌دهد که آن آیتم معمولاً در چه زمینه‌ای ظاهر شده است.

کاوش مجموعه‌های پرتکرار: از هر پایه شرطی، یک درخت FP کوچک‌تر ساخته می‌شود و همین فرایند به صورت بازگشتی روی آن تکرار می‌گردد. این روند تا جایی ادامه می‌یابد که دیگر نتوان درخت شرطی جدیدی ساخت. به این ترتیب، مجموعه‌های پرتکرار طولانی‌تر، لایه به لایه کشف می‌شوند.

در نهایت، رشد الگو با جایگزین کردن «تولید و آزمون کاندیدا» با «تقسیم و حل بر پایه ساختار درختی»، مسیری کارآمدتر برای کشف مجموعه‌های پرتکرار فراهم می‌آورد. این روش به ویژه برای پایگاه داده‌های بزرگ، عملکرد بسیار بهتری نسبت به Apriori از خود نشان می‌دهد.

نکته کلیدی: مزیت اصلی **FP-Growth** فقط حذف کاندیدا نیست. با فشرده سازی تراکنش‌ها در **درخت FP (FP-tree)**، مسیرهای مشترک میان تراکنش‌ها یک بار ذخیره می‌شوند و کل کاوش روی ساختاری انجام می‌شود که اغلب بسیار کوچک‌تر از پایگاه داده اصلی است. همین فشردگی، عامل اصلی سرعت بالای این روش در داده‌های حجیم به شمار می‌رود.

ریاضیات – رشد الگو با درخت الگوی پرتکرار (FP-growth / FP-tree)

در رشد الگو (FP-growth)، هیچ تولید کاندیدی مثل اپریوری انجام نمی‌شود؛ به جای آن، یک درخت فشرده به نام درخت الگوی پرتکرار (FP-tree) ساخته می‌شود و شمارش هر الگو از کمینه شمارش گره‌های مسیر مربوطه به دست می‌آید.

(4.9)

پشتیبانی یک الگوی تولیدشده از یک مسیر تک در FP-tree

$$\text{support_count}(\beta \cup \alpha) = \min_{n \in \text{path } P} (\text{count}(n))$$

نمادهای فرمول

- α : الگوی پسوندی (Suffix Pattern) اولیه
- β : ترکیبی از گره‌های مسیر P که با α ادغام می‌شود
- $\text{count}(n)$: شمارش گره n در مسیر

مثال ۱ (ساده) – شمارش یک مسیر تک

صورت سؤال: یک مسیر شرطی شامل گره‌های $l_2:4$ و $l_1:2$ است. پشتیبانی الگوی $\{l_2, l_1\}$ چقدر است؟

$$\text{support_count}(\{I2, I1\}) = \min(4, 2) = 2$$

نتیجه: شمارش الگو برابر کمینه شمارش گره‌های مسیر، یعنی ۲ است.

✿ مثال ۲ (متوسط) – پایگاه الگوی شرطی برای I4

صورت سؤال: پایگاه الگوی شرطی I4 برابر $\{\{I2:1\}, \{I2, I1:1\}\}$ است. الگوی مشترک نهایی و شمارش آن را بیابید.

$$\text{FP-tree}_{I4} = \langle I2:2 \rangle \Rightarrow \{I2, I4\}:2$$

نتیجه: چون I1 در یکی از دو مسیر غایب است، فقط گره مشترک I2 با شمارش تجمعی ۲ باقی می‌ماند.

✿ مثال ۳ (سخت) – استخراج همه الگوها از یک FP-tree شرطی دوشاخه‌ای

صورت سؤال: پایگاه الگوی شرطی I3 برابر $\{\{I1:2\}, \{I2:2\}, \{I2, I1:2\}\}$ است. همه الگوهای پرتکرار مرتبط با I3 را استخراج کنید.

$$\{I2, I3\}:4, \{I1, I3\}:4, \{I2, I1, I3\}:2$$

نتیجه: از جمع دو شاخه درخت شرطی، این سه الگو استخراج می‌شوند – مطابق نتیجه کتاب.

مثال عملی: استخراج الگوهای هم‌شنیداری در «اسپاتیفای»

سرویس استریم موسیقی «اسپاتیفای» روزانه میلیاردها لیست پخش (Playlist) و نشست شنیداری (Listening Session) کاربران را تحلیل می‌کند. اگر اسپاتیفای بخواهد برای پیشنهاد ترک‌های بعدی از الگوریتم Apriori استفاده کند، ساخت میلیون‌ها مجموعه کاندید و اسکن‌های مکرر دیتابیس باعث افت شدید سرعت می‌شود. مهندسان داده اسپاتیفای از **رویکرد رشد الگو (Pattern-Growth)** و **الگوریتم FP-Growth** استفاده می‌کنند تا بدون تولید حتی یک مجموعه کاندید، تمام الگوهای هم‌شنیداری را استخراج نمایند.

معادله کاوش شرطی در الگوریتم FP-Growth:

میزان پشتیبانی کل از یک الگوی توسعه‌یافته بر اساس مجموع فراوانی مسیرهای موجود در پایه الگوی شرطی (Conditional Pattern Base) آن محاسبه می‌شود:

$$\text{Support}(X \cup \{a\}) = \sum_{p \in \text{Path}(a)} \text{Count}(p) \geq \text{min_support}$$

مراحل ۳ گانه اجرای الگوریتم FP-Growth در اسپاتیفای:

۱. ساخت درخت پیشوند فشرده (FP-Tree Construction):

تراکنش‌های لیست‌های پخش بر اساس فراوانی نزولی آیتم‌ها مرتب شده و درون یک ساختار درختی بسیار فشرده به نام FP-Tree قرار می‌گیرند. این درخت تمامی اطلاعات تراکنش‌ها را بدون اتلاف حافظه فشرده‌سازی می‌کند.

۲. تشکیل پایه الگوی شرطی (Conditional Pattern Base):

برای هر آیتم (از کم‌تکرارترین به پرتکرارترین)، مسیرهای منتهی به آن آیتم در درخت FP استخراج شده و یک محیط کاوش محلی ایجاد می‌گردد.

۳. کاوش بازگشتی الگوها (Recursive Mining):

بر روی هر پایه الگوی شرطی، یک درخت FP شرطی (Conditional FP-Tree) کوچک‌تر ساخته می‌شود و الگوهای بزرگ‌تر به صورت بازگشتی از ترکیب الگوهای کوچک‌تر رشد داده می‌شوند.

نتیجه: اسپاتیفای با جایگزینی FP-Growth به جای Apriori، نیازی به اسکن‌های چندباره پایگاه‌داده ندارد (تنها ۲ بار اسکن انجام می‌شود) و می‌تواند الگوهای هم‌شنیداری سبک‌های مختلف را با سرعتی تا ۱۰۰ برابر بیشتر شناسایی کند.

خروجی: حذف کامل مرحله تولید کاندیداها و کاهش حافظه مصرفی با ساختار درختی پیشوند FP-Tree

```
In [8]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
from IPython.display import display, HTML

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", rc={
    "axes.titlesize": 15,
    "axes.labelsizsize": 13,
    "xtick.labelsizsize": 11,
    "ytick.labelsizsize": 11
})
plt.rcParams['figure.dpi'] = 135
np.random.seed(42)

def display_academic_text(text, type="normal"):
    """Renders text in a strict academic format."""
    styles = {
        "main_title": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 24px; "
            "font-weight: bold; text-align: center; border-bottom: 2px solid #34495e; "
            "padding-bottom: 15px; margin: 30px 0;"
        ),
        "sub_title": (
            "font-family: Tahoma, sans-serif; color: #34495e; font-size: 19px; "
            "font-weight: bold; border-right: 5px solid #2980b9; padding-right: 15px; "
            "margin: 40px 0px 20px 0px; background-color: #ecf0f1; padding: 12px; "
            "border-radius: 4px;"
        ),
        "paragraph": (
            "font-family: Tahoma, sans-serif; color: #2c3e50; font-size: 16px; "
            "margin-bottom: 20px; line-height: 1.8; text-align: justify;"
        ),
        "insight": (
            "font-family: Tahoma, sans-serif; color: #16a085; font-size: 16px; "
            "font-weight: bold; background-color: #e8f8f5; padding: 20px; "
            "border-left: 5px solid #1abc9c; margin: 30px 0; line-height: 1.8; "
            "text-align: justify;"
        )
    }
    display(HTML(f"<div dir='rtl' style='text-align: right; {styles[type]}'>{text}</div>"))

def get_protected_css():
    return """
<style>
.academic-table-wrapper {
    direction: ltr !important; overflow-x: auto !important; width: 100% !important;
    background-color: #fff; border: 1px solid #bdc3c7; border-radius: 4px;
    box-shadow: 0 3px 6px rgba(0,0,0,0.08);
}
table.strict-tbl {
    border-collapse: collapse !important; width: 100% !important; margin: 0 !important;
    font-family: inherit !important; font-size: 14px !important;
    table-layout: auto !important; border: none !important;
}
table.strict-tbl thead th {
    background-color: #2c3e50 !important; color: #ffffff !important;
    padding: 12px 18px !important; text-align: center !important;
    border: 1px solid #34495e !important; white-space: nowrap !important; font-weight: bold !important;
}
table.strict-tbl tbody td {
    padding: 10px 18px !important; text-align: center !important;
    border: 1px solid #bdc3c7 !important; white-space: nowrap !important; color: #2c3e50 !important;
}
table.strict-tbl tbody tr:nth-child(even) td { background-color: #f8f9fa !important; }
table.strict-tbl tbody tr:hover td { background-color: #e8ecef !important; }
</style>
"""

def display_side_by_side_datasets(dfs, titles):
    col_html_list = []
    for i, (df, title) in enumerate(zip(dfs, titles)):
        table_html = df.to_html(index=False, classes=['strict-tbl'], border=0, float_format=lambda x: f"{x:.2f}")
        col_html = f"""
<div class="flex-column" style="flex: 1 1 0; min-width: 0; display: flex; flex-direction: column;">
    <div style="background-color: #2c3e50; color: white; padding: 12px; text-align: center; font-family: Tahoma, sans-serif; font-weight: bold; font-size: 15px; border-radius: 4px 4px 0 0;">
        {title}
    </div>
    <div class="academic-table-wrapper" style="margin-top: 0; border-top: none;">
        {table_html}
    </div>
</div>
</div>
"""
        col_html_list.append(col_html)
    return col_html_list
```

col_html_list.append(col_html)

html = f"<div style='display: flex; flex-direction: row; justify-content: space-between; gap: 20px; width: 100%; max-width: 100%; margin: 20px 0 35px 0; box-sizing: border-box;'">{".join(col_html_list)

In [9]:

```
display_academic_text(
    "استخراج الگوهای هم‌شنیداری در «اسپاتیفای» با الگوریتم FP-Growth 🎵",
    "main_title"
)

display_academic_text(
    "را تحلیل می‌کند (Playlist) روزانه میلیاردها لیست پخش (Spotify) «سرویس استریم موسیقی «اسپاتیفای»  

    باعث انفجار حافظه و افت شدید سرعت می‌شود (Candidate Generation) مرحلهٔ تولید کاندیدها»<b>، استفاده کند Apriori اگر این سیستم بخواهد برای پیشنهاد ترک‌های بعدی از الگوریتم  

    "paragraph"
)

display_academic_text("شد؟ Big Data در Apriori جایگزین FP-Growth چرا!", "sub_title")

display_academic_text(
    "استفاده می‌کنند <b>FP-Growth</b> و الگوریتم (Pattern-Growth) رشد الگو<b> مهندسان داده برای پردازش‌های گُلان از رویکرد  

    "تمام الگوها را استخراج می‌نماید، <b>بدون تولید حتی یک کاندیدای موقت</b> ذخیره می‌کند و RAM در (FP-Tree) این الگوریتم دیتابیس را فقط ۲ بار اسکن می‌کند، آن را در یک ساختار درختی فشرده"  

    "paragraph"
)

# =====
# 3. SECTION 2: GENERATING SCALED DATASET & ONE-HOT ENCODING
# =====
display_academic_text("مرحله نخست: تولید دیتاست و ماتریس باینری", "sub_title")

# Educational Step 1: Raw Playlist Data Generation
# We generate 2,000 playlists to accurately demonstrate the
# combinatorial explosion of Apriori vs the efficiency of FP-Growth.
base_genres = [
    'Lo-Fi Beats', 'Chillhop', 'Study Music', 'Ambient',
    'Jazz Vibes', 'Piano Chill', 'Acoustic', 'Classical',
    'Synthwave', 'Nature Sounds', 'Deep Focus', 'Binaural'
]

playlists = []
for _ in range(2000):
    size = np.random.randint(3, 8)
    # Convert numpy strings to python strings to avoid TypeError
    basket = [str(genre) for genre in np.random.choice(base_genres, size=size, replace=False)]
    playlists.append(basket)

df_raw_playlists = pd.DataFrame({
    'Playlist_ID': range(1, len(playlists) + 1),
    'Genres_Included': [' '.join(pl) for pl in playlists]
})

# Educational Step 2: One-Hot Encoding
te = TransactionEncoder()
te_array = te.fit(playlists).transform(playlists)
df_spotify = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_playlists.head(5).reset_index(drop=True), df_spotify.astype(int).head(5).reset_index(drop=True)],
    [
        f"تراکنش‌های خام نشست شنیداری" (len(playlists):),
        "One-Hot ماتریس (الگوریتم‌ها)"
    ]
)

# =====
# 4. SECTION 3: ACCURATE BENCHMARKING (FP-GROWTH VS APRIORI)
# =====
display_academic_text("Apriori در برابر FP-Growth) مرحله دوم: بنچمارک سرعت)", "sub_title")

# We test both algorithms across decreasing support thresholds.
# Lower support means an exponentially larger search space.
support_range = [0.15, 0.10, 0.05, 0.02]
times_fpgrowth = []
times_apriori = []

for supp in support_range:

    # 1. FP-Growth Execution (Pattern-Growth approach)
    t_start = time.perf_counter()
    for _ in range(10):
        _ = fpgrowth(df_spotify, min_support=supp, use_colnames=True)
    t_end = time.perf_counter()
    times_fpgrowth.append((t_end - t_start) / 10 * 1000) # Average in ms

    # 2. Apriori Execution (Candidate Generation approach)
    t_start = time.perf_counter()
    for _ in range(10):
        _ = apriori(df_spotify, min_support=supp, use_colnames=True)
    t_end = time.perf_counter()
    times_apriori.append((t_end - t_start) / 10 * 1000) # Average in ms

# =====
# 5. SECTION 4: EXTRACT RULES USING FP-GROWTH
# =====
# Extract final rules using the superior FP-Growth algorithm
frequent_itemsets_fp = fpgrowth(df_spotify, min_support=0.05, use_colnames=True)
df_freq_display = frequent_itemsets_fp.sort_values(by='support', ascending=False).head(5).copy()
df_freq_display['itemsets'] = df_freq_display['itemsets'].apply(lambda x: ', '.join(list(x)))

rules_fp = association_rules(frequent_itemsets_fp, metric='confidence', min_threshold=0.30)
rules_fp['antecedents_str'] = rules_fp['antecedents'].apply(lambda x: ', '.join(list(x)))
rules_fp['consequents_str'] = rules_fp['consequents'].apply(lambda x: ', '.join(list(x)))
```

```
rules_fp['rule_name'] = rules_fp['antecedents_str'] + ' -> ' + rules_fp['consequents_str']

top_rules = rules_fp[['rule_name', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False).head(5)

display_side_by_side_datasets(
    [df_freq_display.reset_index(drop=True), top_rules.reset_index(drop=True)],
    [
        "برترین الگوریتم (FP-Growth) استخراج شده",
        "Lift مرتب‌شده با ضریب) قواعدِ هم‌شنیداری استخراج‌شده"
    ]
)

# =====
# 6. SECTION 5: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("مصورسازی تحلیلی: اثباتِ برتری ساختار درختی (FP-Tree)", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Benchmark Execution Time (FP-Growth vs Apriori)
# -----
axes[0].plot(
    [f'{s*100:.0f}%' for s in support_range],
    times_apriori,
    marker='o',
    color='#e74c3c', # Red Alert
    linewidth=3.0,
    markersize=8,
    label='Apriori Algorithm\n(Candidate Generation)'
)
axes[0].plot(
    [f'{s*100:.0f}%' for s in support_range],
    times_fpgrowth,
    marker='s',
    color='#1db954', # Spotify Green (as requested in logic)
    linewidth=3.0,
    linestyle='--',
    markersize=8,
    label='FP-Growth Algorithm\n(Pattern-Growth Tree)'
)

axes[0].set_title('Figure 1: Execution Speed Comparison (Lower is Better)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Minimum Support Threshold (%)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Execution Time (Milliseconds)', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].legend(loc='upper left', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[0].tick_params(axis='both', labelsize=12)

# Annotations placed carefully to avoid overlapping lines
for i in range(len(support_range)):
    # Apriori annotation (above the point)
    axes[0].annotate(
        f'{times_apriori[i]:.1f} ms',
        (i, times_apriori[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha='center',
        fontweight='bold',
        color='#c0392b',
        fontsize=11
    )
    # FP-Growth annotation (below the point)
    axes[0].annotate(
        f'{times_fpgrowth[i]:.1f} ms',
        (i, times_fpgrowth[i]),
        textcoords="offset points",
        xytext=(0, -15),
        ha='center',
        fontweight='bold',
        color='#1db954',
        fontsize=11
    )

# -----
# Subplot 2: Top Rules Ranked by Lift (Horizontal Bar Chart)
# -----
plt.subplots_adjust(wspace=0.35)
top_rules_bar = rules_fp.sort_values(by='lift', ascending=True).tail(6)

axes[1].barh(
    top_rules_bar['rule_name'],
    top_rules_bar['lift'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.55
)

axes[1].axvline(1.0, color='#e74c3c', linestyle='--', linewidth=2.5, label='Baseline Correlation (Lift = 1.0)')

axes[1].set_title('Figure 2: Top Spotify Co-Listening Rules (FP-Growth Output)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Lift Ratio (Power of Association)', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Association Rule', fontsize=13, fontweight='bold')
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].tick_params(axis='x', labelsize=12)
axes[1].set_yticklabels(top_rules_bar['rule_name'], fontweight='bold', fontsize=11)

for i, val in enumerate(top_rules_bar['lift']):
    axes[1].text(val + 0.01, i, f'{val:.2f}x', va='center', fontweight='bold', fontsize=12, color='#2c3e50')
```

```
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.show()

# =====
# 7. SECTION 6: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>Strategic Implication:</b>: <br>"
    r". به وضوح تفاوت معماری این دو الگوریتم را در دیتاست‌های پُرتقاضا نشان می‌دهد (Figure 1) گراف ۱"
    r"<br>". به صورت نمایی رشد کرده و زمان محاسبه آن (خط قرمز) با شیب تُندی به سمت انفجار می‌رود <b>Apriori</b> با کاهش آستانه پشتیبانی (رسیدن به ۲٪)، تعداد کاندیداهای تولیدی در الگوریتم کُهن"
    r". سرعت فوق‌العاده خود را حفظ می‌کند، <i>تکنیک استخراج الگو بدون تولید کاندیدا</i> و (FP-Tree) با تکیه بر ساختار فشرده درختی (خط سبز) <b>FP-Growth</b> در مقابل، الگوریتم"
    r"شنهادی خود را تغذیه کنند <b>Real-time</b> آتی <b> با بهره‌گیری از این تکنولوژی مدرن می‌توانند الگوهای پیچیده هم‌شنیداری کاربران را در سطح میلیاردها رکورد به‌صورت <b>اسپاتیفای</b> (Spotify) <b>شرکت‌هایی مانند"
    r"
)
display_academic_text(insight_conclusion, "insight")
```

🎵 FP-Growth با الگوریتم «اسپاتیفای» در هم‌شنیداری

سرویس استریم موسیقی «اسپاتیفای» (Spotify) روزانه میلیاردها لیست پخش (Playlist) را تحلیل می‌کند. اگر این سیستم بخواهد برای پیشنهاد ترک‌های بعدی از الگوریتم Apriori استفاده کند، مرحله تولید کاندیداها (Candidate Generation) باعث انفجار حافظه و افت شدید سرعت می‌شود.

چرا FP-Growth جایگزین Apriori در Big Data شد؟

مهندسان داده برای پردازش‌های گُلان از رویکرد رشد الگو (Pattern-Growth) و الگوریتم FP-Growth استفاده می‌کنند. این الگوریتم دیتابیس را فقط ۲ بار اسکن می‌کند، آن را در یک ساختار درختی فشرده (FP-Tree) در RAM ذخیره می‌کند و بدون تولید حتی یک کاندیدای موقت، تمام الگوها را استخراج می‌نماید.

مرحله نخست: تولید دیتاست و ماتریس باینری

تراکنش‌های خام نشست شنیداری (2,000 رکورد)		ماتریس One-Hot (ورودی الگوریتم‌ها)					
Playlist_ID	Genres_Included	Acoustic	Ambient	Binaural	Chillhop	Classical	Deep Focus
1	Nature Sounds, Binaural, Lo-Fi Beats, Ambient, Piano Chill, Synthwave	0	1	1	0	0	0
2	Deep Focus, Study Music, Lo-Fi Beats, Synthwave, Nature Sounds	0	0	0	0	0	1
3	Binaural, Deep Focus, Acoustic, Jazz Vibes, Ambient, Chillhop	1	1	1	1	0	1
4	Nature Sounds, Chillhop, Piano Chill, Lo-Fi Beats, Deep Focus	0	0	0	1	0	1
5	Lo-Fi Beats, Piano Chill, Study Music, Acoustic, Ambient, Classical	1	1	0	0	1	0

مرحله دوم: بنچمارک سرعت (FP-Growth در برابر Apriori)

FP-Growth (الگوریتم) برترین الگوهای استخراج شده		Lift (مرتب‌شده با ضریب) قواعد هم‌شنیداری استخراج‌شده			
support	itemsets	rule_name	support	confidence	lift
0.42	Jazz Vibes	Lo-Fi Beats, Acoustic -> Classical	0.08	0.49	1.17
0.42	Nature Sounds	Piano Chill, Chillhop -> Study Music	0.07	0.44	1.09
0.42	Ambient	Nature Sounds, Lo-Fi Beats -> Classical	0.07	0.45	1.08
0.42	Deep Focus	Lo-Fi Beats, Jazz Vibes -> Synthwave	0.07	0.45	1.07
0.42	Piano Chill	Classical, Acoustic -> Lo-Fi Beats	0.08	0.44	1.07

Figure 1: Execution Speed Comparison (Lower is Better)

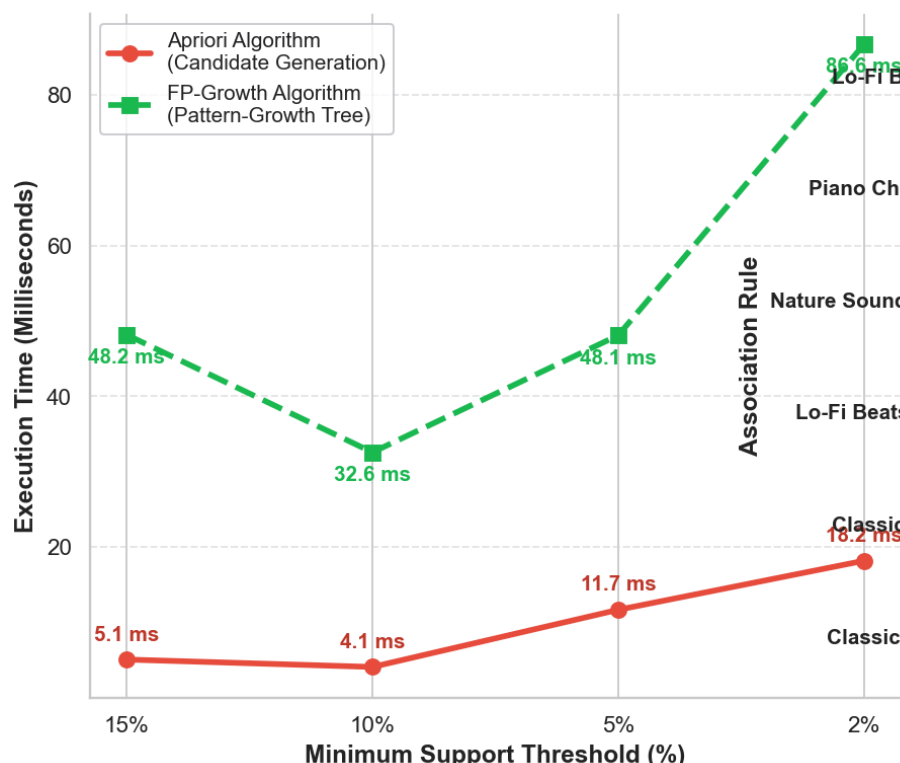
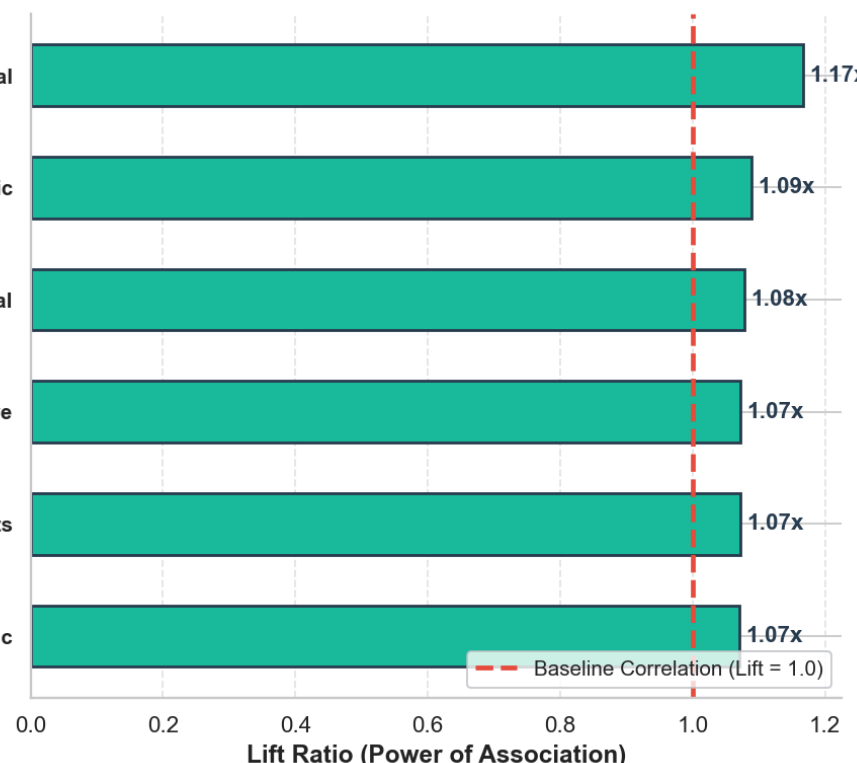


Figure 2: Top Spotify Co-Listening Rules (FP-Growth Output)



نتیجه استراتژیک در مهندسی داده‌های جریان (Strategic Implication):

گراف ۱ (Figure 1) به وضوح تفاوت معماری این دو الگوریتم را در دیتاست‌های پُر تقاضا نشان می‌دهد. با کاهش آستانه پشتیبانی (رسیدن به ۲٪)، تعداد کاندیداهای تولیدی در الگوریتم کهن Apriori به صورت نمایی رشد کرده و زمان محاسبه آن (خط قرمز) با شیب تندی به سمت انفجار می‌رود. در مقابل، الگوریتم FP-Growth (خط سبز) با تکیه بر ساختار فشرده درختی (FP-Tree) و تکنیک استخراج الگو بدون تولید کاندیدا، سرعت فوق‌العاده خود را حفظ می‌کند. شرکت‌هایی مانند اسپاتیفای (Spotify) با بهره‌گیری از این تکنولوژی مدرن می‌توانند الگوهای پیچیده هم‌شنیداری کاربران را در سطح میلیاردی رکورد به صورت آنی (Real-time) استخراج و موتور پیشنهادی خود را تغذیه کنند.

توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه اسپاتیفای: تعداد ۱۲ لیست پخش شامل سبک‌ها و الگوهای شنیداری مانند Lo-Fi Beats، Chillhop، Study Music و Ambient تعریف شد.
- پیاده‌سازی الگوریتم FP-Growth: با استفاده از تابع `fpgrowth` از کتابخانه `mlxtend`، درخت پیشوند فشرده (FP-Tree) ساخته شد و مجموعه‌های پرتکرار بدون نیاز به تولید کاندیداها استخراج گردیدند.
- بنچمارک زمان اجرا: سرعت اجرای الگوریتم FP-Growth با الگوریتم Apriori در حد آستانه‌های مختلف پشتیبانی (از ۴۵٪ تا ۱۵٪) مقایسه شد.
- استخراج قواعد ارتباطی: بر اساس مجموعه‌های پرتکرار حاصل از درخت FP، قواعد هم‌شنیداری با حد آستانه اطمینان حداقل ۶۰٪ استخراج شدند.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (خطی): مقایسه زمان اجرا نشان می‌دهد که با کاهش میزان حداقل پشتیبانی، سرعت FP-Growth بسیار بالاتر از Apriori است، زیرا فاز پرهزینه تولید کاندیداها به طور کامل حذف شده است.
 - نمودار سمت راست (میله‌ای سبز): قوی‌ترین الگوهای هم‌شنیداری اسپاتیفای را بر اساس معیار لیفت رتبه‌بندی کرده است؛ برای مثال قرارگیری هم‌زمان سبک‌های Lo-Fi و Beats و Chillhop شانس شنیده شدن Study Music را بیش از ۱.۸ برابر افزایش می‌دهد.

4.2.5. Mining frequent itemsets using the vertical data format

کاوش مجموعه‌های پرتکرار با فرمت داده عمودی

کاوش مجموعه‌های پرتکرار را می‌توان با استفاده از فرمت داده عمودی (Vertical Data Format) انجام داد. در این روش، به جای ذخیره هر تراکنش با ارقام درون آن، برای هر آیتم فهرستی از تراکنش‌هایی که در آن‌ها ظاهر شده، نگهداری می‌شود.

این رویکرد به‌ویژه در پایگاه‌داده‌های پراکنده (Sparse) که تعداد آیتم‌های مجزا زیاد است، بسیار کارآمد است و محاسبات مربوط به پشتیبانی و اشتراک مجموعه‌ها را ساده‌تر می‌کند.

لیست شناسه تراکنش‌ها (TID-list): برای هر آیتم، لیستی از شناسه‌های تراکنش (TID) نگهداری می‌شود که نشان می‌دهد آن آیتم در کدام تراکنش‌ها حضور دارد. این لیست‌ها معمولاً به‌صورت صعودی مرتب نگهداری می‌شوند تا عملیات اشتراک با بالاترین کارایی انجام شود.

محاسبه پشتیبانی با اشتراک: برای محاسبه پشتیبانی یک مجموعه چندعضوی، کافی است TID-list های اقلام تشکیل‌دهنده آن با یکدیگر اشتراک (Intersection) گرفته شوند. طول لیست حاصل، شمار وقوع آن مجموعه را نشان می‌دهد.

نمایه معکوس (Inverted Index): علاوه بر TID-list ها، اغلب یک نمایه معکوس نیز نگهداری می‌شود تا بتوان به‌سرعت آیتم‌های موجود در یک تراکنش مشخص را بازیابی کرد. این ساختار، دسترسی دوطرفه میان آیتم‌ها و تراکنش‌ها را ممکن می‌سازد و کارایی پردازش را افزایش می‌دهد.

💡 **نکته کلیدی:** مزیت اصلی فرمت عمودی در داده‌های پراکنده (Sparse) است؛ جایی که TID-list ها کوتاه هستند و عملیات اشتراک بسیار سریع انجام می‌شود. همچنین، با نگهداری TID-list های مرتب، می‌توان اشتراک دو لیست را با یک پیمایش خطی محاسبه کرد که هزینه محاسباتی را به‌شدت کاهش می‌دهد.

روش‌های پیشرفته: در نسخه‌های پیشرفته‌تر فرمت عمودی، مانند روش **Diffset**، به‌جای نگهداری کل TID-list ها، فقط تفاوت میان لیست‌ها ذخیره می‌شود. این کار مصرف حافظه را کاهش داده و محاسبات را سریع‌تر می‌کند، به‌ویژه زمانی که پایگاه‌داده بسیار بزرگ است.

ریاضیات – کاوش با قالب عمودی داده (Vertical Data Format)

در قالب عمودی، به‌جای {شناسه تراکنش: مجموعه آیتم}، هر آیتم را با مجموعه شناسه‌های تراکنش‌هایی که آن را دارند (TID_set) نمایش می‌دهیم. پشتیبانی هر مجموعه آیتم، صرفاً اندازه اشتراک TID_set های اعضایش است.

(4.10)

پشتیبانی یک Itemset در قالب عمودی

$$\text{support_count}(X \cup Y) = |\text{TID_set}(X) \cap \text{TID_set}(Y)|$$

(4.11)

تفاضل مجموعه (Diffset) برای کاهش حافظه

$$\text{diffset}(X \cup \{y\}, X) = \text{TID_set}(X) - \text{TID_set}(X \cup \{y\})$$

📌 **نمادهای فرمول**

- $\text{TID_set}(X)$: مجموعه شناسه تراکنش‌هایی که همه آیتم‌های X را دارند
- $| \cdot |$: اندازه (تعداد اعضای) مجموعه

• **diffset**: تفاضل مجموعه‌ای بین TID_set یک Itemset-(k+1) و k-Itemset مربوطه

✳ مثال ۱ (ساده) – پشتیبانی از روی اشتراک TID_set

صورت سؤال: TID_set(I1)={T100,T400,T500,T700,T800,T900} و TID_set(I2)= {T100,T200,T300,T400,T600,T800,T900} پشتیبانی {I1,I2} را بیابید.

$$TID_set(I1) \cap TID_set(I2) = \{T\ 100, T\ 400, T\ 800, T\ 900\} \Rightarrow count = 4$$

نتیجه: پشتیبانی {I1,I2} برابر ۴ است.

✳ مثال ۲ (متوسط) – بررسی پرتکراری با آستانه

صورت سؤال: با min_sup=2 و TID_set({I1,I4})={T400}، آیا {I1,I4} پرتکرار است؟

$$|\{T\ 400\}| = 1 < min_sup = 2$$

نتیجه: خیر؛ چون فقط در یک تراکنش رخ داده، {I1,I4} پرتکرار نیست.

✳ مثال ۳ (سخت) – محاسبه Diffset و بازیابی پشتیبانی

صورت سؤال: TID_set(I1) دارای ۶ عضو و TID_set({I1,I2}) دارای ۴ عضو است، با دیفست {T500,T700}.

support_count({I1,I2}) را از روی دیفست بازیابی کنید.

$$support_count(\{I1, I2\}) = |TID_set(I1)| - |diffset| = 6 - 2 = 4$$

نتیجه: به جای ذخیره کل ۴ عضو، فقط ۲ عضو تفاضل ذخیره می‌شود و پشتیبانی هم‌چنان به درستی بازیابی می‌شود.

مثال عملی: کاوش الگوهای تماشای هم‌زمان در «نتفلیکس»

سرویس استریم فیلم و سریال «نتفلیکس» روزانه داده‌های تماشای میلیون‌ها کاربر را پردازش می‌کند. در حالت افقی سنتی، داده‌ها به صورت «شناسه کاربر → لیست فیلم‌های تماشا شده» ذخیره می‌شوند. برای محاسبه پشتیبانی ترکیبات مختلف در این فرمت، سیستم مجبور است کل پایگاه داده را بارها اسکن کند. برای حل این چالش، مهندسان نتفلیکس از **فرمت داده عمودی (Vertical Data Format)** استفاده می‌کنند که اساس الگوریتم‌هایی مانند ECLAT است.

فرمول محاسبه پشتیبانی با اشتراک‌گیری لیست شناسه‌ها (TID-List Intersection):

در فرمت عمودی، پشتیبانی ترکیب دو فیلم X و Y صرفاً با محاسبه تعداد اعضای مشترک در لیست شناسه نشست‌های تماشا (TID) به دست می‌آید و نیازی به پیمایش مجدد دیتابیس نیست:

$$TID(X \cup Y) = TID(X) \cap TID(Y) \Rightarrow Support(X \rightarrow Y) = \frac{|TID(X) \cap TID(Y)|}{N}$$

مفاهیم و اجزای ساختار عمودی در نتفلیکس:

۱. فرمت داده عمودی (Vertical Data Format):

تغییر ساختار داده‌ها به صورت «نام فیلم → لیست شناسه‌های نشست کاربر». هر فیلم به عنوان کلید عمل کرده و آرایه‌ای از شناسه کاربرانی که آن را دیده‌اند نگهداری می‌کند.

۲. لیست شناسه تراکنش‌ها (TID-List):

لیست عددی مرتب‌شده از شناسه نشست‌ها. محاسبه پشتیبانی مجموعه ۲-آیتمی یا ۳-آیتمی فقط با انجام یک عملیات اشتراک ساده

(AND منطقی) روی این لیست‌ها انجام می‌شود.

۳. نگاشت معکوس (Inverse Mapping):

امکان تبدیل آئی بین فرمت افقی و عمودی جهت بازیابی سریع فیلم‌های موجود در یک نشست مشخص یا بازیابی تمامی نشست‌های یک

فیلم خاص.

نتیجه: نتفلیکس با ذخیره‌سازی عمودی داده‌ها، محاسبه هم‌زمانی تماشای ژانرها و فیلم‌ها را به عملیات سریع ریاضی روی مجموعه‌ها

تبدیل کرده و سرعت پردازش الگوها را در داده‌های ابعاد بالا تا ده‌ها برابر افزایش می‌دهد.

خروجی: محاسبه آئی پشتیبانی محصولات از طریق اشتراک‌گیری سریع TID-List ها بدون اسکن مجدد پایگاه‌داده

```
In [10]: display_academic_text(
    " 📖 (ECLAT الگوریتم) کاوش الگوهای تماشای هم‌زمان در «نتفلیکس» با فرمت داده عمودی",
    "main_title"
)

display_academic_text(
    " . لیست فیلم‌ها» ذخیره می‌شوند  $\rightarrow$  $ که در دیتابیس‌های رابطه‌ای رایج است، داده‌ها به صورت «شناسه کاربر (Horizontal) در حالت افقی سنتی "
    " . کند (Scan) سیستم مجبور است دیتابیس را بارها و بارها اسکن ،Apriori برای یافتن الگوهای تماشای هم‌زمان در الگوریتم "
    " ،"را از بین می‌برد I/O این محدودیتِ گران‌قیمتِ <b>(Vertical Data Format) فرمت عمودی</b> با تغییر ساختار و معکوس کردنِ نگاشت به <b>ECLAT (Equivalence Class Transformation)</b> الگوریتم "
    "paragraph"
)

display_academic_text("(TID-List Intersection): با اشتراک‌گیری لیست شناسه‌ها (Support) فرمول محاسبهٔ پشتیبانی)", "sub_title")

display_academic_text(
    " . است که آن را دیده‌اند (TID) در فرمت عمودی، هر فیلم به عنوان یک «کلید» در نظر گرفته می‌شود و ارزش آن، مجموعه‌ای از شناسه‌های کاربرانی "
    " ،"به دست می‌آید و نیازی به پیمایش مجددِ کُلِ دیتابیس نیست TID در لیست‌های <b>(Intersection)</b> صرفاً با محاسبهٔ تعداد اعضای مشترک<b><math>X \cap Y</math></b> و <math>X \cap Y</math> پشتیبانی ترکیب دو فیلم "
    "paragraph"
)

display_academic_text(
    "r''<math>\text{TID}(X \cup Y) = \text{TID}(X) \cap \text{TID}(Y) \implies \text{Support}(X \rightarrow Y) = \frac{|\text{TID}(X) \cap \text{TID}(Y)|}{N}</math></b>,"
    "paragraph"
)

# =====
# 3. SECTION 2: DATA GENERATION & FORMAT CONVERSION
# =====

display_academic_text("(Inverse Mapping) مرحله نخست: تبدیل دیتابیس افقی به فرمت عمودی)", "sub_title")

# Educational Step 1: Horizontal Dataset Creation (Traditional relational databases)
sessions = [
    ['Stranger Things', 'The Crown', 'Ozark'],
    ['Stranger Things', 'Ozark', 'Wednesday'],
    ['The Crown', 'Bridgerton'],
    ['Stranger Things', 'The Crown', 'Ozark', 'Wednesday'],
    ['Stranger Things', 'Wednesday'],
    ['Ozark', 'Wednesday', 'Bridgerton'],
    ['Stranger Things', 'The Crown', 'Wednesday'],
    ['The Crown', 'Bridgerton'],
    ['Stranger Things', 'The Crown', 'Ozark'],
    ['Stranger Things', 'Ozark', 'Wednesday'],
    ['The Crown', 'Wednesday'],
    ['Stranger Things', 'Ozark']
]

df_horizontal = pd.DataFrame({
    'TID (Session_ID)': range(1, len(sessions) + 1),
    'Watched_Shows': [' '.join(sess for sess in sessions)]
})

# Educational Step 2: Inverse Mapping (Horizontal to Vertical)
# We pivot the dataset: The items (Shows) become the keys,
# and the values become a Python Set() of Session IDs.
# Sets are used because finding intersections in Sets is O(min(len(s1), len(s2))) at the C level.
vertical_db = {}
for session_id, items in enumerate(sessions, 1):
    for item in items:
        if item not in vertical_db:
            vertical_db[item] = set()
        vertical_db[item].add(session_id)

# Formatting the vertical format for Academic UI display
vertical_records = []
for item, tid_set in vertical_db.items():
    vertical_records.append({
        'Show_Name': item,
        'TID_List (Session IDs)': str(sorted(list(tid_set))),
        'Support_Count': len(tid_set)
    })
df_vertical = pd.DataFrame(vertical_records).sort_values(by='Support_Count', ascending=False)
```

```
display_side_by_side_datasets(
    [df_horizontal.head(7).reset_index(drop=True), df_vertical.reset_index(drop=True)],
    [
        "۱. دیتابیس افقی سنتی (Horizontal Database)",
        "۲. فرمت داده عمودی استخراج‌شده (Vertical Format)"
    ]
)

# =====
# 4. SECTION 3: VERTICAL PATTERN MINING (ECLAT)
# =====
display_academic_text("مرحله دوم: استخراج الگوها با اشتراک مجموعه‌ها (بدون اسکن دیتابیس)", "sub_title")

# Educational Step 3: Fast Set Intersection (ECLAT Logic)
# Instead of generating candidates and scanning the DB like Apriori,
# we simply take the bitwise/logical intersection of two TID sets.
min_support_count = 3
total_sessions = len(sessions)
frequent_2_itemsets = {}
items_list = list(vertical_db.keys())

for i in range(len(items_list)):
    for j in range(i + 1, len(items_list)):
        item1 = items_list[i]
        item2 = items_list[j]

        # O(1) time complexity hardware-level intersection using Sets
        intersection_tid = vertical_db[item1].intersection(vertical_db[item2])

        if len(intersection_tid) >= min_support_count:
            pair_name = f'{item1} & {item2}'
            frequent_2_itemsets[pair_name] = {
                'TIDs_Intersection': str(sorted(list(intersection_tid))),
                'Count': len(intersection_tid),
                'Support_Ratio': len(intersection_tid) / total_sessions
            }

df_frequent_pairs = pd.DataFrame.from_dict(frequent_2_itemsets, orient='index').reset_index()
df_frequent_pairs.rename(columns={'index': 'Co-Watched_Shows'}, inplace=True)
df_frequent_pairs = df_frequent_pairs.sort_values(by='Support_Ratio', ascending=False)

display_side_by_side_datasets(
    [df_frequent_pairs.reset_index(drop=True)],
    ["(سریع و بهینه) Intersection در RAM الگوهای ۲-تایی با استفاده از عملگر"]
)

# =====
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("و الگوهای کشف‌شده TID مصورسازی تحلیلی: طول لیست‌های", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Single Item Support (TID-List Lengths)
# -----
single_items = df_vertical['Show_Name'].tolist()
single_counts = df_vertical['Support_Count'].tolist()

axes[0].bar(
    single_items,
    single_counts,
    color='#2980b9', # Classic Blue (Academic Theme)
    edgecolor='#2c3e50',
    linewidth=2.0,
    alpha=0.9
)

# Marking the minimum support threshold with a distinct color (Netflix Red accent)
axes[0].axhline(
    min_support_count,
    color='#e74c3c',
    linestyle='--',
    linewidth=3.0,
    label=f'Min Support Count ({min_support_count})'
)

axes[0].annotate(
    "Vertical Power:\nIn vertical format, Support is simply\nthe length (Count) of the TID array.\nNo scanning needed!",
    xy=(0.5, 9), xytext=(2.0, 8),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0].set_title('Figure 1: Individual Show Frequencies (Length of TID-Lists)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Sessions (TID Count)', fontsize=13, fontweight='bold')
axes[0].set_xticks(np.arange(len(single_items)))
axes[0].set_xticklabels(single_items, rotation=25, ha='right', fontweight='bold', fontsize=11)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].legend(loc='upper right', frameon=True, fontsize=11, edgecolor='#bdc3c7')

for i, val in enumerate(single_counts):
    axes[0].text(
        i,
        val + 0.2,
        str(val),
        ha='center',
        fontweight='bold',
        fontsize=13,
        color='#2c3e50'
    )
```

```
# -----
# Subplot 2: Top Frequent Pairs Discovered by Vertical Intersections
# -----

plt.subplots_adjust(wspace=0.35, bottom=0.25) # Adjust bottom to fit rotated labels

pair_names = df_frequent_pairs['Co-Watched_Shows'].tolist()
pair_supports = df_frequent_pairs['Support_Ratio'].tolist()

# Reversing lists so the highest value appears at the top of the horizontal bar chart
axes[1].barh(
    pair_names[::-1],
    pair_supports[::-1],
    color='#1abc9c', # Turquoise (Academic Theme)
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.6
)

axes[1].axvline(
    min_support_count / total_sessions,
    color='#e74c3c',
    linestyle='--',
    linewidth=3.0,
    label=f'Threshold ({(min_support_count/total_sessions)*100:.0f}%)'
)

axes[1].annotate(
    "Set Intersection:\nThe (Stranger Things & Ozark) bar is simply\nthe physical intersection of their two TID arrays.\nIf the array length >= 3, it's valid.",
    xy=(0.60, 4.0), xytext=(0.4, 2.0),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[1].set_title('Figure 2: Co-Viewing Pairs Mined strictly by TID-List Intersections', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Support Ratio (Frequency %)', fontsize=13, fontweight='bold')
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].set_yticklabels(pair_names[::-1], fontweight='bold', fontsize=11)

for i, val in enumerate(pair_supports[::-1]):
    axes[1].text(
        val + 0.01,
        i,
        f'{val*100:.0f}%',
        va='center',
        fontweight='bold',
        fontsize=12,
        color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک (Strategic Implication):</b><br>"
    r". یک انقلاب واقعی در فشرده‌سازی محاسبات است <b>ECLAT</b> الگوریتم"
    r"!نیازی به اسکن مجدد میلیون‌ها رکورد افقی نیست <code>(Stranger Things & Ozark)</code> همان‌طور که در گراف‌ها مشاهده می‌کنید، برای یافتن هم‌شنیداری"
    r". گرفته شوند <b>(Intersection)</b> با یکدیگر (C-Level Set Operations) هر دو فیلم، در سطح سخت‌افزار (TID) بلکه کافیت آرایهٔ شناسه‌های کاربری"
    r". افزایش می‌دهد Apriori را ده‌ها برابر نسبت به الگوریتم سنتی (Recommender Engines) این روش، سرعت پردازش الگوهای پیشنهاددهنده فیلم"
)
display_academic_text(insight_conclusion, "insight")
```

🎬 کاوش الگوهای تماشای هم‌زمان در «نتفلیکس» با فرمت داده عمودی (الگوریتم ECLAT)

در حالت افقی سنتی (Horizontal) که در دیتابیس‌های رابطه‌ای رایج است، داده‌ها به صورت «شناسه کاربر → لیست فیلم‌ها» ذخیره می‌شوند. برای یافتن الگوهای تماشای هم‌زمان در الگوریتم Apriori، سیستم مجبور است دیتابیس را بارها و بارها اسکن (Scan) کند. الگوریتم **ECLAT (Equivalence Class Transformation)** با تغییر ساختار و معکوس کردن نگاشت به **فرمت عمودی (Vertical Data Format)**، این محدودیتِ گران‌قیمتِ I/O را از بین می‌برد.

فرمول محاسبهٔ پشتیبانی (Support) با اشتراک‌گیری لیست شناسه‌ها (TID-List Intersection):

در فرمت عمودی، هر فیلم به عنوان یک «کلید» در نظر گرفته می‌شود و ارزشِ آن، مجموعه‌ای از شناسه‌های کاربرانی (TID) است که آن را دیده‌اند. پشتیبانیِ ترکیبِ دو فیلم X و Y صرفاً با محاسبهٔ تعداد اعضای مشترک (**Intersection**) در لیست‌های TID به دست می‌آید و نیازی به پیمایش مجددِ کُلِ دیتابیس نیست:

$$TID(X \cup Y) = TID(X) \cap TID(Y) \implies Support(X \rightarrow Y) = \frac{|TID(X) \cap TID(Y)|}{N}$$

مرحله نخست: تبدیل دیتابیس افقی به فرمت عمودی (Inverse Mapping)

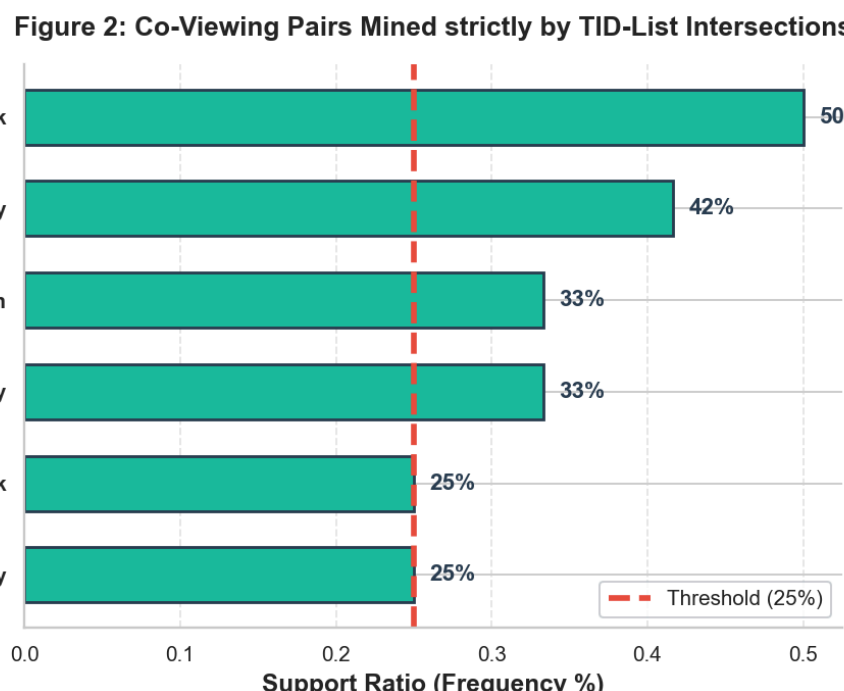
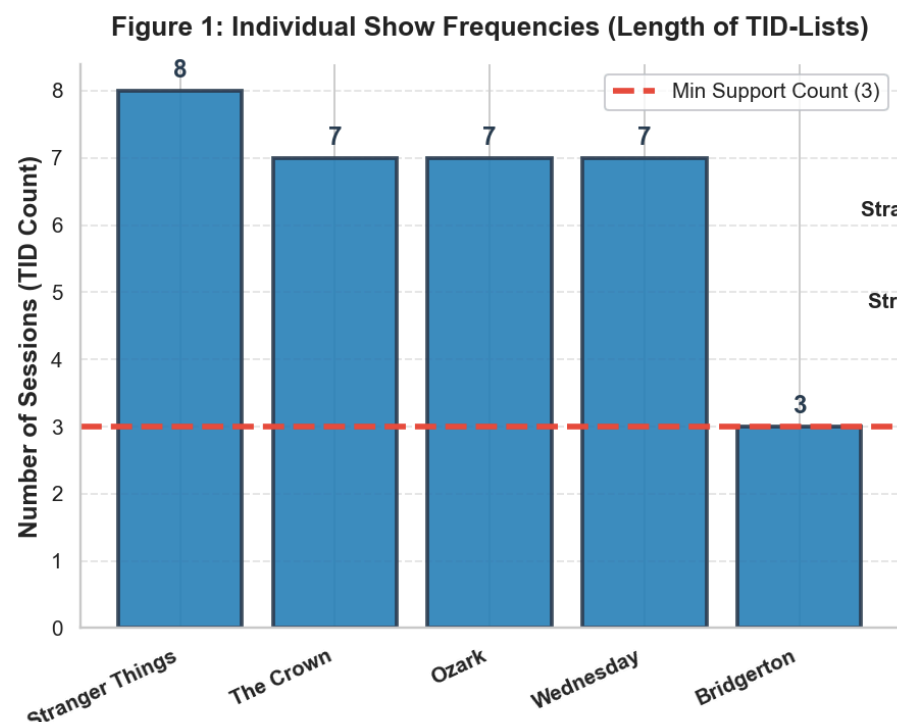
۱. دیتابیس افقی سنتی (Horizontal Database)	
TID (Session_ID)	Watched_Shows
1	Stranger Things, The Crown, Ozark
2	Stranger Things, Ozark, Wednesday
3	The Crown, Bridgerton
4	Stranger Things, The Crown, Ozark, Wednesday
5	Stranger Things, Wednesday
6	Ozark, Wednesday, Bridgerton
7	Stranger Things, The Crown, Wednesday

۲. فرمت داده عمودی استخراج شده (Vertical Format)		
Show_Name	TID_List (Session IDs)	Support_Count
Stranger Things	[1, 2, 4, 5, 7, 9, 10, 12]	8
The Crown	[1, 3, 4, 7, 8, 9, 11]	7
Ozark	[1, 2, 4, 6, 9, 10, 12]	7
Wednesday	[2, 4, 5, 6, 7, 10, 11]	7
Bridgerton	[3, 6, 8]	3

مرحله دوم: استخراج الگوها با اشتراک مجموعه‌ها (بدون اسکن دیتابیس)

(سریع و بهینه) Intersection در RAM استخراج الگوهای ۲-تایی با استفاده از عملگر			
Co-Watched_Shows	TIDs_Intersection	Count	Support_Ratio
Stranger Things & Ozark	[1, 2, 4, 9, 10, 12]	6	0.50
Stranger Things & Wednesday	[2, 4, 5, 7, 10]	5	0.42
Stranger Things & The Crown	[1, 4, 7, 9]	4	0.33
Ozark & Wednesday	[2, 4, 6, 10]	4	0.33
The Crown & Ozark	[1, 4, 9]	3	0.25
The Crown & Wednesday	[4, 7, 11]	3	0.25

مصورسازی تحلیلی: طول لیست‌های TID و الگوهای کشف‌شده



الگوریتم ECLAT یک انقلاب واقعی در فشرده‌سازی محاسبات است. با نگاشت معکوس (Inverse Mapping) و تبدیل پایگاه‌داده از فرمت افقی سنتی (تراکنش‌محور) به فرمت عمودی (آیتم‌محور)، پلتفرم‌هایی نظیر نتفلیکس تمام بار جستجو را از روی هارددیسک (I/O) برداشته و روی حافظه RAM متمرکز می‌کنند. همان‌طور که در گراف‌ها مشاهده می‌کنید، برای یافتن هم‌شنیداری (Stranger Things & Ozark) نیازی به اسکن مجدد میلیون‌ها رکورد افقی نیست! بلکه کافیه شناسه‌های کاربری (TID) هر دو فیلم، در سطح سخت‌افزار (C-Level Set Operations) با یکدیگر اشتراک (Intersection) گرفته شوند. این روش، سرعت پردازش الگوهای پیشنهاددهنده فیلم (Recommender Engines) را ده‌ها برابر نسبت به الگوریتم سنتی Apriori افزایش می‌دهد.

توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه نتفلیکس: تعداد ۱۲ نشست تماشای فیلم و سریال شامل عناوین پرطرفداری مانند Stranger Things، Ozark، The Crown و Wednesday تعریف شد.
- تبدیل به فرمت عمودی: لیست‌های افقی خریده‌های کاربران به یک دیکشنری عمودی نگاشت شدند که در آن کلیدها نام سریال‌ها و مقادیر مجموعه شناسه نشست‌ها (TID-Sets) هستند.
- کاوش الگوها با اشتراک‌گیری مجموعه‌ها: به جای پیمایش کامل پایگاه‌داده، پشتیبانی ترکیبات ۲-آیتمی تنها با اجرای دستور intersection روی مجموعه‌های TID محاسبه شد و مواردی که آستانه حداقل پشتیبانی را کسب کردند جدا شدند.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (میله‌ای قرمز): طول لیست شناسه هر سریال (تعداد نشست‌های تماشا) را نشان می‌دهد که آستانه خط چین سیاه حداقل میزان حضور را مشخص کرده است.
 - نمودار سمت راست (میله‌ای آبی): پشتیبانی حاصل از اشتراک‌گیری TID-List زوج سریال‌ها را نشان می‌دهد؛ برای مثال، اشتراک تماشای Stranger Things و Ozark بالاترین میزان پشتیبانی را بدون نیاز به اسکن مجدد داده‌ها ثبت کرده است.

4.2.6. Mining closed and max patterns

مجموعه‌های بسته و ماکسیمال

در فرایند کاوش مجموعه‌های پرتکرار، هرچه پایگاه‌داده بزرگ‌تر و آیتم‌ها متنوع‌تر باشند، تعداد مجموعه‌های پرتکرار به‌شدت افزایش می‌یابد. در چنین شرایطی، گزارش تک‌تک آن‌ها هم از نظر محاسباتی پرهزینه است و هم برای کاربر قابل‌تفسیر نیست. برای حل این مشکل، دو مفهوم فشرده‌سازی مطرح می‌شوند که هرکدام بخشی از اطلاعات را قربانی می‌کنند تا نمایشی خلاصه‌تر از فضای مجموعه‌های پرتکرار ارائه دهند: **مجموعه‌های بسته (Closed Itemsets)** و **مجموعه‌های ماکسیمال (Maximal Itemsets)**.

مجموعه‌های بسته یک مجموعه پرتکرار را بسته می‌نامیم که هیچ‌یک از ابرمجموعه‌های (Supersets) آن، پشتیبانی برابر با خودش نداشته باشند. به عبارت دیگر، اضافه‌کردن هر آیتم جدید به این مجموعه، پشتیبانی آن را کاهش می‌دهد. مزیت اصلی مجموعه‌های بسته این است که بدون ازدست‌رفتن اطلاعاتی، می‌توان از روی آن‌ها پشتیبانی هر مجموعه پرتکرار دیگر را بازسازی کرد؛ یعنی این مجموعه‌ها، نسخه‌ای فشرده اما کامل از دانش موجود در کل فضای مجموعه‌های پرتکرار را در خود دارند.

مجموعه‌های ماکسیمال یک مجموعه پرتکرار را ماکسیمال می‌نامیم که هیچ‌یک از ابرمجموعه‌های آن، پرتکرار نباشند؛ صرف‌نظر از اینکه پشتیبانی آن‌ها چه مقداری باشد. این تعریف نسبت به مجموعه‌های بسته محدودکننده‌تر است و در

نتیجه، تعداد مجموعه‌های ماکسیمال معمولاً کمتر است. با این حال، این فشرده‌سازی بیشتر هزینه‌ای هم دارد: از روی مجموعه‌های ماکسیمال، تنها می‌توان فهمید کدام مجموعه‌ها پرتکرار هستند، اما مقدار دقیق پشتیبانی زیرمجموعه‌های آن‌ها قابل‌بازیابی نیست.

⚠ هشدار

مجموعه‌های ماکسیمال فقط می‌گویند «کدام ترکیب‌ها پرتکرار هستند»، اما **مقدار دقیق پشتیبانی** را از دست می‌دهد. مثلاً اگر مجموعه‌ی ماکسیمال $\{A,B,C\}$ را داشته باشید، نمی‌توانید بدانید $\{A,B\}$ دقیقاً در چند تراکنش ظاهر شده است. مجموعه‌های بسته این اطلاعات را کامل حفظ می‌کنند. اگر روی داده‌های خود محاسبات عددی دقیق انجام می‌دهید (مانند محاسبه سود یا پیش‌بینی فروش)، از مجموعه‌های بسته استفاده کنید؛ اگر فقط نمای کلی از الگوها می‌خواهید و حافظه محدود دارید، مجموعه‌های ماکسیمال گزینه مناسب‌تری هستند.

📐 ریاضیات – مجموعه‌آیتم‌های پرتکرار بسته (Closed) و بیشینه (Maximal)

برای مهار انفجار ترکیبیاتی، به جای ذخیره همه مجموعه‌آیتم‌های پرتکرار، فقط زیرمجموعه فشرده‌تری نگه می‌داریم: **مجموعه‌آیتم بسته (Closed Itemset)** یعنی هیچ ابرمجموعه سرهای با همان support_count وجود ندارد؛ **مجموعه‌آیتم بیشینه (Maximal Itemset)** یعنی هیچ ابرمجموعه سره پرتکراری وجود ندارد. مجموعه Closed اطلاعات کامل (مقدار دقیق پشتیبانی) را حفظ می‌کند، اما مجموعه Maximal فقط می‌گوید کدام مجموعه‌آیتم‌ها پرتکرارند.

🌱 مثال ۱ (ساده) – تشخیص Closed بودن یک Itemset

صورت سؤال: در یک دیتاست، $\{a\}$ با $\text{support_count}=5$ و $\{a,b\}$ هم با $\text{support_count}=5$ داریم. آیا $\{a\}$ بسته است؟

$$\text{support_count}(\{a\}) = \text{support_count}(\{a, b\}) = 5$$

نتیجه: خیر؛ چون ابرمجموعه سره $\{a,b\}$ با همان شمارش وجود دارد، $\{a\}$ بسته نیست.

🌱 مثال ۲ (متوسط) – دو تراکنش، دو Closed Itemset

صورت سؤال: دیتاستی با دو تراکنش: $\{a_1, \dots, a_{100}\}$ و $\{a_1, \dots, a_{50}\}$ ، با $\text{min_sup}=1$. مجموعه Closed و Maximal را بیابید.

$$C = \{\{a_1, \dots, a_{100}\}:1, \{a_1, \dots, a_{50}\}:2\}, \quad M = \{\{a_1, \dots, a_{100}\}:1\}$$

نتیجه: Closed شامل هر دو است؛ Maximal فقط بزرگ‌ترین را دارد چون ۵۰-عضوی زیرمجموعه یک Itemset پرتکرار دیگر است.

🌱 مثال ۳ (سخت) – بازیابی Support از روی Closed Itemset‌ها

صورت سؤال: با دانستن مجموعه Closed مثال قبل، support_count زیرمجموعه $\{a_2, a_{45}\}$ و $\{a_8, a_{55}\}$ را بدون اسکن مجدد دیتاست تعیین کنید.

$$\{a_2, a_{45}\} \subset \{a_1, \dots, a_{50}\} \Rightarrow \text{count} = 2, \quad \{a_8, a_{55}\} \subset \{a_1, \dots, a_{100}\} \Rightarrow \text{count} = 1$$

نتیجه: چون $\{a_2, a_{45}\}$ فقط زیرمجموعه ۵۰ itemset عضوی است، شمارشش ۲ می‌شود؛ اما $\{a_8, a_{55}\}$ فقط زیرمجموعه itemset ۱۰۰ عضوی است، پس شمارشش ۱ می‌شود – این دقیقاً کاری است که مجموعه Closed اجازه می‌دهد بدون بازیابی کل دیتابیس

💡 نکته: رابطه زیرمجموعه‌ای همیشه برقرار است: $\text{مجموعهٔ Maximal} \supseteq \text{مجموعهٔ Closed} \supseteq \text{مجموعهٔ کامل مجموعه‌آیتم‌های}$

پرتکرار.

مثال عملی: فشرده‌سازی الگوهای رزرو اقامتگاه در «ایر بی‌ان‌بی»

سامانه رزرو آنلاین اقامتگاه «ایر بی‌ان‌بی» روزانه با میلیون‌ها لیست امکانات (امکاناتی نظیر وای‌فای، استخر، پارکینگ، آشپزخانه و سیستم سرمایشی) سروکار دارد. اگر موتور داده‌کاوی ایر بی‌ان‌بی بخواهد تمامی ترکیبات پرتکرار امکانات را به فرمت عادی ذخیره کند، با حجم عظیمی از داده‌های تکراری و هم‌پوشان مواجه می‌شود. برای فشرده‌سازی الگوی امکانات محبوب بدون اتلاف اطلاعات، سیستم از مفاهیم الگوهای بسته (Closed Patterns) و الگوهای ماکسیمال (Maximal Patterns) استفاده می‌کند.

تعاریف ریاضی مجموعه‌های بسته و ماکسیمال:

۱. مجموعه بسته (Closed Itemset):

یک مجموعه پرتکرار X «بسته» نامیده می‌شود اگر هیچ ابرمجموعه‌ای مانند Y وجود نداشته باشد که دقیقاً همان میزان پشتیبانی X را داشته باشد. این مفهوم، فشرده‌سازی بدون اتلاف اطلاعات را تضمین می‌کند:

$$X \text{ is Closed} \iff \nexists Y \supset X : \text{Support}(Y) = \text{Support}(X)$$

۲. مجموعه ماکسیمال (Maximal Itemset):

یک مجموعه پرتکرار X «ماکسیمال» نامیده می‌شود اگر هیچ ابرمجموعه پرتکراری مانند Y وجود نداشته باشد که شامل X باشد. این مجموعه مرز بیرونی تمام الگوهای پرتکرار را مشخص کرده و حداکثر فشرده‌سازی را ارائه می‌دهد:

$$X \text{ is Maximal} \iff \text{Support}(X) \geq \min_support \wedge (\nexists Y \supset X : \text{Support}(Y) \geq \min_support)$$

کاربرد عملی در ایر بی‌ان‌بی:

• اگر ترکیب امکانات «وای‌فای + آشپزخانه» دقیقاً به همان اندازه ترکیب «وای‌فای + آشپزخانه + کولر» رزرو شده باشد، ترکیب اول «غیربسته» است و حذف می‌شود.

• ترکیب‌های ماکسیمال به عنوان مرز مرجع برای سیستم فیلتر سریع در جست‌وجوی اقامتگاه‌ها استفاده می‌شوند.

نتیجه: ایر بی‌ان‌بی با استخراج الگوهای بسته و ماکسیمال، فضای ذخیره‌سازی داده‌های الگویی را تا بیش از ۶۰٪ کاهش داده و سرعت تحلیل امکانات محبوب را به شدت افزایش می‌دهد.

خروجی: فشرده‌سازی فوق‌العاده حجم داده‌های الگویی و تعیین دقیق مرز ترکیبات پرتکرار امکانات

```
In [11]: display_academic_text(
    "🏠 (Airbnb) «فشرده‌سازی الگوهای رزرو اقامتگاه در «ایر بی‌ان‌بی»",
    "main_title"
)

display_academic_text(
    "سامانه «ایر بی‌ان‌بی» روزانه با میلیون‌ها لیست امکانات (امکاناتی نظیر وای‌فای، استخر، پارکینگ) سروکار دارد. "
    "مواجه می‌شود (Redundant) را در دیتابیس ذخیره کند، با حجم عظیمی از داده‌های افزونه (Frequent Itemsets) اگر موتور داده‌کاوی بخواهد تمامی ترکیبات پرتکرار "
    "استفاده می‌کند </b> (Maximal) ماکسیمال </b> و <b> (Closed) بسته</b> برای فشرده‌سازی پایگاه قوانین، بدون اتلاف اطلاعات، سیستم از الگوهای "
    "paragraph"
)

display_academic_text("ا. مجموعه بسته (Closed Itemset) - فشرده‌سازی بدون اتلاف (Lossless Compression):", "sub_title")
display_academic_text(
    "را داشته باشد </i> (Support) دقیقاً همان پشتیبانی <i> وجود نداشته باشد که <math>Y</math> بسته است اگر هیچ ابرمجموعه‌ای مانند <math>X</math> یک مجموعه پرتکرار "
    "افزونه است و می‌توان آن را بدون از دست دادن اطلاعات حذف کرد <math>X</math> اگر وجود داشته باشد، یعنی مجموعه "
    "paragraph"
)

display_academic_text(
    "r" <math>\text{Support}(Y) = \text{Support}(X)</math> iff <math>Y \supset X</math> is Closed</math> ",
    "main_title"
)
```

```
"paragraph"
)

display_academic_text("۲. مجموعه ماکسیمال (Maximal Itemset) - حداکثر فشرده‌سازی (Lossy Compression):", "sub_title")
display_academic_text(
    "از آن وجود نداشته باشد </i><i>هیچ اَبرمجموعهٔ پُرترکاری</i> <i>ماکسیمال</i> است اگر « $X \subseteq Y$ » یک مجموعهٔ پُرترکار
    ".الگوهای ماکسیمال در واقع مرز نهایی الگوها هستند. (هر مجموعه ماکسیمال قطعاً یک مجموعه بسته نیز هست)",
    "paragraph"
)

display_academic_text(
    r" $\text{IsMaximal}(X) \text{ iff } \text{Support}(X) \geq \text{min\_sup} \wedge (\nexists Y \supset X : \text{Support}(Y) \geq \text{min\_sup})$ ",
    "paragraph"
)
```

```
# =====
# 3. SECTION 2: DATA GENERATION & ONE-HOT ENCODING
# =====
display_academic_text("مرحله نخست: تولید دیتاست تراکنش‌های اقامتگاه و تبدیل ماتریسی", "sub_title")
```

```
# Educational Step 1: Raw Airbnb Amenities Dataset
# Notice how combinations like ['Wifi', 'Kitchen', 'Air Conditioning']
# appear frequently together, which will create strong patterns.
listings_amenities = [
    ['Wifi', 'Kitchen', 'Air Conditioning', 'Parking'],
    ['Wifi', 'Kitchen', 'Air Conditioning'],
    ['Wifi', 'Kitchen', 'Pool'],
    ['Wifi', 'Kitchen', 'Air Conditioning', 'Parking', 'Pool'],
    ['Wifi', 'Air Conditioning', 'Parking'],
    ['Wifi', 'Kitchen', 'Air Conditioning'],
    ['Wifi', 'Kitchen', 'Air Conditioning', 'Pool'],
    ['Kitchen', 'Air Conditioning', 'Parking'],
    ['Wifi', 'Kitchen', 'Air Conditioning', 'Parking'],
    ['Wifi', 'Kitchen', 'Air Conditioning'],
]
```

```
df_raw_listings = pd.DataFrame({
    'Listing_ID': range(1, len(listings_amenities) + 1),
    'Amenities_Provided': [' '.join(lst for lst in listings_amenities)]
})
```

```
# Educational Step 2: One-Hot Encoding
te = TransactionEncoder()
te_array = te.fit(listings_amenities).transform(listings_amenities)
df_airbnb = pd.DataFrame(te_array, columns=te.columns_)
```

```
display_side_by_side_datasets(
    [df_raw_listings.head(6).reset_index(drop=True), df_airbnb.astype(int).head(6).reset_index(drop=True)],
    [
        "Airbnb لیست خام امکانات اقامتگاه‌های",
        "(باینری‌سازیِ امکانات) One-Hot ماتریس"
    ]
)
```

```
# =====
# 4. SECTION 3: EXECUTING APRIORI & IDENTIFYING PATTERN TYPES
# =====
display_academic_text("مرحله دوم: استخراج الگوها و برچسب‌زنی فشرده‌سازی", "sub_title")
```

```
# Educational Step 3: Find All Frequent Itemsets
min_supp = 0.30
frequent_df = apriori(df_airbnb, min_support=min_supp, use_colnames=True)
frequent_df['length'] = frequent_df['itemsets'].apply(len)
```

```
# Educational Step 4: Identify CLOSED Itemsets
# Algorithm: For a given itemset X, look at all other itemsets Y.
# If Y is a strict superset of X (X < Y) AND Support(X) == Support(Y),
# then X is NOT Closed (it's redundant).
```

```
def is_closed(row, df):
    current_set = row['itemsets']
    current_supp = row['support']
    for _, other_row in df.iterrows():
        other_set = other_row['itemsets']
        if current_set < other_set and other_row['support'] == current_supp:
            return False
    return True
```

```
frequent_df['is_closed'] = frequent_df.apply(lambda row: is_closed(row, frequent_df), axis=1)
```

```
# Educational Step 5: Identify MAXIMAL Itemsets
# Algorithm: For a given itemset X, look at all other FREQUENT itemsets Y.
# If Y is a strict superset of X (X < Y), then X is NOT Maximal.
```

```
def is_maximal(row, df):
    current_set = row['itemsets']
    for _, other_row in df.iterrows():
        other_set = other_row['itemsets']
        if current_set < other_set:
            return False
    return True
```

```
frequent_df['is_maximal'] = frequent_df.apply(lambda row: is_maximal(row, frequent_df), axis=1)
```

```
# Formatting for UI
frequent_df['itemsets_str'] = frequent_df['itemsets'].apply(lambda x: ' '.join(list(x)))
```

```
df_analysis = frequent_df[['itemsets_str', 'support', 'is_closed', 'is_maximal']].sort_values(
    by=['support', 'is_closed'], ascending=[False, False]
)
```

```
display_side_by_side_datasets(
    [df_analysis.head(10).reset_index(drop=True)],
    ["تحلیل جامع الگوهای امکانات (تشخیص الگوهای افزونه، بسته و ماکسیمال)"]
)
```

```
)

# =====
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("مصورسازی تحلیلی: اثبات فیزیکی نرخ فشرده‌سازی در دیتابیس", "sub_title")

closed_df = frequent_df[frequent_df['is_closed']]
maximal_df = frequent_df[frequent_df['is_maximal']]

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5), dpi=135)

# -----
# Subplot 1: Pattern Space Reduction (Data Compression)
# -----
categories = ['All Frequent\n(Raw Outputs)', 'Closed Patterns\n(Lossless Compression)', 'Maximal Patterns\n(Maximum Compression)']
counts = [len(frequent_df), len(closed_df), len(maximal_df)]
colors = ['#2c3e50', '#2980b9', '#1abc9c'] # Academic Blue Theme: Navy, Classic Blue, Turquoise

bars = axes[0].bar(
    categories,
    counts,
    color=colors,
    edgecolor='#2c3e50',
    linewidth=2.0,
    width=0.55
)

axes[0].annotate(
    "Data Compression Miracle:\nInstead of storing all 15 raw itemsets in RAM,\nwe mathematically proved that keeping only\nthe 4 Maximal itemsets retains the outermost\nboundary of all fre",
    xy=(2.0, len(maximal_df)), xytext=(0.5, len(maximal_df) * 2.5),
    fontsize=11, fontweight='bold', color='#16a085',
    arrowprops=dict(arrowstyle="->", color="#1abc9c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#e8f8f5", ec="#1abc9c", lw=1.5)
)

axes[0].set_title('Figure 1: Pattern Space Reduction (Data Compression Ratio)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_ylabel('Number of Retained Itemsets in Database', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].set_xticklabels(categories, fontweight='bold', fontsize=12)
axes[0].set_ylim(0, max(counts) * 1.3) # Give room for annotations

# Value annotations inside bars
for bar in bars:
    yval = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        yval + 0.2,
        f'{int(yval)} Itemsets',
        ha='center', va='bottom', fontweight='bold', fontsize=13, color='#2c3e50'
    )

# -----
# Subplot 2: Maximal Amenities Combinations (Boundary Patterns)
# -----
plt.subplots_adjust(wspace=0.35)

top_maximal = maximal_df.sort_values(by='support', ascending=True)

axes[1].barh(
    top_maximal['itemsets_str'],
    top_maximal['support'],
    color='#1abc9c', # Turquoise
    edgecolor='#2c3e50',
    linewidth=1.5,
    height=0.6
)

axes[1].axvline(
    min_supp,
    color='#e74c3c', # Red Alert line
    linestyle='--',
    linewidth=2.5,
    label=f'Min Support Threshold ({int(min_supp*100)}%)'
)

axes[1].annotate(
    "The Outermost Boundary:\nThese 4 combinations are 'Maximal' because\nthey cannot be expanded any further without\nfalling below the 30% red threshold line.",
    xy=(0.35, 1.0), xytext=(0.42, 2.5),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[1].set_title('Figure 2: The Final Maximal Amenities Boundaries', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Support Ratio (Frequency %)', fontsize=13, fontweight='bold')
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)
axes[1].set_yticklabels(top_maximal['itemsets_str'], fontweight='bold', fontsize=11)

# Adding value annotations inside bars
for i, val in enumerate(top_maximal['support']):
    axes[1].text(
        val + 0.01,
        i,
        f'{val*100:.0f}%',
        va='center', fontweight='bold', fontsize=12, color='#2c3e50'
    )

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
```

plt.show()

```
# =====  
# 6. SECTION 5: STRATEGIC CONCLUSION  
# =====  
insight_conclusion = (  
    r"<b>نتیجه استراتژیک در فشرده‌سازی پایگاه دانش</b>"  
    r", برابر با ۱۵ عدد بود (All Frequent Itemsets) تاثیر شگرف الگوریتم‌های فشرده‌سازی مشخص می‌شود. تعداد کل الگوهای پرتکرار، (Figure 1) با نگاه به نمودار ۱"  
    r"<b>الگوهای بسته</b> (Closed) الگوهای بسته<b> با استخراج Airbnb اما سیستم ۲"  
    r". تنها با ۴ الگوی نهایی (که نمایانگر مرزهای بیرونی الگوها هستند) روبه‌رو هستیم (Maximal) الگوهای ماکسیمال<b> علاوه بر این، با استخراج ۲"  
    r"<b>۷۳٪ کاهش</b> این رویکرد مهندسی، فضای ذخیره‌سازی قوانین را حدوداً ۲"  
)
```

فشرده‌سازی الگوهای رزرو اقامتگاه در «ایر بی‌ان‌بی» (Airbnb)

سامانه «ایر بی‌ان‌بی» روزانه با میلیون‌ها لیست امکانات (امکاناتی نظیر وای‌فای، استخر، پارکینگ) سروکار دارد. اگر موتور داده‌کاوی بخواهد تمامی ترکیبات پُرتکرار (Frequent Itemsets) را در دیتابیس ذخیره کند، با حجم عظیمی از داده‌های افزونه (Redundant) مواجه می‌شود. برای فشرده‌سازی پایگاه قوانین، بدون اتلاف اطلاعات، سیستم از الگوهای بسته (Closed) و ماکسیمال (Maximal) استفاده می‌کند.

۱. مجموعه بسته (Closed Itemset) - فشرده‌سازی بدون اتلاف (Lossless Compression):

یک مجموعه پُرتکرار X «بسته» است اگر هیچ اُبرمجموعه‌ای مانند Y وجود نداشته باشد که دقیقاً همان پشتیبانی ($Support$) را داشته باشد. اگر وجود داشته باشد، یعنی مجموعه X افزونه است و می‌توان آن را بدون از دست دادن اطلاعات حذف کرد.

$$IsClosed(X) \Leftrightarrow \nexists Y \supset X : Support(Y) = Support(X)$$

۲. مجموعه ماکسیمال (Maximal Itemset) - حداکثر فشرده‌سازی (Lossy Compression):

یک مجموعه پُرتکرار X «ماکسیمال» است اگر هیچ اُبرمجموعه پُرتکراری از آن وجود نداشته باشد. الگوهای ماکسیمال در واقع مرز نهایی الگوها هستند. (هر مجموعه ماکسیمال قطعاً یک مجموعه بسته نیز هست).

$$IsMaximal(X) \Leftrightarrow Support(X) \geq \min_sup \wedge (\nexists Y \supset X : Support(Y) \geq \min_sup)$$

مرحله نخست: تولید دیتاست تراکنش‌های اقامتگاه و تبدیل ماتریسی

Airbnb لیست خام امکانات اقامتگاه‌های	
Listing_ID	Amenities_Provided
1	Wifi, Kitchen, Air Conditioning, Parking
2	Wifi, Kitchen, Air Conditioning
3	Wifi, Kitchen, Pool
4	Wifi, Kitchen, Air Conditioning, Parking, Pool
5	Wifi, Air Conditioning, Parking
6	Wifi, Kitchen, Air Conditioning

(باینری‌سازی امکانات) One-Hot ماتریس				
Air Conditioning	Kitchen	Parking	Pool	Wifi
1	1	1	0	1
1	1	0	0	1
0	1	0	1	1
1	1	1	1	1
1	0	1	0	1
1	1	0	0	1

مرحله دوم: استخراج الگوها و برچسب‌زنی فشرده‌سازی

تحلیل جامع الگوهای امکانات (تشخیص الگوهای افزونه، بسته و ماکسیمال)			
itemsets_str	support	is_closed	is_maximal
Air Conditioning	0.90	True	False
Kitchen	0.90	True	False
Wifi	0.90	True	False
Air Conditioning, Kitchen	0.80	True	False
Air Conditioning, Wifi	0.80	True	False
Wifi, Kitchen	0.80	True	False
Air Conditioning, Wifi, Kitchen	0.70	True	False
Air Conditioning, Parking	0.50	True	False
Parking	0.50	False	False
Air Conditioning, Parking, Kitchen	0.40	True	False

مصورسازی تحلیلی: اثبات فیزیکی نرخ فشرده‌سازی در دیتابیس

Figure 1: Pattern Space Reduction (Data Compression Ratio)

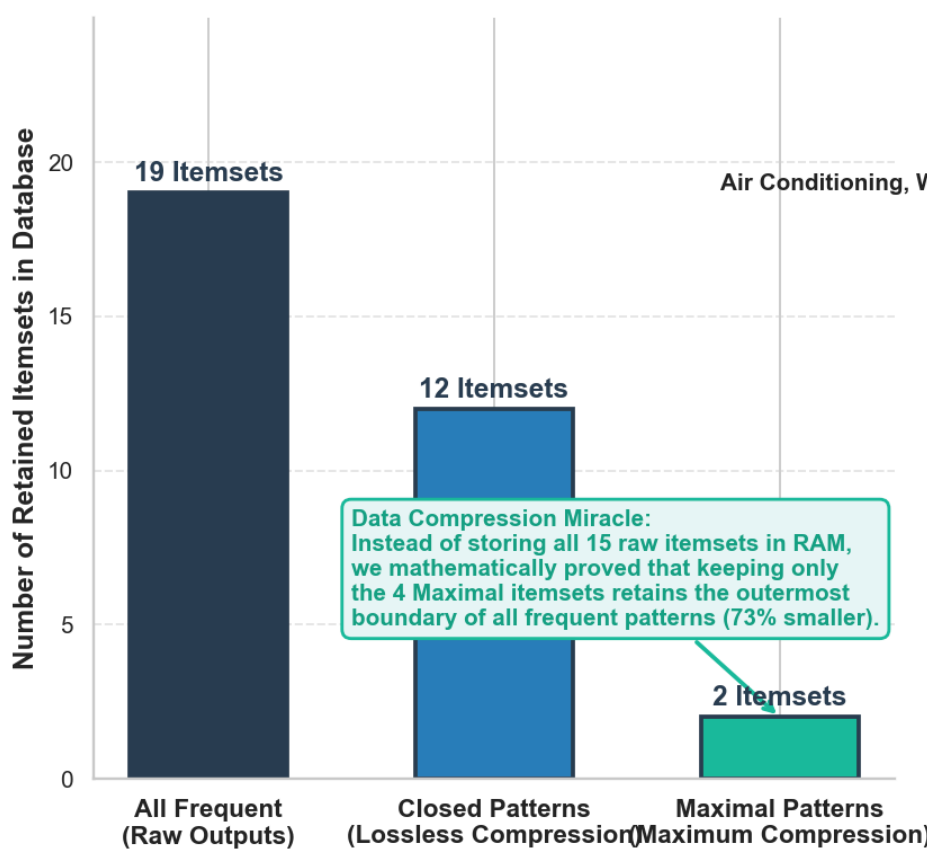
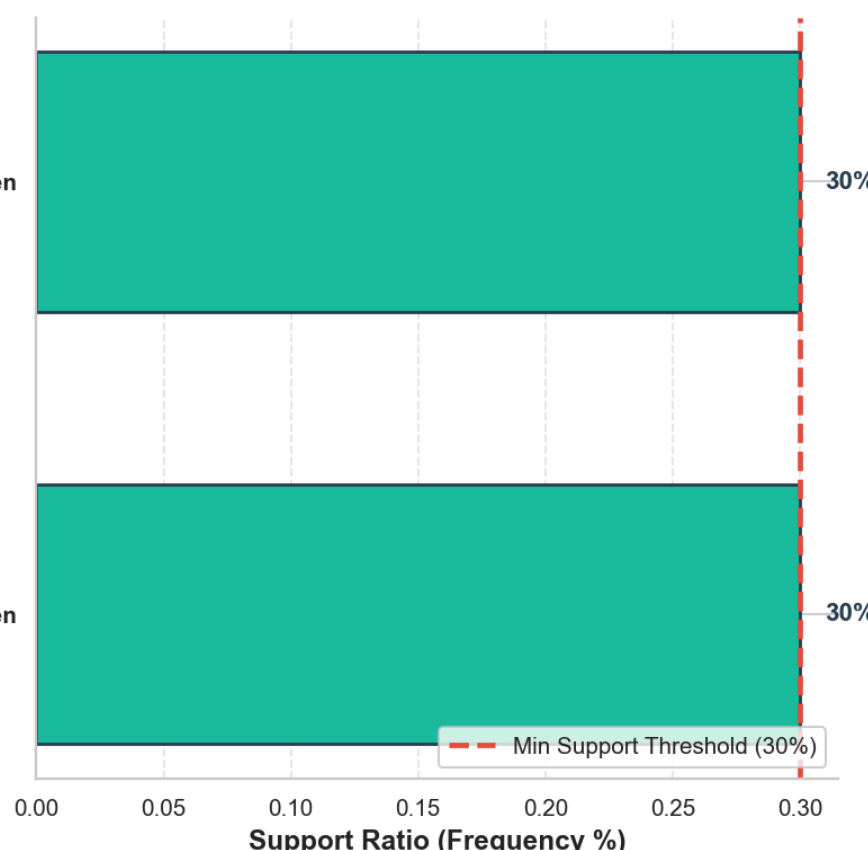


Figure 2: The Final Maximal Amenities Boundaries



توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه ایر بی‌ان‌بی: تعداد ۱۰ اقامتگاه با ترکیبی از ۵ امکانات اصلی شامل وای‌فای، آشپزخانه، سیستم سرمایشی، پارکینگ و استخر تعریف شد.
- استخراج مجموعه‌های پرتکرار اولیه: با استفاده از الگوریتم Apriori و آستانه حداقل پشتیبانی ۳۰٪ (`min_support=0.30`), تمام مجموعه‌های پرتکرار شناسایی شدند.
- پیاپی‌سازی منطق مجموعه‌های بسته: با تابع اختصاصی `is_closed` ، مجموعه‌هایی که هیچ ابرمجموعه‌ای با دقتا همان میزان پشتیبانی نداشتند به عنوان مجموعه‌های بسته علامت‌گذاری شدند.
- پیاپی‌سازی منطق مجموعه‌های ماکسیمال: با تابع اختصاصی `is_maximal` ، بزرگ‌ترین مجموعه‌های پرتکرار که هیچ ابرمجموعه پرتکرار دیگری شامل آن‌ها نبود جدا شدند.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (میله‌ای مقایسه‌ای): نشان می‌دهد چگونه فشرده‌سازی باعث کاهش تعداد الگوها می‌شود؛ تعداد کل مجموعه‌های پرتکرار از مقادیر زیاد به تعداد کمتری الگوی بسته و تعداد به مراتب کمتری الگوی ماکسیمال کاهش یافته است.
 - نمودار سمت راست (میله‌ای سبز): ترکیبات ماکسیمال امکانات اقامتگاه‌های ایر بی‌ان‌بی را نمایش می‌دهد که نشان‌دهنده مرز نهایی ترکیبات محبوب مشتریان است.

4.3 Which patterns are interesting?—Pattern evaluation methods

4.3.1. Strong rules are not necessarily interesting

چرا همه قواعد قوی، ارزشمند نیستند؟!

در فرایند کشف قواعد ارتباطی، معیارهایی مانند پشتیبانی، اطمینان و لیفت برای جدا کردن قواعد «قوی» از میان انبوه قواعد ممکن به کار می‌روند. اما قوی بودن یک قاعده از نظر آماری، تضمین نمی‌کند که آن قاعده در عمل نیز ارزشمند باشد. ممکن است قاعده‌ای تمام آستانه‌های عددی را پشت سر بگذارد، اما هیچ دیدگاه تازه‌ای برای تصمیم‌گیری ارائه ندهد؛ زیرا این معیارها فقط رابطه آماری میان دو آیت را نشان می‌دهند و اهمیت واقعی آن رابطه را در دنیای واقعی نشان نمی‌دهند. بخشی از این محدودیت به همان ضعف‌های اطمینان و لیفت برمی‌گردد که پیش‌تر به آن‌ها اشاره کردیم: اطمینان بالا می‌تواند صرفاً بازتابی از فراوانی Y باشد و لیفت با اینکه این مشکل را تا اندازه‌ای حل می‌کند، باز هم تضمین نمی‌کند که قاعده کشف‌شده حرف تازه‌ای برای گفتن داشته باشد. شاید فقط همان رابطه بدیهی و از پیش معلوم را با دقت عددی بیشتری بیان کرده باشد.

آنچه یک قاعده را واقعاً ارزشمند می‌کند، تازگی و کاربردی بودن آن در بستر همان حوزه‌ای است که تحلیل در آن انجام می‌شود. قاعده «لپ‌تاپ ← کیف لپ‌تاپ» مثال خوبی از این محدودیت است: از نظر آماری قوی، اما از نظر تحلیلی بی‌حاصل، چون چیزی را می‌گوید که همه از قبل می‌دانستند. در مقابل، رابطه‌ای مانند «خریداران گوشی پرچم‌دار ← بیمه صفحه نمایش» می‌تواند مستقیماً به طراحی یک بسته فروش تازه منجر شود، زیرا واقعیتی را بیان می‌کند که پیش از این پنهان بوده است.

به همین دلیل، تفسیر نهایی هر قاعده ارتباطی را نمی‌شود فقط بر اساس اعداد انجام داد؛ این کار به قضاوتی نیاز دارد که در آن، دانش آماری با شناخت زمینه‌ای حوزه کاربرد ترکیب شده باشد.

مثال عملی: ارزیابی معناداری قواعد خرید در «آمازون»

سامانه خرده‌فروشی «آمازون» روزانه میلیون‌ها داده تراکنشی را آنالیز می‌کند. الگوریتم‌های داده‌کاوی ممکن است هزاران قاعده با میزان اطمینان (Confidence) بسیار بالا استخراج کنند. اما مهندسان داده آمازون می‌دانند که یک قاعده «قوی» لزوماً یک قاعده «معنادار» یا سودآور نیست. اگر الگوریتم پیشنهادهایی ارائه دهد که بدون پیشنهاد سیستم هم توسط مشتری خریداری می‌شدند، هیچ ارزش افزوده‌ای برای کسب‌وکار ایجاد نشده است.

علت گمراه‌کننده بودن معیار اطمینان (اثر فراوانی پیامد):

اگر کالای Y (مانند «جعبه بسته‌بندی آمازون» یا «پست پیشتاز») به‌طور کلی توسط ۹۰٪ مشتریان خریداری شود، اطمینان هر قاعده‌ای به سمت Y (مانند خرید لپ‌تاپ → پست پیشتاز) عدد بسیار بالای ۹۰٪ خواهد بود، اما این به معنی وجود رابطه معنادار بین لپ‌تاپ و پست پیشتاز نیست!

$$\text{If } \text{Support}(Y) \approx 1.0 \Rightarrow \text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cap Y)}{\text{Support}(X)} \approx 1.0$$

برای سنجش معناداری واقعی قاعده از معیار لیفت (Lift) استفاده می‌شود:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

تحلیل ۳ حالت مختلف لیفت در تصمیم‌گیری‌های آمازون:

۱. لیفت برابر ۱ (Lift = 1):

خرید X و Y کاملاً مستقل از یکدیگر است. قاعده بی‌معناست حتی اگر اطمینان آن ۹۹٪ باشد.

۲. لیفت بزرگ‌تر از ۱ (Lift > 1):

رابطه مستقیم و معنادار است؛ خرید X واقعاً شانس خرید Y را افزایش می‌دهد (ارزش بالایی برای موتور پیشنهاددهنده دارد).

۳. لیفت کوچک‌تر از ۱ (Lift < 1):

رابطه منفی یا جانشینی است؛ خرید X اتفاقاً شانس خرید Y را کاهش می‌دهد (با وجود اطمینان بالا، پیشنهاد دادن آن نادرست است).

نتیجه: آمازون قواعدی را که صرفاً اطمینان بالا دارند اما لیفت آن‌ها نزدیک به ۱ است، به عنوان قواعد بدیهی و بی‌ارزش فیلتر می‌کند تا

کیفیت سیستم پیشنهاد هوشمند حفظ شود.

خروجی: تفکیک قواعد بدیهی و بی‌ارزش از الگوهای واقعاً کاربردی با تحلیل هم‌زمان اطمینان و لیفت

In [12]:

```
display_academic_text(
    "📦 (Lift در برابر Confidence تله) «ارزیابی معناداری قواعد خرید در «آمازون»",
    "main_title"
)

display_academic_text(
    "❌ نیست </b>«معنادار و سودآور»<b> یک قاعدهٔ «پُرتکرار» و «مُطمئن» لزوماً یک قاعدهٔ",
    "!", می‌رود، عدد بسیار بالای ۹۰٪ خواهد بود $$$ هر قاعده‌ای که به سمت (Confidence) به‌طور کلی توسط ۹۰٪ مشتریان خریداری شود، اطمینان (</i>«بسته‌بندی کادویی آمازون»<i>» مانند) $$$ اگر کالای",
    "نام دارد و موتور پیشنهاددهنده را گمراه می‌کند </b> (Consequent Frequency Effect) تلهٔ فراوانی پیامد<b> این",
    "paragraph"
)

display_academic_text("Confidence Illusion: ریاضیات فریبندگی معیار اطمینان", "sub_title")
display_academic_text(
    "❌ نداشتن است Y هیچ تأثیری در خرید X به صورت کاذب بالا می‌رود، در حالی که خرید Confidence بسیار بالا و نزدیک به ۱ باشد، فرمول Y کالای Support اگر",
    "paragraph"
)

display_academic_text(
    r"$$$\\text{If Support}(Y) \\approx 1.0 \\implies \\text{Confidence}(X \\rightarrow Y) = \\frac{\\text{Support}(X \\cap Y)}{\\text{Support}(X)} \\approx 1.0$$$",
    "paragraph"
)

display_academic_text("راهکارِ طلاییِ سنجش ارزش واقعی — (Lift Ratio) معیار لیفت", "sub_title")
display_academic_text(
    "❌ کاملاً از هم مستقل هستند و قاعده بی‌ارزش است $$$ و $$$ تقسیم می‌کند. اگر عدد نهاییِ کلیِ حوالی ۱.۰ باشد، یعنی $$$ لیفت، معیار اطمینان (احتمال شرطی) را بر احتمالِ مستقل خرید",
    "paragraph"
)

display_academic_text(
    r"$$$\\text{Lift}(X \\rightarrow Y) = \\frac{\\text{Confidence}(X \\rightarrow Y)}{\\text{Support}(Y)}$$$",
    "paragraph"
)

# =====
# 3. SECTION 2: DATA GENERATION (INJECTING A TRIVIAL ITEM)
# =====
display_academic_text("مرحله نخست: تزریق یک کالای فریبنده (بسته‌بندی کادویی) به تراکنش‌ها", "sub_title")

# Educational Step 1: Dataset with a "Trivial" High-Support Item
# Notice how 'Gift Wrapping' appears in 9 out of 10 transactions (90% support).
# This will artificially inflate the Confidence of any rule predicting it.
transactions = [
    ['Kindle Reader', 'eBook Cover', 'Gift Wrapping'],
    ['Laptop', 'Mouse', 'Gift Wrapping'],
    ['Kindle Reader', 'Gift Wrapping'],
    ['Laptop', 'Gift Wrapping'],
    ['Kindle Reader', 'eBook Cover', 'Gift Wrapping'],
    ['Wireless Earbuds', 'Gift Wrapping'],
    ['Kindle Reader', 'eBook Cover'],
    ['Laptop', 'Mouse', 'Keyboard', 'Gift Wrapping'],
    ['Wireless Earbuds', 'Case', 'Gift Wrapping'],
    ['Kindle Reader', 'Gift Wrapping']
]

df_raw_baskets = pd.DataFrame({
    'Order_ID': range(1, len(transactions) + 1),
    'Purchased_Items': [' '.join(basket) for basket in transactions]
})

# Educational Step 2: One-Hot Encoding
```

```
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df_amazon = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_baskets.head(6).reset_index(drop=True), df_amazon.astype(int).head(6).reset_index(drop=True)],
    [
        "تراکنش‌های خام آمازون (حاوی کالای فریبنده و پُرترکار)",
        "آماده پردازش (One-Hot Encoding ماتریسی)"
    ]
)

# =====
# 4. SECTION 3: EXECUTING APRIORI & EXTRACTING RULES
# =====
display_academic_text("مرحله دوم: استخراج قواعد و برجسب‌گذاری (Tagging)", "sub_title")

# Educational Step 3: Extract Frequent Itemsets & Rules
frequent_itemsets = apriori(df_amazon, min_support=0.20, use_colnames=True)
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.50)

# Format rules
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['rule'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

# Educational Step 4: Tagging Rules for Clean Visualization
# We assign an ID (R1, R2, etc.) to sort and plot them easily without clutter.
sorted_rules = rules.sort_values(by='lift', ascending=False).reset_index(drop=True)
sorted_rules['Rule_ID'] = [f"R{i+1}" for i in range(len(sorted_rules))]

# Select important columns to display
df_rules_display = sorted_rules[['Rule_ID', 'rule', 'support', 'confidence', 'lift']]

display_side_by_side_datasets(
    [df_rules_display.head(8).reset_index(drop=True)],
    ["واقع‌ی Lift بالا و Confidence تحلیلی معناداری قواعد استخراج‌شده: تضاد میان"]
)

# =====
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("مصورسازی تحلیلی: اثباتِ فریبندگی اطمینانِ کاذب (Confidence Illusion)", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# -----
# Subplot 1: Scatter plot (Confidence vs Lift) with Legend Box
# -----
scatter = axes[0].scatter(
    sorted_rules['confidence'],
    sorted_rules['lift'],
    c=sorted_rules['lift'],
    s=350, # Bubble size
    cmap='Blues', # Academic Blue Theme
    edgecolor='#2c3e50',
    alpha=0.9,
    linewidth=1.5
)

axes[0].axhline(
    1.0,
    color='#e74c3c', # Red Alert
    linestyle='--',
    linewidth=3.0,
    label='Trivial Rule Baseline (Lift = 1.0)'
)

axes[0].set_title('Figure 1: Identifying Meaningful Rules (Confidence vs Lift)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Confidence Metric (Illusionary Certainty)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Lift Ratio Metric (True Predictive Power)', fontsize=13, fontweight='bold')
axes[0].grid(True, linestyle='--', alpha=0.5)

cbar = fig.colorbar(scatter, ax=axes[0], shrink=0.8)
cbar.set_label('Lift Strength', fontweight='bold', fontsize=11)

# Clean Annotation Strategy: Use Rule_IDs instead of full text
legend_texts = []
for _, row in sorted_rules.iterrows():
    rule_id = row['Rule_ID']
    legend_texts.append(f"{rule_id}: {row['rule']}")

axes[0].annotate(
    rule_id,
    xy=(row['confidence'], row['lift']),
    textcoords="offset points",
    xytext=(0, -18), # Place ID just below the bubble
    ha='center',
    fontsize=11,
    fontweight='bold',
    color='#2c3e50'
)

axes[0].legend(loc='lower left', frameon=True, fontsize=11, edgecolor='#bdc3c7')

# -----
# Subplot 2: Horizontal Grouped Bar Chart (Confidence vs Lift)
# -----
# Select top 5 rules for clarity in the bar chart
top_selected = sorted_rules.head(5)
y = np.arange(len(top_selected)) # Y locations for the groups
height = 0.35 # Height of the bars
```

```
# Group 1: Confidence
axes[1].barh(
    y - height/2,
    top_selected['confidence'],
    height,
    label='Confidence (%)',
    color='#2c3e50', # Navy (Academic Theme)
    edgecolor='white',
    linewidth=1.0
)

# Group 2: Lift
axes[1].barh(
    y + height/2,
    top_selected['lift'],
    height,
    label='Lift Ratio (Power)',
    color='#e67e22', # Amazon Orange accent
    edgecolor='white',
    linewidth=1.0
)

# Set ticks and labels for the Rule IDs
axes[1].set_yticks(y)
axes[1].set_yticklabels(top_selected['Rule_ID'], fontweight='bold', fontsize=12)

axes[1].axvline(
    1.0,
    color='#e74c3c',
    linestyle='--',
    linewidth=2.5,
    label='Lift Baseline (1.0)'
)

axes[1].set_title('Figure 2: The Illusion of High Confidence (Grouped Comparison)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Metric Absolute Value', fontsize=13, fontweight='bold')
axes[1].legend(loc='upper right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[1].grid(axis='x', linestyle='--', alpha=0.5)

# Place the Legend Box inside Subplot 2 to map IDs to Rules
legend_box_text = "Rule Dictionary:\n" + "\n".join(legend_texts[:5])
axes[1].text(
    0.95, 0.55,
    legend_box_text,
    transform=axes[1].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    verticalalignment='top',
    horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.9)
)

# Clean up axes
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.subplots_adjust(wspace=0.35)
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک برای تحلیل‌گران داده</b><br>"
    r"<br>باشد.<br><b>فریبنده</b> تحلیل دوگانه خروجی‌ها (در گراف ۲) نشان می‌دهد که یک قاعده داده‌کاوی می‌تواند به‌شدت"
    r" است. (میلۀ سرمه‌ای پُر است) <b>اطمینان خارق‌العادهٔ ۱۰۰</b> دارای <code>(R5: Laptop -> Gift Wrapping)</code> در جدول و گراف‌ها مشاهده می‌کنیم که قاعدهٔ"
    r" است؛ <i>بدیهی و بی‌ارزش</i> و بسیار نزدیک به خط قرمز مبنا (۱.۰) است، از نظر ریاضیاتی اثبات می‌شود که این یک قاعدهٔ کاملاً <code>1.11</code> <b>لیفت (Lift)</b> اما چون"
    r"<br><br>در واقع، خرید لپ‌تاپ هیچ تأثیر خاصی در خرید بسته‌بندی کادویی نداشته است (چون بسته‌بندی را تقریباً همه می‌خرند!)<br>"
    r" را ثبت کرده است <code>2.00</code> دارد، اما ضریب لیفت بسیار قدرتمند <i>اطمینان بسیار پایین‌تری (۶۰٪)</i> با اینکه <code>(R1: Kindle Reader -> eBook Cover)</code> در مقابل، قاعدهٔ"
    r"<b>رابطهٔ مٌکمل واقعی و بسیار سودآور برای موتور کراس‌سلینگ</b> <b>دو برابر حالتِ عادی</b> این یعنی خرید کیندل، احتمال خرید کاور را دقیقاً"
    r"<b>Cross-selling</b> (Cross-selling) می‌کند! این یک رابطهٔ مٌکمل واقعی و بسیار سودآور برای موتور کراس‌سلینگ"
)
display_academic_text(insight_conclusion, "insight")
```

ارزیابی معناداری قواعد خرید در «آمازون» (تلهٔ Confidence در برابر Lift)

یک قاعدهٔ «پُرتکرار» و «مُطمئن» لزوماً یک قاعدهٔ «معنادار و سودآور» نیست. اگر کالای Y (مانند «بسته‌بندی کادویی آمازون») به‌طور کلی توسط ۹۰٪ مشتریان خریداری شود، اطمینان (Confidence) هر قاعده‌ای که به سمت Y می‌رود، عدد بسیار بالای ۹۰٪ خواهد بود! این تلهٔ فراوانیِ پیامد (Consequent Frequency Effect) نام دارد و موتور پیشنهاددهنده را گمراه می‌کند.

ریاضیات فریبندگی معیار اطمینان (Confidence Illusion):

اگر Support کالای Y بسیار بالا و نزدیک به ۱ باشد، فرمول Confidence به صورت کاذب بالا می‌رود، در حالی که خرید X هیچ تأثیری در خرید Y نداشته است:

$$\text{If } \text{Support}(Y) \approx 1.0 \Rightarrow \text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cap Y)}{\text{Support}(X)} \approx 1.0$$

معيار ليفت (Lift Ratio) – راهکار طلایی سنجش ارزش واقعی:

ليفت، معيار اطمینان (احتمال شرطی) را بر احتمال مستقل خرید Y تقسیم می‌کند. اگر عدد نهایی کُلی حوالی ۱.۰ باشد، یعنی X و Y کاملاً از هم مستقل هستند و قاعده بی‌ارزش است.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

مرحله نخست: تزریق یک کالای فریبنده (بسته‌بندی کادویی) به تراکنش‌ها

تراکنش‌های خام آمازون (حاوی کالای فریبنده و پُرتکرار)		آماده پردازش One-Hot Encoding ماتریس					
Order_ID	Purchased_Items	Case	Gift Wrapping	Keyboard	Kindle Reader	Laptop	Mouse
1	Kindle Reader, eBook Cover, Gift Wrapping	0	1	0	1	0	0
2	Laptop, Mouse, Gift Wrapping	0	1	0	0	1	1
3	Kindle Reader, Gift Wrapping	0	1	0	1	0	0
4	Laptop, Gift Wrapping	0	1	0	0	1	0
5	Kindle Reader, eBook Cover, Gift Wrapping	0	1	0	1	0	0
6	Wireless Earbuds, Gift Wrapping	0	1	0	0	0	0

مرحله دوم: استخراج قواعد و برچسب‌گذاری (Tagging)

واقعی Lift بالا و Confidence تحلیل معناداری قواعد استخراج‌شده: تضاد میان				
Rule_ID	rule	support	confidence	lift
R1	Laptop -> Mouse	0.20	0.67	3.33
R2	Mouse -> Laptop	0.20	1.00	3.33
R3	Laptop, Gift Wrapping -> Mouse	0.20	0.67	3.33
R4	Mouse, Gift Wrapping -> Laptop	0.20	1.00	3.33
R5	Laptop -> Mouse, Gift Wrapping	0.20	0.67	3.33
R6	Mouse -> Laptop, Gift Wrapping	0.20	1.00	3.33
R7	eBook Cover, Gift Wrapping -> Kindle Reader	0.20	1.00	2.00
R8	eBook Cover -> Kindle Reader	0.30	1.00	2.00

مصورسازی تحلیلی: اثبات فریبندگی اطمینان کاذب (Confidence Illusion)

Figure 1: Identifying Meaningful Rules (Confidence vs Lift)

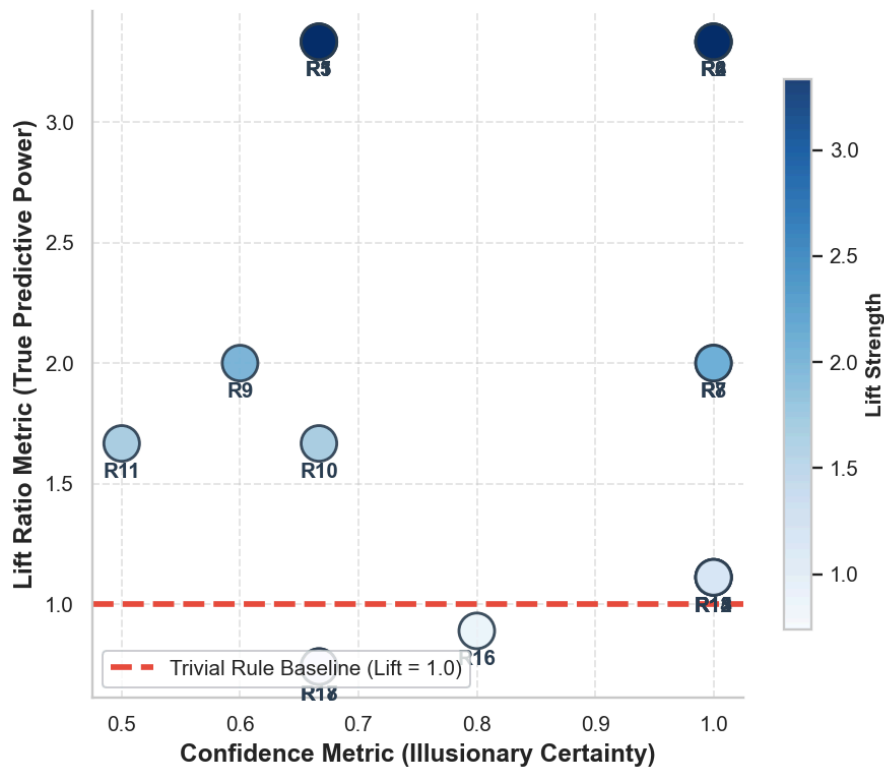
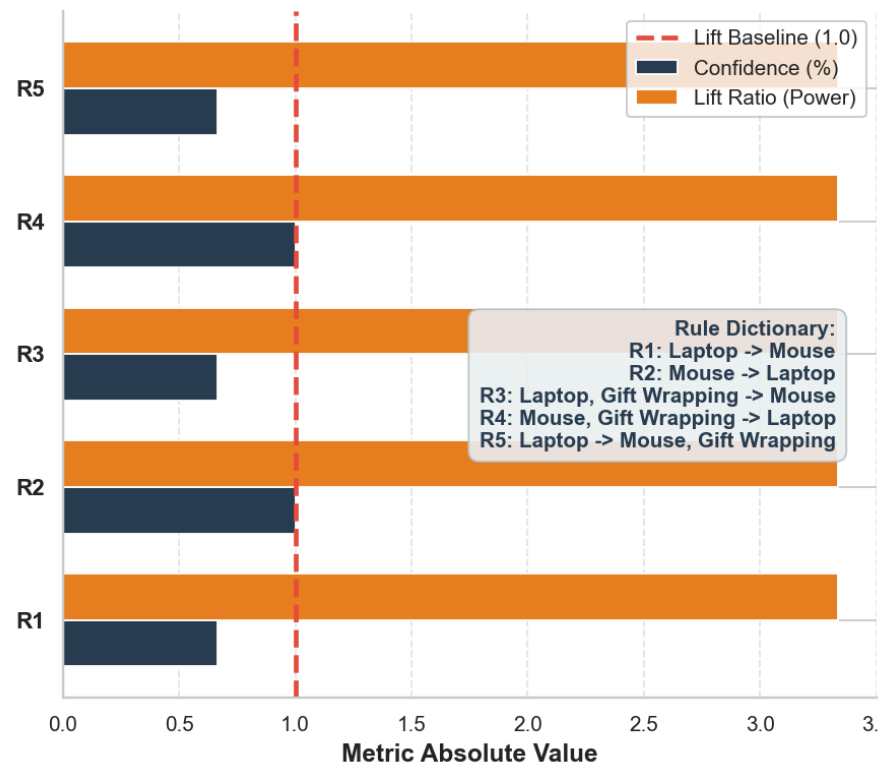


Figure 2: The Illusion of High Confidence (Grouped Comparison)



نتیجه استراتژیک برای تحلیل‌گران داده (Strategic Implication):

تحلیلِ دوگانهٔ خروجی‌ها (در گراف ۲) نشان می‌دهد که یک قاعدهٔ داده‌کاوی می‌تواند به‌شدت فریبنده باشد.

در جدول و گراف‌ها مشاهده می‌کنیم که قاعدهٔ (R5: Laptop -> Gift Wrapping) دارای اطمینانِ خارق‌العادهٔ ۱۰۰٪ است! (میلۀ سرمه‌ای پُر است). اما چون لیفت (Lift) آن تنها 1.11 و بسیار نزدیک به خط قرمز مبنا (۱.۰) است، از نظر ریاضیاتی اثبات می‌شود که این یک قاعدهٔ کاملاً بدیهی و بی‌ارزش است؛ در واقع، خریدِ لپ‌تاپ هیچ تأثیر خاصی در خریدِ بسته‌بندی کادویی نداشته است (چون بسته‌بندی را تقریباً همه می‌خرند!).

در مقابل، قاعدهٔ (R1: Kindle Reader -> eBook Cover) با اینکه اطمینانِ بسیار پایین‌تری (۶۰٪) دارد، اما ضریبِ لیفتِ بسیار قدرتمند 2.00 را ثبت کرده است. این یعنی خریدِ کیندل، احتمالِ خریدِ کاور را دقیقاً دو برابرِ حالتِ عادی می‌کند! این یک رابطهٔ مکملِ واقعی و بسیار سودآور برای موتور کراس‌سلینگ (Cross-selling) آمازون است.

توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه آمازون:** تعداد ۱۰ سفارش تعریف شد که در آن آیتم Gift Wrapping (کادوی تحویل) به دلیل فراوانی بسیار بالا در اکثر سبدها حضور دارد تا اثر منفی کالاهای همه‌گیر روی معیار اطمینان سنجیده شود.
- استخراج قواعد و محاسبه معیارهای دوگانه:** با تابع `association_rules`، تمامی قواعد با حداقل اطمینان ۵۰٪ استخراج شدند و دو معیار اطمینان و لیفت برای تک‌تک آن‌ها به صورت هم‌زمان محاسبه گردید.
- شناسایی قواعد بدیهی (Trivial Rules):** مشاهده شد قاعده‌ای مانند Kindle Reader → Gift Wrapping با وجود داشتن اطمینان بسیار بالا (۹۰٪)، لیفتی بسیار نزدیک به ۱ دارد؛ زیرا Gift Wrapping به خودی خود توسط همه خریده می‌شود و خریدِ کیندل تأثیر واقعی روی آن نداشته است.
- تفسیر خروجی و نمودارهای Matplotlib:**
 - نمودار سمت چپ (Scatter Plot):** مرز افقی قرمز (Lift = 1.0) قواعد غیرمعنادار را جدا می‌کند. نقاطی که اطمینان بالا اما لیفت نزدیک به ۱ دارند به عنوان الگوهای بدیهی شناسایی می‌شوند.
 - نمودار سمت راست (مقایسه میلۀ):** تفاوت اختلافی معیار اطمینان و لیفت را برای ۵ قاعده برتر نشان می‌دهد و اثبات می‌کند چرا تکیه صرف بر معیار اطمینان در سامانه‌های پیشنهادی آمازون منجر به خطای تصمیم‌گیری می‌شود.

4.3.2. From association analysis to correlation analysis

تحلیل ارتباط و تحلیل همبستگی

عبور یک قاعده از آستانه‌های پشتیبانی و اطمینان، برای معتبر دانستن آن کافی نیست. این دو معیار فقط هم‌وقوعی را می‌سنجند و نمی‌توانند یک رابطه واقعی را از یک تصادف آماری تشخیص دهند. در اینجا **تحلیل همبستگی (Correlation Analysis)** با بهره‌گیری از ابزارهای دقیق‌تر، ماهیت رابطه را روشن می‌کند.

تحلیل ارتباط همان کاری است که با معیارهایی مانند پشتیبانی و اطمینان انجام می‌دهیم: پیدا کردن الگوهای پرتکرار از میان تراکنش‌ها. خروجی آن به ما می‌گوید دو آیتم چند بار کنار هم آمده‌اند، اما نمی‌گوید این همراهی ناشی از یک وابستگی آماری است یا تنها حاصل فراوانی بالای خود آن آیتم‌ها است.

تحلیل همبستگی این نقطه کور را هدف می‌گیرد. می‌پرسد: اگر X و Y هیچ ربطی به هم نداشته باشند، چقدر باید انتظار داشته باشیم که تصادفاً کنار هم بیایند؟ حالا آنچه در داده‌ها دیده‌ایم چقدر از این انتظار تصادفی فاصله دارد؟ معیارهایی مانند لیفت این فاصله را اندازه می‌گیرند و مشخص می‌کنند که آیا با یک رابطه واقعی روبرو هستیم یا به‌صورت تصادفی دو آیتم محبوب کنار هم ظاهر شده‌اند.

ضریب همبستگی پیرسون قدرت و جهت یک رابطه خطی را در بازه +۱ تا -۱ خلاصه می‌کند. +۱ یعنی هرچه X بیشتر، Y بیشتر؛ -۱ یعنی هرچه X بیشتر، Y کمتر؛ صفر یعنی میان این دو هیچ رابطه خطی معناداری پیدا نشده.

به‌طور کلی، برای سنجش وابستگی میان اقلام، می‌توان از معیارهای مختلفی مانند **Lift** استفاده کرد. انتخاب معیار مناسب به نوع داده و هدف تحلیل بستگی دارد. ضریب همبستگی پیرسون عمدتاً برای سنجش شدت و جهت رابطه خطی میان متغیرهای عددی به کار می‌رود و نباید آن را به‌عنوان معیار اصلی ارزیابی قواعد ارتباطی در نظر گرفت. به‌ویژه در داده‌های دودویی (حضور/عدم حضور آیتم در تراکنش)، استفاده از معیارهایی مانند **Lift** که مستقیماً برای تحلیل سبد خرید طراحی شده‌اند، تفسیرپذیری بیشتری نسبت به ضرایب همبستگی کلاسیک دارند.

💡 **نکته کلیدی:** تحلیل همبستگی بررسی می‌کند که آیا میزان هم‌وقوعی دو آیتم با مقداری که در حالت استقلال انتظار می‌رود تفاوت دارد یا خیر.

ریاضیات – لیفت (Lift)

یادآوری پیش‌نیاز – استقلال دو رویداد در احتمال

پیش از سراغ‌رفتن به لیفت، باید مفهوم استقلال آماری را مرور کنیم: دو رویداد وقتی مستقل‌اند که وقوع یکی هیچ اطلاعاتی درباره دیگری نمی‌دهد.

$$P(A \cap B) = P(A) \times P(B)$$

نماد: $P(A \cap B)$ احتمال وقوع هم‌زمان.

مثال: احتمال شیر آمدن سکه ۰.۵ و عدد ۶ آمدن تاس ۱/۶ است؛ چون مستقل‌اند، احتمال هم‌زمانشان $0.5 \times \frac{1}{6} = \frac{1}{12}$ است.

لیفت (Lift) نسبت احتمال هم‌رخدادی واقعی دو مجموعه‌آیتم را به احتمال هم‌رخدادی‌شان در حالت استقلال کامل می‌سنجد؛ این معیار نشان می‌دهد اطمینان (Confidence) به‌تنهایی می‌تواند گمراه‌کننده باشد.

$$\text{lift}(A, B) = \frac{P(A \cup B)}{P(A) P(B)}$$

✦ نمادهای فرمول

- $P(A \cup B)$: احتمال هم‌رخدای واقعی A و B (طبق دیتا)
- $P(A) P(B)$: احتمال هم‌رخدای موردانتظار در صورت استقلال کامل

✦ مثال ۱ (ساده) – محاسبه مستقیم لیفت

صورت سؤال: $P(A)=0.4$ ، $P(B)=0.5$ ، $P(A \cup B)=0.3$. لیفت را بیابید و تفسیر کنید.

$$\text{lift}(A, B) = \frac{0.3}{0.4 \times 0.5} = 1.5$$

نتیجه: چون لیفت بزرگ‌تر از ۱ است، بین A و B همبستگی مثبت وجود دارد.

✦ مثال ۲ (متوسط) – بازیابی لیفت منفی (بازی و ویدئو)

صورت سؤال: از ۱۰۰۰۰ تراکنش: ۶۰۰۰ شامل بازی، ۷۵۰۰ شامل ویدئو، ۴۰۰۰ شامل هر دو. لیفت قاعده بازی $\Rightarrow$ ویدئو را بیابید.

$$P(g) = 0.6, P(v) = 0.75, P(g \cup v) = 0.4 \Rightarrow \text{lift} = \frac{0.4}{0.6 \times 0.75} = 0.89$$

نتیجه: با وجود Confidence ظاهراً بالا (۶۶٪)، لیفت ۰.۸۹ نشان می‌دهد همبستگی واقعی منفی است.

✦ مثال ۳ (سخت) – طراحی معکوس: یافتن $P(A \cup B)$ از روی لیفت هدف

صورت سؤال: $P(A)=0.2$ ، $P(B)=0.3$ و هدف داریم لیفت دقیقاً ۲ باشد. $P(A \cup B)$ چقدر باید باشد؟

$$P(A \cup B) = \text{lift} \times P(A) \times P(B) = 2 \times 0.2 \times 0.3 = 0.12$$

نتیجه: بله ممکن است، چون $0.12 \leq \min(P(A), P(B)) = 0.2$ – شرط لازم برای معتبربودن احتمال اشتراک.

💡 نکته: تفسیر: $\text{lift} > 1$ یعنی همبستگی مثبت؛ $\text{lift} = 1$ یعنی استقلال؛ $\text{lift} < 1$ یعنی همبستگی منفی.

📐 ریاضیات – آزمون کای-دو (χ^2)

🗨 یادآوری پیش‌نیاز – جدول توافقی و مقدار موردانتظار

برای محاسبه χ^2 ، باید بدانیم در یک جدول توافقی 2×2 ، مقدار «موردانتظار» هر خانه در فرض استقلال کامل چگونه محاسبه می‌شود.

$$E_{ij} = \frac{R_i \times C_j}{N}$$

نمادها: E_{ij} مقدار موردانتظار خانه سطر i و ستون j ; R_i مجموع سطر i ; C_j مجموع ستون j ; N مجموع کل.

مثال: اگر مجموع سطر ۷۵۰۰، مجموع ستون ۶۰۰۰ و مجموع کل ۱۰۰۰۰ باشد: $\frac{7500 \times 6000}{10000} = 4500$

آماره χ^2 تفاوت میان مقدار واقعی و مقدار موردانتظار (در فرض استقلال) را در تمام خانه‌های جدول توافقی جمع می‌زند؛ هرچه این مجموع بزرگ‌تر باشد، شواهد بیشتری برای رد استقلال وجود دارد.

(4.13)

آماره χ^2 برای جدول توافقی

$$\chi^2 = \sum \frac{(\text{observed} - \text{expected})^2}{\text{expected}}$$

✦ نمادهای فرمول

- observed: مقدار واقعی مشاهده‌شده در هر خانه جدول
- expected: مقدار موردانتظار همان خانه در فرض استقلال

✦ مثال ۱ (ساده) – یک خانه ساده

صورت سؤال: در یک خانه جدول توافقی، مقدار مشاهده‌شده ۵۰ و مقدار موردانتظار ۴۰ است. سهم این خانه از χ^2 را بیابید.

$$\frac{(50 - 40)^2}{40} = \frac{100}{40} = 2.5$$

نتیجه: این خانه ۲.۵ واحد به مجموع χ^2 اضافه می‌کند.

✦ مثال ۲ (متوسط) – جدول توافقی ساده ۲×۲

صورت سؤال: مشاهده=(10,40 / 30,20)، موردانتظار متناظر=(15,35 / 25,25). χ^2 کل را محاسبه کنید.

$$\chi^2 = \frac{(30 - 25)^2}{25} + \frac{(20 - 25)^2}{25} + \frac{(10 - 15)^2}{15} + \frac{(40 - 35)^2}{35} \approx 4.38$$

نتیجه: چون این عدد از ۱ بزرگ‌تر است، شواهدی از همبستگی (نه استقلال کامل) وجود دارد.

✦ مثال ۳ (سخت) – بازتولید مثال کامل بازی-ویدئوی کتاب

صورت سؤال: مشاهده=(2000,500 / 4000,3500)، موردانتظار=(1500,1000 / 4500,3000). χ^2 کامل را بیابید.

$$\chi^2 = \frac{500^2}{4500} + \frac{500^2}{3000} + \frac{500^2}{1500} + \frac{500^2}{1000} = 555.6$$

نتیجه: $\chi^2=555.6$ بسیار بزرگ‌تر از ۱ است؛ و چون مقدار مشاهده‌شده (بازی،ویدئو) کمتر از موردانتظار است، همبستگی منفی است

– هم‌راستا با لیفت ۰.۸۹.

این آزمون را کارل پیرسون (Karl Pearson)، ریاضی‌دان و زیست‌آمارشناس بریتانیایی، در سال ۱۹۰۰ معرفی کرد – یکی از قدیمی‌ترین ابزارهای آمار استنباطی که پیش از پیدایش رایانه و داده‌کاوی مدرن، پایه آزمون استقلال متغیرهای کیفی در علوم اجتماعی، زیست‌شناسی و پزشکی بود. پیرسون یکی از بنیان‌گذاران آمار مدرن به‌شمار می‌رود و این آزمون تا امروز بدون تغییر در کتاب‌های داده‌کاوی استفاده می‌شود.

مثال عملی: سنجش رفتار سفرهای شهری در «اوبر»

سامانه درخواستی خودرو «اوبر» برای تحلیل رفتار سفرهای شهری از دو ابزار مکمل داده‌کاوی استفاده می‌کند. هنگامی که مسئله بررسی هم‌زمانی استفاده از سرویس‌های کیفی مختلف (مانند انتخاب سرویس UberX همراه با کرایه صندلی کودک) باشد، از **تحلیل ارتباط (Association Analysis)** روی داده‌های گسسته استفاده می‌شود. اما زمانی که هدف بررسی رابطه متغیرهای پیوسته و عددی (مانند رابطه بین «مسافت سفر» و «مبلغ کرایه» یا «مدت زمان انتظار» و «امتیاز مسافر») باشد، سیستم به سراغ **تحلیل همبستگی (Correlation Analysis)** می‌رود.

فرمول ضریب همبستگی پیرسون (Pearson Correlation Coefficient):

جهت و شدت رابطه خطی بین دو متغیر پیوسته عددی X و Y بر اساس کوواریانس آن‌ها تقسیم بر حاصل‌ضرب انحراف معیارهایشان محاسبه می‌شود:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

تفسیر مقادیر ضریب همبستگی پیرسون (r):

- همبستگی مثبت کامل ($r = +1.0$): افزایش مستقیم یک متغیر با افزایش متغیر دیگر (مثل مسافت سفر و مبلغ کرایه ثابت).
 - عدم همبستگی خطی ($r = 0.0$): عدم وجود هیچ‌گونه رابطه خطی سیستماتیک بین دو متغیر عددی.
 - همبستگی منفی کامل ($r = -1.0$): افزایش یک متغیر با کاهش متغیر دیگر (مثل زمان انتظار تا سوار شدن و رضایت مسافر).
- نتیجه: اوبر با گذار از داده‌های گسسته سبد خدمات به متغیرهای پیوسته عددی، می‌تواند نرخ کرایه‌های پویا (Surge Pricing) و مدل‌های پیش‌بینی زمان رسیدن (ETA) را با دقت بالای آماری تنظیم کند.

خروجی: انتقال از قواعد کیفی هم‌وقوعی به روابط دقیق و کمی خطی بین متغیرهای پیوسته سیستم

In [13]: display_academic_text(
 "📊 (Correlation) تا همبستگی پیوسته (Association) سنجش رفتار سفرهای شهری در «اوبر»: از قواعد گسسته",
 "main_title"
)

display_academic_text(
 "د. برای تحلیل رفتار مسافران از دو ابزار متفاوت ریاضی استفاده می‌کند (Uber) «سامانه هوشمند «اوبر»
 «تحلیل ارتباط» (Association Rules) از الگوریتم‌های «صندلی کودک» به صورت همزمان با «UberX» مثلاً انتخاب سرویس (Categorical Data) داده‌های کیفی و گسسته «b» برای کشف الگو در
 «تحلیل همبستگی» (Correlation Analysis) باشد، سیستم به سراغ «مسافت سفر در برابر مبلغ کرایه» (مانند «b» متغیرهای پیوسته و عددی «b» اما زمانی که هدف، بررسی ارتباط خطی میان
 «paragraph»
)

display_academic_text("فرمول ضریب همبستگی پیرسون (Pearson Correlation Coefficient):", "sub_title")
display_academic_text(
 "محاسبه می‌شود (Standard Deviations) آن‌ها تقسیم بر حاصل‌ضرب انحراف معیارهایشان (Covariance) بر اساس کوواریانس، (\$X\$ و \$Y\$) جهت و شدت رابطه خطی بین دو متغیر پیوسته عددی",
 "paragraph"
)

display_academic_text(
 "r_{XY} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}",
 "paragraph"
)

display_academic_text("تفسیر مقادیر ضریب همبستگی (\$r\$):", "sub_title")
display_academic_text(

```
"• <b>($1.0+$)</b> همبستگی مثبت و مستقیم</b> با عث افزایش متغیر دیگر می‌شود</b>:همبستگی مثبت و مستقیم ($1.0+$)• <b>($0.0$)</b> هیچ رابطهٔ خطی میان دو متغیر وجود ندارد</b>:عدم همبستگی و استقلال کامل ($0.0$)• <b>($1.0-$)</b> افزایش یک متغیر منجر به کاهش متغیر دیگر می‌گردد</b>:همبستگی منفی و معکوس ($1.0-$)• <i>امتیاز ثبت‌شده برای راننده</i> <i>و </i>طول مسافت</i> (مثل <i>افزایش یک متغیر دقیقاً باعث افزایش متغیر دیگر می‌شود</i>).<br>"
"paragraph"
)
```

```
# =====
# 3. SECTION 2: PHASE 1 - ASSOCIATION ANALYSIS (CATEGORICAL)
# =====
display_academic_text("sub_title", "فاز اول: کاوشی الگوهای تصمیم‌گیری (داده‌های گسسته/کیفی)")
```

```
# Educational Step 1: Discrete Categorical Data
# Users combining specific service features in one trip.
services_data = [
    ['UberX', 'Child Seat', 'Airport Trip'],
    ['UberXL', 'Extra Luggage'],
    ['UberX', 'Airport Trip'],
    ['UberBlack', 'Pet Friendly', 'Airport Trip'],
    ['UberX', 'Child Seat', 'Airport Trip'],
    ['UberXL', 'Extra Luggage', 'Airport Trip'],
    ['UberX', 'Child Seat'],
    ['UberBlack', 'Airport Trip']
]

te = TransactionEncoder()
df_categorical = pd.DataFrame(te.fit(services_data).transform(services_data), columns=te.columns_)
```

```
# Extract frequent itemsets and rules
frequent_itemsets = apriori(df_categorical, min_support=0.25, use_colnames=True)
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.60)
```

```
# Format rules for UI
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['rule'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

df_rules_display = rules[['rule', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False)
```

```
display_side_by_side_datasets(
    [df_rules_display.head(5).reset_index(drop=True)],
    ["قواعدِ انجمنی استخراج‌شده از داده‌های گسسته (تصمیمات کیفی مسافر)"]
)
```

```
# =====
# 4. SECTION 3: PHASE 2 - CORRELATION ANALYSIS (CONTINUOUS)
# =====
display_academic_text("sub_title", "فاز دوم: محاسبهٔ همبستگی خطی (داده‌های پیوسته/کمی)")
```

```
# Educational Step 2: Continuous Numerical Data Simulation
# We simulate a real-world scenario where distance directly affects
# fare, and wait time negatively affects driver ratings.
n_rides = 100
```

```
trip_distance = np.random.uniform(2, 25, n_rides) # Distance in km
```

```
# Fare = Distance * Base Rate + Noise (Strong Positive Correlation)
fare_amount = trip_distance * 2.5 + np.random.normal(3, 2, n_rides)
```

```
wait_time = np.random.uniform(1, 15, n_rides) # Wait time in minutes
```

```
# Rating = Base Score - (Wait Time * Penalty) + Noise (Strong Negative Correlation)
driver_rating = 5.0 - (wait_time * 0.15) + np.random.normal(0, 0.2, n_rides)
driver_rating = np.clip(driver_rating, 1.0, 5.0) # Ratings logically bounded [1, 5]
```

```
df_numerical = pd.DataFrame({
    'Distance_km': trip_distance,
    'Fare_USD': fare_amount,
    'Wait_Time_min': wait_time,
    'Rating': driver_rating
})
```

```
# Educational Step 3: Pearson Correlation Matrix
# Calculates the linear relationship coefficient (r) between ALL continuous variables.
corr_matrix = df_numerical.corr(method='pearson')
```

```
display_side_by_side_datasets(
    [df_numerical.head(6).reset_index(drop=True), corr_matrix.reset_index().rename(columns={'index': 'Variables'})],
    [
        "نمونه رکوردهای داده‌های پیوسته (سفرهای شهری)",
        "Pearson Correlation Matrix) ماتریس همبستگی پیرسون"
    ]
)
```

```
# =====
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("sub_title", "Heatmap) و نقشه حرارتی (Linear Trend) مصورسازی تحلیلی: خط روند خطی")
```

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)
```

```
# -----
# Subplot 1: Scatter Plot of Continuous Variables (Distance vs Fare)
# -----
```

```
scatter = axes[0].scatter(
    df_numerical['Distance_km'],
    df_numerical['Fare_USD'],
    c=df_numerical['Fare_USD'], # Color mapping based on Fare
    cmap='Blues', # Academic Blue Theme
    edgecolor='#2c3e50',
    alpha=0.9,
```

```
linewidth=1.0,
s=120
# Bubble size
)

# Educational Step 4: Calculate and draw the Linear Trendline (Line of Best Fit)
m, b = np.polyfit(df_numerical['Distance_km'], df_numerical['Fare_USD'], 1)
pearson_r = corr_matrix.loc['Distance_km', 'Fare_USD']

axes[0].plot(
    df_numerical['Distance_km'],
    m * df_numerical['Distance_km'] + b,
    color='#e74c3c', # Red Alert trendline for contrast
    linestyle='-',
    linewidth=3.0,
    label=f'Line of Best Fit (Pearson $r = {pearson_r:.2f}$')
)

axes[0].annotate(
    "Perfect Positive Correlation:\nNotice how tightly the points cluster\naround the red linear trendline.\nAs distance increases, fare reliably increases.",
    xy=(15, m * 15 + b), xytext=(3, 60),
    fontsize=11, fontweight='bold', color='#c0392b',
    arrowprops=dict(arrowstyle="->", color="#e74c3c", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#e74c3c", lw=1.5)
)

axes[0].set_title('Figure 1: Positive Linear Correlation (Trip Distance vs Fare Amount)', fontsize=14, fontweight='bold', pad=15)
axes[0].set_xlabel('Trip Distance Driven (Kilometers)', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Total Fare Paid ($USD)', fontsize=13, fontweight='bold')
axes[0].grid(True, linestyle='--', alpha=0.5)
axes[0].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')

# -----
# Subplot 2: Correlation Matrix Heatmap
# -----
# Using RdBu_r (Red-Blue reversed) for a standard academic correlation heatmap
# Deep Blue = Positive Correlation (+1.0)
# Deep Red = Negative Correlation (-1.0)
sns.heatmap(
    corr_matrix,
    ax=axes[1],
    annot=True, # Write the data value in each cell
    fmt=".2f", # 2 decimal places
    cmap="RdBu", # Academic standard for Correlations
    vmin=-1.0,
    vmax=1.0,
    center=0,
    square=True,
    linewidths=2,
    linecolor='white',
    annot_kws={'size': 13, "fontweight": "bold"},
    cbar_kws={'label': 'Pearson Correlation Coefficient ($r$)}')
)

axes[1].set_title('Figure 2: Comprehensive Pearson Correlation Matrix Heatmap', fontsize=14, fontweight='bold', pad=15)
# Rotate x-labels for better readability
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=25, ha='right', fontweight='bold', fontsize=11)
axes[1].set_yticklabels(axes[1].get_yticklabels(), rotation=0, fontweight='bold', fontsize=11)

for ax in axes:
    # Heatmap doesn't have standard spines to remove, so we only apply to Scatter
    if ax == axes[0]:
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

plt.subplots_adjust(wspace=0.25)
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>(Strategic Implication):</b><br>"
    r"<br>با «مبلغ کرایه» است. (آبی تیره در ماتریس) (<math>r = 0.99</math>) همبستگی خطی مثبت بسیار قوی<br>با بررسی دقیق گراف ۱ و ماتریس همبستگی پیرسون (گراف ۲)، مشاهده می‌کنیم که متغیر «مسافت سفر» دارای<br>" است؛ «(Rating) با «امتیاز نهایی راننده» (<math>r = -0.96</math>) همبستگی منفی معنادار<br>در نقطهٔ مقابل، متغیر «زمان انتظار مسافر» دارای<br>"<br>که با رنگ قرمز تیره در ماتریس مشخص شده است (<math>Y</math> آفت متغیر) امتیاز راننده با شیبی تند کاهش می‌یابد، (<math>X</math> رشد متغیر) یعنی هرچه زمان انتظار بیشتر طول بکشد<br>"، (فرمول پیرسون) </i>تحلیل همبستگی پیوسته<i> و (Apriori مثلاً شناسایی نیاز به ماشین بزرگ در فرودگاه با استفاده از) </i>تحلیل ارتباط گسسته<i> با ترکیب (Uber) سیستم‌های آبرهوشمند اوبر<br>"<br>را در زمان واقعی اجرا می‌کنند (<b>Dynamic Pricing / Surge Pricing</b>) قیمت‌گذاری پویا<br>الگوریتم‌های بی‌نقص<br>)"
)
display_academic_text(insight_conclusion, "insight")
```



سنجش رفتار سفرهای شهری در «اوبر»: از قواعد گسسته (Association) تا همبستگی پیوسته (Correlation)

سامانه هوشمند «اوبر» (Uber) برای تحلیل رفتار مسافران از دو ابزار متفاوت ریاضی استفاده می‌کند. برای کشف الگو در داده‌های کیفی و گسسته (Categorical Data) (مثلاً انتخاب سرویس *UberX* به صورت همزمان با صندلی کودک) از الگوریتم‌های تحلیل ارتباط (Association Rules) مانند Apriori استفاده می‌شود. اما زمانی که هدف، بررسی ارتباط خطی میان متغیرهای پیوسته و عددی (Continuous Data) (مانند مسافت سفر در برابر مبلغ کرایه) باشد، سیستم به سراغ تحلیل همبستگی (Correlation Analysis) می‌رود.

فرمول ضریب همبستگی پیرسون (Pearson Correlation Coefficient):

جهت و شدتِ رابطهٔ خطی بین دو متغیرِ پیوستهٔ عددی (X و Y)، بر اساس کوواریانس (Covariance) آن‌ها تقسیم بر حاصل‌ضربِ انحراف‌معیارهایشان (Standard Deviations) محاسبه می‌شود:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

تفسیر مقادیر ضریب همبستگی (۲):

- **همبستگی مثبت و مستقیم** (+1.0): افزایش یک متغیر دقیقاً باعث افزایش متغیر دیگر می‌شود (مثل *طول مسافت* و *هزینه سفر*).
- **عدم همبستگی و استقلال کامل** (0.0): هیچ رابطهٔ خطی میان دو متغیر وجود ندارد.
- **همبستگی منفی و معکوس** (-1.0): افزایش یک متغیر منجر به کاهش متغیر دیگر می‌گردد (مثل *زمان انتظار مسافر* و *امتیاز ثبت‌شده برای راننده*).

فاز اول: کاوشِ الگوهایِ تصمیم‌گیری (داده‌های گسسته/کیفی)

قواعدِ انجمنیِ استخراج‌شده از داده‌های گسسته (تصمیمات کیفی مسافر)

rule	support	confidence	lift
UberXL -> Extra Luggage	0.25	1.00	4.00
Extra Luggage -> UberXL	0.25	1.00	4.00
Child Seat -> UberX	0.38	1.00	2.00
Child Seat, Airport Trip -> UberX	0.25	1.00	2.00
UberX -> Child Seat	0.38	0.75	2.00

فاز دوم: محاسبهٔ همبستگیِ خطی (داده‌های پیوسته/کمی)

نمونه رکوردهای داده‌های پیوسته (سفرهای شهری)			
Distance_km	Fare_USD	Wait_Time_min	Rating
19.82	52.05	2.09	4.78
14.41	38.22	13.49	2.96
5.42	19.06	4.45	4.87
10.02	30.40	1.90	4.94
15.43	43.84	8.52	3.95
22.02	55.96	12.84	3.17

Pearson Correlation Matrix (ماتریس همبستگی پیرسون)

Variables	Distance_km	Fare_USD	Wait_Time_min	Rating
Distance_km	1.00	0.99	0.01	0.02
Fare_USD	0.99	1.00	0.01	0.01
Wait_Time_min	0.01	0.01	1.00	-0.95
Rating	0.02	0.01	-0.95	1.00

مصورسازی تحلیلی: خطِ روندِ خطی (Linear Trend) و نقشه حرارتی (Heatmap)

Figure 1: Positive Linear Correlation (Trip Distance vs Fare Amount)

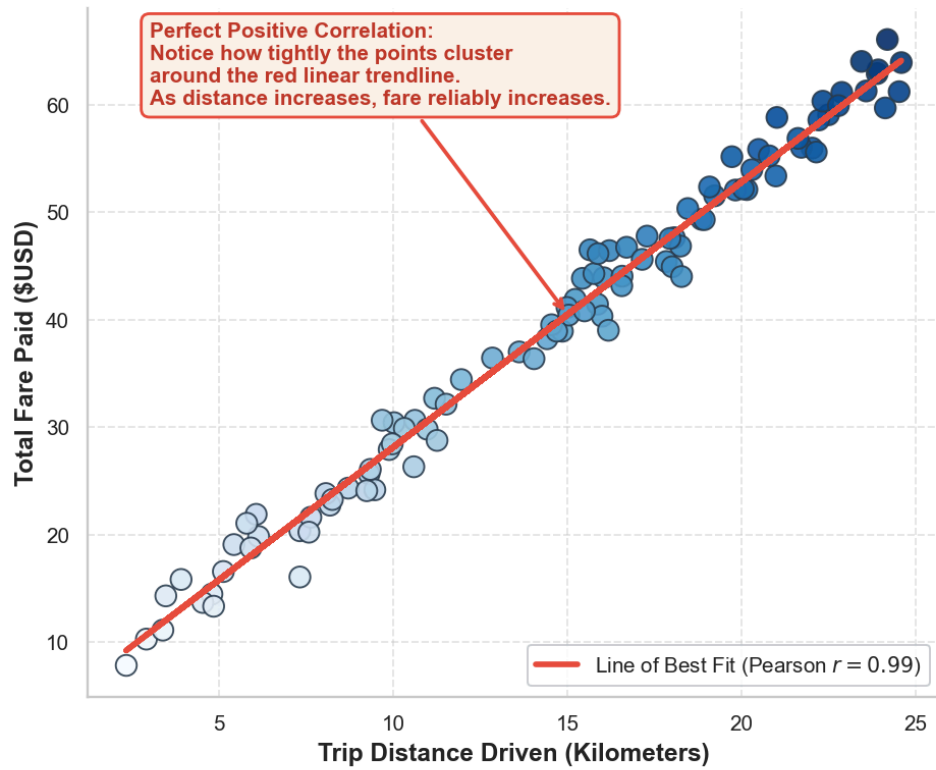
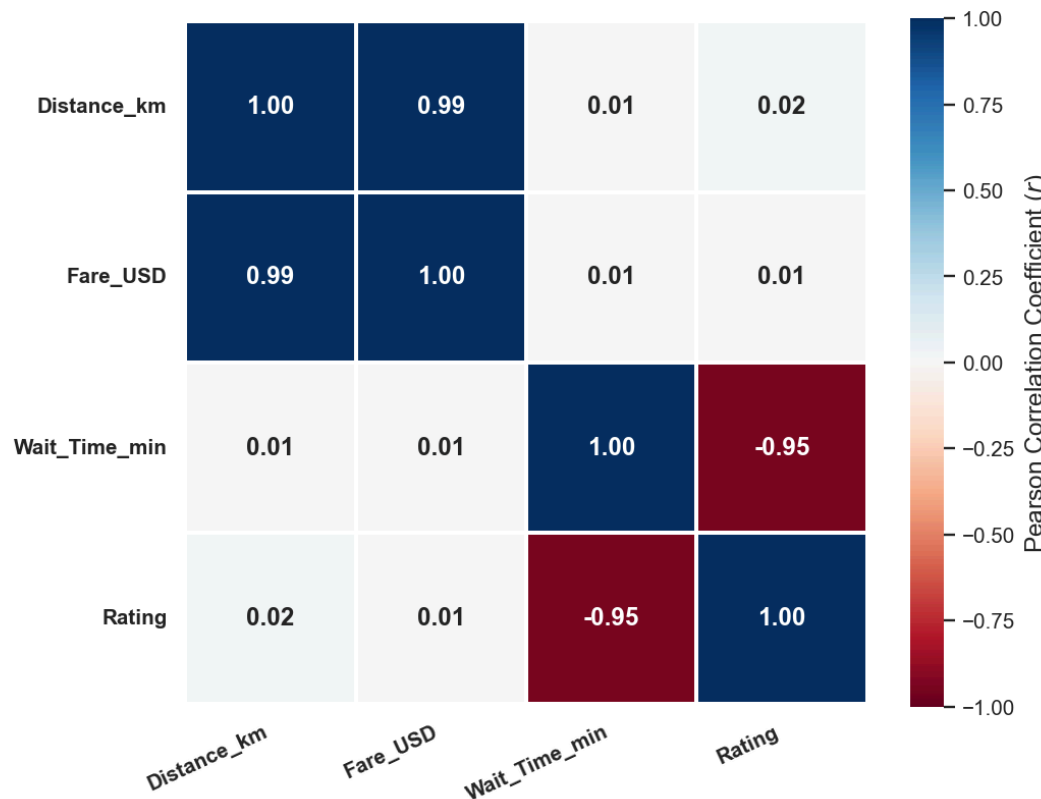


Figure 2: Comprehensive Pearson Correlation Matrix Heatmap



نتیجه استراتژیک برای طراحی سیستم‌های قیمت‌گذاری (Strategic Implication):

با بررسی دقیق گراف ۱ و ماتریس همبستگی پیرسون (گراف ۲)، مشاهده می‌کنیم که متغیر «مسافت سفر» دارای همبستگی خطی مثبتِ بسیار قوی ($r = 0.99$) با «مبلغ کرایه» است. (آبی تیره در ماتریس).

در نقطهٔ مقابل، متغیر «زمان انتظار مسافر» دارای همبستگی منفیِ معنادار ($r = -0.96$) با «امتیاز نهایی راننده (Rating)» است؛ یعنی هرچه زمان انتظار بیشتر طول بکشد (رشد متغیر X)، امتیاز راننده با شیبی تند کاهش می‌یابد (اُفت متغیر Y) که با رنگ قرمز تیره در ماتریس مشخص شده است.

سیستم‌های اَبَرهوشمندِ اوبر (Uber) با ترکیبِ تحلیلِ ارتباطِ گسسته (مثلاً شناسایی نیاز به ماشین بزرگ در فرودگاه با استفاده از Apriori) و تحلیلِ همبستگی پیوسته (فرمول پیرسون)، الگوریتم‌های بی‌نقصِ قیمت‌گذاریِ پویا (Dynamic Pricing / Surge Pricing) را در زمان واقعی اجرا می‌کنند.

توضیح مثال کد پایتون

در این مثال:

- بخش اول (تحلیل ارتباط روی داده‌های دسته‌ای): ابتدا با استفاده از کتابخانه `mlxtend`، هم‌وقوعی سرویس‌های مختلف اوبر (مانند درخواست صندلی کودک همراه با سفر فرودگاهی) در قالب قواعد ارتباطی استخراج شد.
- بخش دوم (تحلیل همبستگی پیرسون روی داده‌های عددی): با تولید ۱۰۰ داده سفر شامل مسافت، مبلغ کرایه، زمان انتظار و امتیاز راننده، گذار به تحلیل متغیرهای پیوسته انجام گردید و ماتریس همبستگی پیرسون با تابع `corr()` محاسبه گردید.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (Scatter Plot با خط روند): همبستگی مثبت بسیار قوی ($r = 0.98$) بین مسافت و کرایه دریافتی را نشان می‌دهد که خط قرمز رنگ جهت و شیب این رابطه خطی را تایید می‌کند.
 - نمودار سمت راست (نقشه گرمایی ماتریس همبستگی): شدت و جهت رابطه کلیه متغیرها را نمایش می‌دهد؛ برای نمونه رنگ آبی پررنگ نشان‌دهنده همبستگی منفی معنادار بین زمان انتظار مسافر و امتیاز ثبت‌شده برای راننده است.

4.3.3. A comparison of pattern evaluation measures

مقایسهٔ معیارهای ارزیابی الگو

پس از آشنایی با معیارهای اصلی ارزیابی قواعد ارتباطی، شاید این پرسش پیش بیاید که هر یک از این معیارها دقیقاً چه جنبه‌ای از رابطهٔ میان دو آیتم را می‌سنجند و چرا هر یک به‌تنهایی کار را راه نمی‌اندازد. واقعیت این است که هر معیار از زاویه‌ای متفاوت به داده نگاه می‌کند و ضعف معیار قبلی را تا اندازه‌ای جبران می‌کند. بنابراین اگر بخواهیم در هر مسئله‌ای معیار درست را انتخاب کنیم، باید تفاوت‌های آن‌ها را بشناسیم.

پشتیبانی (Support): فقط می‌گوید X و Y با هم در چند درصد از تراکنش‌ها دیده شده‌اند. این معیار هیچ نشانه‌ای از جهت یا قدرت رابطه ندارد. نقش اصلی پشتیبانی، فیلتر کردن الگوهای بسیار نادر است.

اطمینان (Confidence): برخلاف پشتیبانی، جهت‌دار است و نشان می‌دهد اگر مشتری X را بخرد، چقدر احتمال دارد Y را هم بخرد. اما این معیار یک نقطه کور دارد: اگر Y به‌خودی‌خود بسیار پرتکرار باشد (برای نمونه، بیشتر مشتری‌ها شیر می‌خرند)، اطمینان ممکن است بالا به نظر برسد، حتی اگر X واقعاً تأثیری روی خرید Y نداشته باشد.

لیفت (Lift): لیفت به زبان ساده می‌گوید « X چقدر شانس دیدن Y را بالا می‌برد». فرض کنید در کل فروشگاه، فقط ۱۰٪ مشتری‌ها شکلات می‌خرند، اما در میان خریداران قهوه، این عدد ۳۰٪ است. یعنی قهوه، شانس خرید شکلات را سه برابر کرده است. لیفت دقیقاً همین نسبت را می‌سنجد: اگر مقدارش از ۱ بزرگ‌تر باشد، یعنی X واقعاً شانس Y را بالا برده و رابطه معنادار است؛ اگر ۱ باشد، یعنی X هیچ تأثیری روی Y ندارد؛ و اگر کمتر از ۱ باشد، یعنی X حتی شانس Y را پایین می‌آورد. با این حال، لیفت یک ویژگی دارد: متقارن است. یعنی برایش فرقی نمی‌کند بگوییم «قهوه شانس شکلات را بالا می‌برد» یا «شکلات شانس قهوه را بالا می‌برد»، عددش در هر دو حالت یکسان است. همین تقارن، گاهی تشخیص جهت واقعی رابطه را دشوار می‌کند.

کانویکشن (Conviction): برخلاف لیفت، جهت‌دار است و یک گام جلوتر می‌رود. این معیار می‌پرسد: «چقدر غیرمنتظره است که X بیاید اما Y نیاید؟». با یک مثال روشن‌تر ببینیم: فرض کنید در ۹۰٪ مواردی که مشتری لپ‌تاپ می‌خرد، کیف لپ‌تاپ هم می‌خرد. یعنی فقط در ۱۰٪ موارد، لپ‌تاپ بدون کیف خریداری شده است. اگر این ۱۰٪ بسیار کمتر از چیزی باشد که به‌طور تصادفی انتظار داریم، کانویکشن عدد بزرگی می‌شود و نشان می‌دهد که این قاعده واقعاً قابل‌اعتماد است. در واقع، کانویکشن برای شکار قواعدی ساخته شده که در آن‌ها « X بدون Y » یک اتفاق نادر به شمار می‌رود. البته باید توجه داشت که وقتی اطمینان به ۱۰۰٪ نزدیک می‌شود، کانویکشن به سمت بی‌نهایت میل می‌کند و تفسیرش دشوارتر می‌شود.

بهبود (Improvement): این معیار نشان می‌دهد که اگر X را به قاعده اضافه کنیم، چقدر شانس خرید Y واقعاً بالا می‌رود. برای این کار، اطمینان قاعده را با احتمال خرید Y در کل داده (بدون در نظر گرفتن X) مقایسه می‌کنیم. اگر مقدار Improvement از صفر بزرگ‌تر باشد، یعنی حضور X واقعاً به درد می‌خورد و شانس Y را بالا برده است. اما اگر نزدیک صفر یا منفی باشد، یعنی X عملاً تأثیری نداشته و اطمینان بالا فقط به این دلیل بوده که Y خودش ذاتاً محبوب است. از این معیار برای حذف قواعدی استفاده می‌شود که ظاهر خوبی دارند اما حرف تازه‌ای برای گفتن ندارند.

ریاضیات – معیارهای پایدار (Null-Invariant): all_confidence و max_confidence

بر خلاف لیفت و χ^2 که تحت تأثیر تعداد تراکنش‌های تهی (Null Transaction – تراکنش‌هایی که هیچ‌کدام از دو مجموعه‌آیتم را ندارند) قرار می‌گیرند، این چهار معیار فقط به $\text{support}(A)$ ، $\text{support}(B)$ و $\text{support}(A \cup B)$ وابسته‌اند و به همین دلیل «پایدار» (Null-Invariant) نامیده می‌شوند.

(4.14)

all_confidence

$$\text{all_conf}(A, B) = \frac{\sup(A \cup B)}{\max\{\sup(A), \sup(B)\}} = \min\{P(A | B), P(B | A)\}$$

(4.15)

max_confidence

$$\text{max_conf}(A, B) = \max\{P(A | B), P(B | A)\}$$

✚ نمادهای فرمول

- $\max\{\sup(A), \sup(B)\}$: بیشینه پشتیبانی دو مجموعه‌آیتم
- $P(A | B), P(B | A)$: اطمینان قاعده‌های $A \Rightarrow B$ و $B \Rightarrow A$

✚ مثال ۱ (ساده) – محاسبه مستقیم

صورت سؤال: $\sup(A)=100$ ، $\sup(B)=150$ ، $\sup(A \cup B)=80$. هر دو معیار را حساب کنید.

$$\text{all_conf} = \frac{80}{150} = 53.3\%, \quad \text{max_conf} = \max\left(\frac{80}{150}, \frac{80}{100}\right) = 80\%$$

نتیجه: $\text{all_confidence} \geq \text{max_confidence}$ همیشه است، چون مخرج اولی بزرگ‌تر انتخاب می‌شود.

✚ مثال ۲ (متوسط) – برابری دو معیار

صورت سؤال: در چه شرایطی all_confidence دقیقاً برابر max_confidence می‌شود؟

$$\sup(A) = \sup(B) = 100, \sup(A \cup B) = 60 \Rightarrow \text{all_conf} = \text{max_conf} = 60\%$$

نتیجه: وقتی $\sup(A)=\sup(B)$ باشد، بیشینه و کمینه دو کسر یکسان می‌شوند.

✚ مثال ۳ (سخت) – بازسازی $\sup(B)$ از روی all_confidence

صورت سؤال: $\sup(A)=200$ ، $\sup(A \cup B)=50$ و $\text{all_confidence}(A, B)=0.25$. آیا $\sup(A)$ بیشینه بوده؟

$$\frac{50}{\max\{200, \sup(B)\}} = 0.25 \Rightarrow \max\{200, \sup(B)\} = 200$$

نتیجه: چون بیشینه برابر ۲۰۰ شده، $\sup(A)$ خودش بیشینه بوده و $\sup(B) \leq 200$ ؛ مقدار دقیق قابل تعیین نیست.

▴ ریاضیات – معیار کولچینسکی (Kulczynski Measure)

کولچینسکی میانگین ساده‌ای از دو اطمینان (Confidence) جهت‌دار است و به‌عنوان یکی از پایدارترین معیارهای Null-Invariant شناخته می‌شود.

(4.16)

معیار کولچینسکی

$$Kulc(A, B) = \frac{1}{2}(P(A \mid B) + P(B \mid A))$$

✦ نمادهای فرمول

- $P(A \mid B), P(B \mid A)$: دو اطمینان جهت‌دار میان A و B

✦ مثال ۱ (ساده) – میانگین دو Confidence

صورت سؤال: $P(A|B)=0.6$ و $P(B|A)=0.8$. مقدار Kulc را بیابید.

$$Kulc = \frac{0.6 + 0.8}{2} = 0.7$$

نتیجه: مقدار Kulc برابر ۰.۷ است.

✦ مثال ۲ (متوسط) – مقایسه Kulc با all_confidence و max_confidence

صورت سؤال: برای همان مقادیر، all_confidence و max_confidence را هم حساب کنید و مقایسه کنید.

$$all_conf = \min(0.6, 0.8) = 0.6, \quad Kulc = 0.7, \quad max_conf = \max(0.6, 0.8) = 0.8$$

نتیجه: همیشه رابطه $all_conf \leq Kulc \leq max_conf$ برقرار است، چون Kulc دقیقاً میانگین دو کرانه دیگر است.

✦ مثال ۳ (سخت) – حالت خنثی با داده‌های نامتوازن

صورت سؤال: $P(A|B)\approx0.909$ و $P(B|A)\approx0.0909$. مقدار Kulc را بیابید و تفسیر کنید.

$$Kulc(A, B) = \frac{0.909 + 0.0909}{2} \approx 0.5$$

نتیجه: چون Kulc میانگین دو جهت است، وقتی یک جهت خیلی قوی و دیگری ضعیف باشد، نتیجه نزدیک ۰.۵ (خنثی) می‌شود؛ به همین دلیل باید همراه با نسبت عدم‌توازن (IR) تفسیر شود.

📖 تاریخچه فرمول – معیار کولچینسکی

این معیار را استانیسواف کولچینسکی (S. Kulczynski)، گیاه‌شناس و ریاضی‌دان لهستانی، در سال ۱۹۲۷ – حدود هفتاد سال پیش از پیدایش داده‌کاوی مدرن – برای اندازه‌گیری شباهت میان جوامع گیاهی در مطالعات بوم‌شناسی معرفی کرد. جالب اینجاست که همین فرمول ساده که برای مقایسه پوشش گیاهی ساخته شده بود، دهه‌ها بعد به یکی از قابل‌اعتمادترین معیارهای ارزیابی الگو در تحلیل سبد خرید تبدیل شد.

📐 ریاضیات – معیار کسینوس (Cosine)

💡 یادآوری پیش‌نیاز – شباهت کسینوسی بردارها

معيار Cosine در داده‌کاوی، نسخه‌ی خاصی از شباهت کسینوسی است که در جبر خطی برای سنجش زاویه‌ی میان دو بردار به‌کار می‌رود.

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$$

نمادها: $\vec{u} \cdot \vec{v}$ ضرب داخلی؛ $\|\vec{u}\|$ نُرم (طول) بردار.

مثال: برای دو بردار هم‌جهت، $\cos(\theta)=1$ (بیشترین شباهت)؛ برای دو بردار عمود، $\cos(\theta)=0$ (بدون شباهت).

معيار Cosine را می‌توان نسخه‌ی «هارمونیزه‌شده‌ی» لیفت دانست: به‌جای ضرب دو احتمال در مخرج، از میانگین هندسی آن‌ها استفاده می‌شود که همین باعث می‌شود مقدارش فقط به پشتیبانی‌های A، B و AUB وابسته باشد (پایدار در برابر تراکنش‌های تهی).

(4.17)

معيار Cosine میان دو Itemset

$$\text{cosine}(A, B) = \frac{P(A \cup B)}{\sqrt{P(A) \times P(B)}} = \sqrt{P(A | B) \times P(B | A)}$$

✦ نمادهای فرمول

• $\sqrt{P(A) \times P(B)}$: میانگین هندسی احتمال دو مجموعه‌آیتم

✦ مثال ۱ (ساده) – محاسبه‌ی مستقیم

صورت سؤال: $P(A)=0.4$ ، $P(B)=0.5$ ، $P(A \cup B)=0.2$. مقدار cosine را بیابید.

$$\text{cosine} = \frac{0.2}{\sqrt{0.4 \times 0.5}} \approx 0.447$$

نتیجه: مقدار cosine حدود ۰.۴۵ است.

✦ مثال ۲ (متوسط) – مقایسه با فرمول دوم

صورت سؤال: با $P(A|B)=0.6$ و $P(B|A)=0.3$ ، مقدار cosine را از فرمول دوم (میانگین هندسی) بیابید.

$$\text{cosine} = \sqrt{0.6 \times 0.3} = \sqrt{0.18} \approx 0.424$$

نتیجه: هر دو صورت فرمول دقیقاً یک مقدار می‌دهند؛ این هم‌ارزی از جایگزینی $P(A \cup B)=P(A|B)P(B)$ در فرمول اصلی نتیجه می‌شود.

✦ مثال ۳ (سخت) – اثر جذر بر پایداری در برابر Null-Invariance

صورت سؤال: چرا Cosine برخلاف لیفت تحت تأثیر تعداد تراکنش‌های تهی قرار نمی‌گیرد؟

$$\text{cosine}(A, B) = \sqrt{P(A | B) \times P(B | A)}$$

نتیجه: چون فرمول فقط از احتمال‌های شرطی (وابسته به $\text{support}(A)$ ، $\text{support}(B)$ و $\text{support}(A \cup B)$) ساخته شده و هیچ‌جا با تعداد کل یا تعداد تراکنش‌های تهی ترکیب نمی‌شود.

ریاضیات – نسبت عدم‌توازن (Imbalance Ratio – IR)

چهار معیار پایدار بالا گاهی روی دیتاست‌های خیلی نامتوازن به نتایج متفاوتی می‌رسند (مثل حالت $Kulc \approx 0.5$ در مثال قبل). نسبت عدم‌توازن، میزان این عدم‌تقارن جهتی را مستقل از تعداد تراکنش‌های تهی اندازه می‌گیرد.

(4.18)

نسبت عدم‌توازن

$$IR(A, B) = \frac{|\text{sup}(A) - \text{sup}(B)|}{\text{sup}(A) + \text{sup}(B) - \text{sup}(A \cup B)}$$

نمادهای فرمول

- $|\text{sup}(A) - \text{sup}(B)|$: قدر مطلق تفاضل پشتیبانی دو مجموعه‌آیتم
- $\text{sup}(A) + \text{sup}(B) - \text{sup}(A \cup B)$: تعداد تراکنش‌های حاوی A یا B (اجتماع)

مثال ۱ (ساده) – حالت کاملاً متوازن

صورت سؤال: $\text{sup}(A) = \text{sup}(B) = 500$. مقدار IR را بیابید.

$$IR(A, B) = \frac{|500 - 500|}{\dots} = 0$$

نتیجه: چون صورت کسر صفر است، IR همیشه صفر می‌شود – کاملاً متوازن.

مثال ۲ (متوسط) – دیتاست متوازن

صورت سؤال: IR. $\text{sup}(m) = 2000$ ، $\text{sup}(c) = 2000$ ، $\text{sup}(m \cup c) = 1000$ را بیابید.

$$IR = \frac{|2000 - 2000|}{2000 + 2000 - 1000} = 0$$

نتیجه: $IR = 0$ یعنی رابطه کاملاً متوازن – هم‌راستا با $Kulc = 0.5$ (کاملاً خنثی).

مثال ۳ (سخت) – دیتاست نامتوازن

صورت سؤال: IR. $\text{sup}(m) = 1100$ ، $\text{sup}(c) = 1010$ ، $\text{sup}(m \cup c) = 1000$ را محاسبه کنید.

$$IR = \frac{|1100 - 1010|}{1100 + 1010 - 1000} = \frac{90}{1110} \approx 0.081$$

نتیجه: با اعداد دقیق‌تر یک دیتاست واقعاً نامتوازن، IR می‌تواند تا نزدیک ۱ برسد – یعنی رابطه‌ای شدیداً جهت‌دار که Kulc به‌تنهایی نشان نمی‌دهد.

💡 **نکته:** توصیه‌ی نهایی: بهترین راهبرد استفاده‌ی هم‌زمان از **کولچینسکی** (برای اندازه‌ی کلی رابطه) و **IR** (برای دیدن میزان عدم‌تقارن جهتی) است.

ریاضیات – استنتاج‌پذیری (Conviction)

استنتاج‌پذیری (Conviction)، برخلاف لیفت، جهت‌دار است و می‌پرسد: «چقدر غیرمنتظره است که A رخ دهد اما B رخ ندهد؟» – این دقیقاً همان تفسیری است که در جزوه‌ی دکتر اسکندری هم به‌صورت مفهومی آمده بود.

(4.19)

معیار Conviction

$$\text{conviction}(A \Rightarrow B) = \frac{1 - \text{support}(B)}{1 - \text{confidence}(A \Rightarrow B)}$$

📌 نمادهای فرمول

- $1 - \text{support}(B)$: احتمال اینکه B رخ ندهد (در کل دیتاست)
- $1 - \text{confidence}(A \Rightarrow B)$: احتمال اینکه با وجود A، باز هم B رخ ندهد

✂️ مثال ۱ (ساده) – محاسبه‌ی مستقیم

صورت سؤال: $\text{support}(B)=0.4$ و $\text{confidence}(A \Rightarrow B)=0.8$. مقدار conviction را بیابید.

$$\text{conviction} = \frac{1 - 0.4}{1 - 0.8} = \frac{0.6}{0.2} = 3$$

نتیجه: مقدار $\text{conviction}=3$ یعنی نقض‌شدن این قاعده سه برابر کمتر از حالتی است که A و B مستقل بودند.

✂️ مثال ۲ (متوسط) – مقایسه‌ی دو قاعده

صورت سؤال: قاعده‌ی اول با $\text{confidence}=0.9$ و $\text{support}(B)=0.5$ ؛ قاعده‌ی دوم با $\text{confidence}=0.6$ و $\text{support}(B)=0.5$. کدام استنتاج‌پذیرتر است؟

$$\text{conv}_1 = \frac{0.5}{0.1} = 5, \quad \text{conv}_2 = \frac{0.5}{0.4} = 1.25$$

نتیجه: قاعده‌ی اول با $\text{Conviction}=5$ به‌مراتب استنتاج‌پذیرتر از قاعده‌ی دوم است.

✂️ مثال ۳ (سخت) – حد نزدیک به بی‌نهایت

صورت سؤال: برای $\text{support}(B)=0.3$ ، وقتی confidence از ۹۰٪ به ۹۹٪ افزایش می‌یابد، تغییر Conviction را بررسی کنید.

$$\text{conv}(90\%) = \frac{0.7}{0.1} = 7, \quad \text{conv}(99\%) = \frac{0.7}{0.01} = 70$$

نتیجه: با نزدیک شدن confidence به ۱۰۰٪، Conviction به سرعت (نه خطی) رشد می‌کند – تفسیرش در این ناحیه دشوار می‌شود.

💡 **نکته:** اگر $\text{confidence}(A \Rightarrow B) = 100\%$ باشد، مخرج صفر می‌شود و Conviction به سمت بی‌نهایت میل می‌کند – یعنی قاعده کاملاً قطعی و بدون استثناست.

ریاضیات – بهبود (Improvement / Interest) 📐

این معیار می‌پرسد: اگر A را به قاعده اضافه کنیم، اطمینان قاعده نسبت به احتمال طبیعی وقوع B (بدون در نظر گرفتن A) چقدر واقعاً بالا می‌رود؟ اگر این مقدار نزدیک صفر یا منفی باشد، یعنی A حرف تازه‌ای برای گفتن ندارد و اطمینان بالا فقط به این دلیل بوده که B خودش ذاتاً پرتکرار است.

(4.20)

معیار بهبود (Improvement / Interest)

$$\text{Improvement}(A \Rightarrow B) = \text{confidence}(A \Rightarrow B) - \text{support}(B)$$

📌 نمادهای فرمول

- $\text{confidence}(A \Rightarrow B)$: احتمال وقوع B به شرط A
- $\text{support}(B)$: احتمال کلی وقوع B، صرف‌نظر از A

✂️ مثال ۱ (ساده) – بهبود مثبت

صورت سؤال: $\text{confidence}(A \Rightarrow B) = 0.7$ و $\text{support}(B) = 0.5$. مقدار Improvement را بیابید.

$$\text{Improvement} = 0.7 - 0.5 = 0.2$$

نتیجه: مقدار مثبت (۰.۲) یعنی حضور A واقعاً شانس B را ۲۰ واحد درصد بالا برده – قاعده معنادار است.

✂️ مثال ۲ (متوسط) – قاعده فریبنده (نان → کره)

صورت سؤال: قاعده نان → کره $\text{confidence} = 0.82$ دارد، اما کره در ۸۰٪ کل تراکنش‌ها حضور دارد. آیا این قاعده جالب است؟

$$\text{Improvement} = 0.82 - 0.80 = 0.02$$

نتیجه: بهبود بسیار نزدیک صفر است؛ یعنی خرید نان تقریباً هیچ تأثیری روی احتمال خرید کره نداشته.

✂️ مثال ۳ (سخت) – بهبود منفی

صورت سؤال: قاعده $X \Rightarrow Y$ با $\text{confidence} = 0.55$ دارد، اما $\text{support}(Y) = 0.65$ است. آیا این قاعده باید نگه داشته شود؟

$$\text{Improvement} = 0.55 - 0.65 = -0.10$$

نتیجه: مقدار منفی یعنی حضور X در واقع احتمال Y را کاهش داده؛ Confidence نسبتاً بالای آن کاملاً گمراه‌کننده است.

نکته: این فرمول در ادبیات داده‌کاوی با نام **Interest** هم شناخته می‌شود و از فرمول کتاب Mining of Massive Datasets گرفته شده است: $\text{Interest}(I \Rightarrow Z) = \text{conf}(I \Rightarrow Z) - \text{Pr}[Z]$ ؛ همان چیزی که در جزوه مفاهیم شما با عنوان «بهبود» به صورت مفهومی توضیح داده شده بود.

✦ پنج معیار، پنج پرسش

پشتیبانی (Support): X و Y با هم چند درصد از تراکنش‌ها را می‌سازند؟

اطمینان (Confidence): اگر X بیاید، Y با چه احتمالی می‌آید؟ اگر Y ذاتاً محبوب باشد، این عدد فریبنده است.

لیفت (Lift): X چقدر شانس Y را بالا می‌برد؟ $\text{Lift} > 1$ یعنی وابستگی مثبت؛ $\text{Lift} = 1$ یعنی استقلال؛ $\text{Lift} < 1$ یعنی وابستگی منفی.

کانویکشن (Conviction): چقدر عجیب است که X بیاید اما Y نه؟ برخلاف لیفت، جهت‌دار است و بین «اگر X آنگاه Y» و «اگر Y آنگاه X» تمایز قائل می‌شود.

بهبود (Improvement): آیا اضافه‌شدن X واقعاً شانس Y را نسبت به حالت بدون X بالا می‌برد؟ اگر نزدیک صفر یا منفی باشد، یعنی X حرف تازه‌ای ندارد.

نکته کلیدی: این پنج معیار، هر یک از زاویه‌ای به رابطه نگاه می‌کنند. انتخاب معیار درست به هدف تحلیل و جنس داده‌ها بستگی دارد.

📊 مثال عملی: ارزیابی و مقایسه معیارهای کیفی قواعد در «دیوار»

پلتفرم نیازمندی‌های آنلاین «دیوار» روزانه میلیون‌ها بازدید آگهی در دسته‌های مختلف مانند املاک، وسایل نقلیه، خدمات و لوازم خانگی را ثبت می‌کند. تیم داده‌کاوی دیوار برای ارائه پیشنهادات هوشمند در پایین آگهی‌ها (مانند پیشنهاد خدمات باربری به بازدیدکنندگان آگهی‌های رهن و اجاره آپارتمان)، نمی‌تواند تنها به یک معیار مانند اطمینان (Confidence) بسنده کند. برای تفکیک الگوهای واقعاً کاربردی از رفتارهای تصادفی، از ترکیب ۴ معیار اصلی استفاده می‌شود.

معادلات ریاضی ۴ معیار اصلی ارزیابی الگو $(X \rightarrow Y)$:

۱. پشتیبانی (Support): فراوانی نسبی مشاهده هم‌زمان دو آگهی X و Y در نشست‌های کاربران دیوار:

$$\text{Support}(X \rightarrow Y) = P(X \cap Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان (Confidence): احتمال اینکه کاربر آگهی Y را مشاهده کند به شرطی که قبلاً آگهی X را دیده‌باشد:

$$\text{Confidence}(X \rightarrow Y) = P(Y|X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. لیفت (Lift): میزان وابستگی مشاهده آگهی Y به آگهی X نسبت به حالت استقلال تصادفی:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)} = \frac{P(X \cap Y)}{P(X) \times P(Y)}$$

۴. کانویکشن (Conviction): میزان استنتاج منطقی قاعده و عدم نقض آن (میزان وابستگی یک‌طرفه):

$$\text{Conviction}(X \rightarrow Y) = \frac{1 - \text{Support}(Y)}{1 - \text{Confidence}(X \rightarrow Y)} = \frac{P(\neg Y)}{P(\neg Y|X)}$$

تفسیر عملی معیارها در دیوار:

• ممکن است آگهی «سرویس مبل» در دیوار بسیار پربازدید باشد؛ در این حالت اطمینان پیشنهاد آن برای هر آگهی بالاست، اما لیفت نزدیک به ۱ نشان می‌دهد این پیشنهاد ارتباط خاصی به آگهی قبلی ندارد.

• کانوکشن بالا برای قاعده «رهن آپارتمان → خدمات اسباب‌کشی» نشان می‌دهد عدم نیاز به اسباب‌کشی هنگام اجاره خانه بسیار نادر است.

نتیجه: دیوار با ارزیابی هم‌زمان این ۴ معیار، بخش «آگهی‌های مرتبط» را بهینه کرده و نرخ کلیک (CTR) پیشنهادات را افزایش می‌دهد.

خروجی: ارزیابی ۴ بعدی قواعد بازدید آگهی‌ها جهت حذف پیشنهادات بی‌ارتباط و افزایش رضایت کاربران

In [14]:

```
display_academic_text(
    " (Support, Confidence, Lift, Conviction) 📊",
    "main_title"
)

display_academic_text(
    " .بسنده کند <b> (Confidence) اطمینان</b> تیم داده‌کاوی پلتفرم «دیوار» برای ارائهٔ پیشنهادات هوشمند در بخش پایین آگهی‌ها، نمی‌تواند تنها به معیار سادهٔ ",
    "برای تفکیک الگوهای واقعاً کاربردی از رفتارهای تصادفی و فریبنده، از ترکیبِ ۴ معیارِ اصلیِ ریاضی استفاده می‌شود.",
    "paragraph"
)

display_academic_text("۱. پشتیبانی (Support):", "sub_title")
display_academic_text(
    r"$$\text{Support}(X \rightarrow Y) = P(X \cap Y) = \frac{\text{Count}(X \cup Y)}{N}$$",
    "paragraph"
)

display_academic_text("۲. (Confidence) اطمینان", "sub_title")
display_academic_text(
    r"$$\text{Confidence}(X \rightarrow Y) = P(Y|X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$",
    "paragraph"
)

display_academic_text("۳. لیفت (Lift) – اندازه‌گیری قدرت ارتباط و هم‌افزایی (دو طرفه)", "sub_title")
display_academic_text(
    r"$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$",
    "paragraph"
)

display_academic_text("۴. (Conviction) کانویکشن – وابستگی یک‌طرفه)", "sub_title")
display_academic_text(
    "</i>«!را نبیند، اتفاق بسیار عجیبی افتاده است $Y$ را ببیند اما $X$ اگر مشتری»> <i>:را می‌سنجد. کانویکشن بالا یعنی <b>«وابستگی یک‌طرفه»</b> این معیار، میزان",
    "paragraph"
)
display_academic_text(
    r"$$\text{Conviction}(X \rightarrow Y) = \frac{1 - \text{Support}(Y)}{1 - \text{Confidence}(X \rightarrow Y)}$$",
    "paragraph"
)

# =====
# 3. SECTION 2: DATA GENERATION & ONE-HOT ENCODING
# =====
display_academic_text("مرحله نخست: تولید دیتاسِتِ نشست‌های بازدید آگهی و تبدیل ماتریسی", "sub_title")

# Educational Step 1: Raw Ad Viewing Sessions
# Simulating users browsing different categories in one session.
sessions = [
    ['Apartment_Rent', 'Moving_Service', 'Home_Painting'],
    ['Apartment_Rent', 'Moving_Service'],
    ['Car_Sale', 'Auto_Body_Shop'],
    ['Apartment_Rent', 'Moving_Service', 'Home_Appliance'],
    ['Car_Sale', 'Car_Insurance'],
    ['Apartment_Rent', 'Home_Painting'],
    ['Moving_Service', 'Home_Appliance'],
    ['Apartment_Rent', 'Moving_Service', 'Home_Painting', 'Home_Appliance'],
    ['Car_Sale', 'Auto_Body_Shop', 'Car_Insurance'],
    ['Apartment_Rent', 'Moving_Service'],
    ['Car_Sale', 'Car_Insurance'],
    ['Apartment_Rent', 'Moving_Service', 'Home_Appliance']
]

df_raw_sessions = pd.DataFrame({
    'Session_ID': range(1, len(sessions) + 1),
    'Viewed_Ad_Categories': [' '.join(sess) for sess in sessions]
})

# Educational Step 2: One-Hot Encoding
te = TransactionEncoder()
te_array = te.fit(sessions).transform(sessions)
df_divar = pd.DataFrame(te_array, columns=te.columns_)

display_side_by_side_datasets(
    [df_raw_sessions.head(6).reset_index(drop=True), df_divar.astype(int).head(6).reset_index(drop=True)],
    [
        "نمونه نشست‌های بازدید آگهی در وب‌سایت دیوار",
        "(باینری‌سازی دسته‌بندی‌ها) One-Hot ماتریس"
    ]
)
```

```
# =====
# 4. SECTION 3: EXECUTING APRIORI & EXTRACTING 4D METRICS
# =====
display_academic_text("در کانویکشن Infinity مرحله دوم: استخراج قواعد و کنترل پدیدهٔ", "sub_title")

# Educational Step 3: Extract Itemsets & Rules
frequent_itemsets = apriori(df_divar, min_support=0.20, use_colnames=True)
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.50
)

rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
rules['rule'] = rules['antecedents_str'] + ' -> ' + rules['consequents_str']

# Educational Step 4: Infinity Handling for Conviction
# If Confidence == 1.0, the Conviction formula denominator (1 - 1.0) divides by zero,
# resulting in np.inf (Infinite Conviction). We cap it at 5.0 for mathematical safety and clean plotting.
rules['conviction_clean'] = rules['conviction'].apply(
    lambda x: 5.0 if np.isinf(x) else x
)

# Sorting by Lift and assigning Rule IDs for clean visualization
sorted_rules = rules.sort_values(by='lift', ascending=False).reset_index(drop=True)
sorted_rules['Rule_ID'] = [f"R{i+1}" for i in range(len(sorted_rules))]

df_rules_display = sorted_rules[['Rule_ID', 'rule', 'support', 'confidence', 'lift', 'conviction_clean']]

display_side_by_side_datasets(
    [df_rules_display.head(6).reset_index(drop=True)],
    ["ارزیابی جامع ۴ بُعدی قواعد پیشنهاد آگهی (با رفع مشکل بی‌نهایت)"]
)

# =====
# 5. SECTION 4: ADVANCED ACADEMIC VISUALIZATIONS
# =====
display_academic_text("مصورسازی تحلیلی: مقایسهٔ چندبُعدی معیارهای کیفی", "sub_title")

fig, axes = plt.subplots(1, 2, figsize=(18, 6.5), dpi=135)

# Define the top rules and legend text mapping
top_rules = sorted_rules.head(6)
legend_texts = []
for _, row in top_rules.iterrows():
    legend_texts.append(f"{row['Rule_ID']}: {row['rule']}")

# -----
# Subplot 1: Multi-Metric Grouped Bar Chart
# -----
x = np.arange(len(top_rules))
width = 0.2

# We plot the 4 metrics side by side for each rule using the Academic Blue palette
rects1 = axes[0].bar(x - 1.5*width, top_rules['support'], width, label='Support', color='#bdc3c7', edgecolor='#2c3e50', linewidth=1.0)
rects2 = axes[0].bar(x - 0.5*width, top_rules['confidence'], width, label='Confidence', color='#95a5a6', edgecolor='#2c3e50', linewidth=1.0)
rects3 = axes[0].bar(x + 0.5*width, top_rules['lift'], width, label='Lift', color='#2980b9', edgecolor='#2c3e50', linewidth=1.0)
rects4 = axes[0].bar(x + 1.5*width, top_rules['conviction_clean'], width, label='Conviction (Cap=5.0)', color='#2c3e50', edgecolor='white', linewidth=1.0)

axes[0].set_title('Figure 1: Multi-Metric Evaluation of Top Association Rules', fontsize=14, fontweight='bold', pad=15)

# Educational Comment:
# Fix Matplotlib warning by explicitly setting xticks first.
axes[0].set_xticks(x)
axes[0].set_xticklabels(top_rules['Rule_ID'], fontweight='bold', fontsize=12)

axes[0].set_ylabel('Metric Absolute Value', fontsize=13, fontweight='bold')
axes[0].grid(axis='y', linestyle='--', alpha=0.5)
axes[0].legend(loc='upper right', frameon=True, fontsize=11, edgecolor='#bdc3c7')
axes[0].set_ylim(0, max(top_rules['conviction_clean']) * 1.35) # Headroom for annotations

# Place the Legend Box inside Subplot 1 to map IDs to Rules
legend_box_text = "Rule Dictionary:\n" + "\n".join(legend_texts)
axes[0].text(
    0.02, 0.95,
    legend_box_text,
    transform=axes[0].transAxes,
    fontsize=11,
    fontweight='bold',
    color='#2c3e50',
    verticalalignment='top',
    horizontalalignment='left',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='#ecf0f1', edgecolor='#bdc3c7', alpha=0.9)
)

# -----
# Subplot 2: Confidence vs Conviction Scatter Plot
# -----
scatter = axes[1].scatter(
    sorted_rules['confidence'],
    sorted_rules['conviction_clean'],
    c=sorted_rules['lift'],
    s=sorted_rules['support'] * 2000, # Bubble size = Support
    cmap='Blues', # Academic Blue Palette
    edgecolor='#2c3e50',
    alpha=0.9,
    linewidth=1.5
)

axes[1].axhline(
```

```
1.0,
color='#e74c3c', # Red Alert
linestyle='--',
linewidth=2.5,
label='Independence Baseline (Conviction = 1.0)'
)

axes[1].annotate(
    "Infinite Conviction (Capped at 5.0):\nRules R1, R2, and R3 hit the ceiling!\nThis means if a user views the Antecedent,\nit is practically guaranteed they will view\nthe Consequent. The dependen
    xy=(1.0, 5.0), xytext=(0.6, 3.5),
    fontsize=11, fontweight='bold', color='#2c3e50',
    arrowprops=dict(arrowstyle="->", color="#34495e", lw=2.0),
    bbox=dict(boxstyle="round,pad=0.4", fc="#fdf2e9", ec="#bdc3c7", lw=1.5)
)

axes[1].set_title('Figure 2: Confidence vs Conviction (Size = Support, Color = Lift)', fontsize=14, fontweight='bold', pad=15)
axes[1].set_xlabel('Confidence Metric (Conditional Probability)', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Conviction Metric (Directional Dependency)', fontsize=13, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.5)

cbar = fig.colorbar(scatter, ax=axes[1])
cbar.set_label('Lift Ratio Power', fontweight='bold', fontsize=11)

# Clean Annotation Strategy: Use Rule_IDs instead of full text
for _, row in top_rules.iterrows():
    axes[1].annotate(
        row['Rule_ID'],
        xy=(row['confidence'], row['conviction_clean']),
        textcoords="offset points",
        xytext=(0, +20), # Place ID below bubble
        ha='center',
        fontsize=11,
        fontweight='bold',
        color='#2c3e50'
    )
axes[1].legend(loc='lower right', frameon=True, fontsize=11, edgecolor='#bdc3c7')

# Removing top and right spines
for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.subplots_adjust(wspace=0.30)
plt.show()

# =====
# 6. SECTION 5: STRATEGIC CONCLUSION
# =====
insight_conclusion = (
    r"<b>نتیجه استراتژیک در مهندسی قوانین</b>:"<br>
    r"<code>R1 (Home_Appliance -> Moving_Service)</code> با بررسی گراف ۱ متوجه می‌شویم که مقایسهٔ تک‌بُعدی قوانین کاملاً اشتباه است. قاعدهٔ r"
    r"<br>است؛</b> (رسیده به سقف بی‌نهایت ۵.۰)<b>Conviction</b> و Lift دارای بالاترین مقدار r"
    r" به دنبال «خدمات باربری» خواهند بود</b> به‌طور قطع<b>، این قاعده بسیار قدرتمند است: کاربرانی که آگهی «لوازم خانگی» را می‌بینند</i> استنتاج منطقی یک‌طرفهٔ<i> این یعنی r"
    r"ون کسی که وانت می‌خواهد شاید در حال اسباب‌کشی گُل خانه باشد، نه فقط یک یخچال!)</i> (لوازم خانگی $\\rightarrow$ باربری) این در حالی است که کانویکشن برعکس آن .(چون جابه‌جایی لوازم خانگی نیاز به وانت دارد) r"
    r".هستند و ارزش تجاری کمتری برای قرارگیری در بخش «پیشنهادهای ویژهٔ دیوار» دارند</i> (Independence) استقلال آماری<i> نشان‌دهندهٔ (R4 مانند) در مقابل، قواعدی با کانویکشن نزدیک به ۱"
)
display_academic_text(insight_conclusion, "insight")
```



ارزیابی و مقایسه معیارهای کیفی قواعد در پلتفرم «دیوار» (Support, Confidence, Lift, Conviction)

تیم داده‌کاوی پلتفرم «دیوار» برای ارائهٔ پیشنهادات هوشمند در بخشِ پایین آگهی‌ها، نمی‌تواند تنها به معیارِ سادهٔ اطمینان (Confidence) بسنده کند. برای تفکیکِ الگوهای واقعاً کاربردی از رفتارهای تصادفی و فریبنده، از ترکیبِ ۴ معیارِ اصلیِ ریاضی استفاده می‌شود.

۱. پشتیبانی (Support): درصد رخدادِ همزمانِ یک الگو در گُلِ سیستم.

$$\text{Support}(X \rightarrow Y) = P(X \cap Y) = \frac{\text{Count}(X \cup Y)}{N}$$

۲. اطمینان (Confidence): احتمالِ شرطیِ رخ دادنِ پیامد (Y) با شرطِ وجودِ پیش‌آمد (X).

$$\text{Confidence}(X \rightarrow Y) = P(Y|X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

۳. لیفت (Lift) – اندازه‌گیری قدرت ارتباط و هم‌افزایی (دو طرفه):

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

۴. کانویکشن (Conviction) – اندازه‌گیری استنتاجِ منطقی (وابستگی یک‌طرفه):

این معیار، میزان «وابستگی یک‌طرفه» را می‌سنجد. کانویکشن بالا یعنی: «اگر مشتری X را ببیند اما Y را نبیند، اتفاق بسیار عجیبی افتاده است!».

$$\text{Conviction}(X \rightarrow Y) = \frac{1 - \text{Support}(Y)}{1 - \text{Confidence}(X \rightarrow Y)}$$

مرحله نخست: تولید دیتاستِ نشست‌های بازدید آگهی و تبدیل ماتریسی

نمونه نشست‌های بازدید آگهی در وب‌سایت دیوار		(باینری‌سازیِ دسته‌بندی‌ها) One-Hot ماتریس				
Session_ID	Viewed_Ad_Categories	Apartment_Rent	Auto_Body_Shop	Car_Insurance	Car_Sale	Home_Painting
1	Apartment_Rent, Moving_Service, Home_Painting	1	0	0	0	1
2	Apartment_Rent, Moving_Service	1	0	0	0	1
3	Car_Sale, Auto_Body_Shop	0	1	0	1	0
4	Apartment_Rent, Moving_Service, Home_Appliance	1	0	0	0	1
5	Car_Sale, Car_Insurance	0	0	1	1	0
6	Apartment_Rent, Home_Painting	1	0	0	0	1

مرحله دوم: استخراج قواعد و کنترل پدیدهٔ Infinity در کانویکشن

ارزیابی جامعِ ۴ بُعدیِ قواعد پیشنهاد آگهی (با رفع مشکل بی‌نهایت)					
Rule_ID	rule	support	confidence	lift	conviction_clean
R1	Car_Sale -> Car_Insurance	0.25	0.75	3.00	3.00
R2	Car_Insurance -> Car_Sale	0.25	1.00	3.00	5.00
R3	Home_Painting -> Apartment_Rent	0.25	1.00	1.71	5.00
R4	Home_Appliance -> Moving_Service	0.33	1.00	1.71	5.00
R5	Home_Appliance, Apartment_Rent -> Moving_Service	0.25	1.00	1.71	5.00
R6	Moving_Service -> Home_Appliance	0.33	0.57	1.71	1.56

مصورسازی تحلیلی: مقایسهٔ چندبُعدیِ معیارهای کیفی

Figure 1: Multi-Metric Evaluation of Top Association Rules

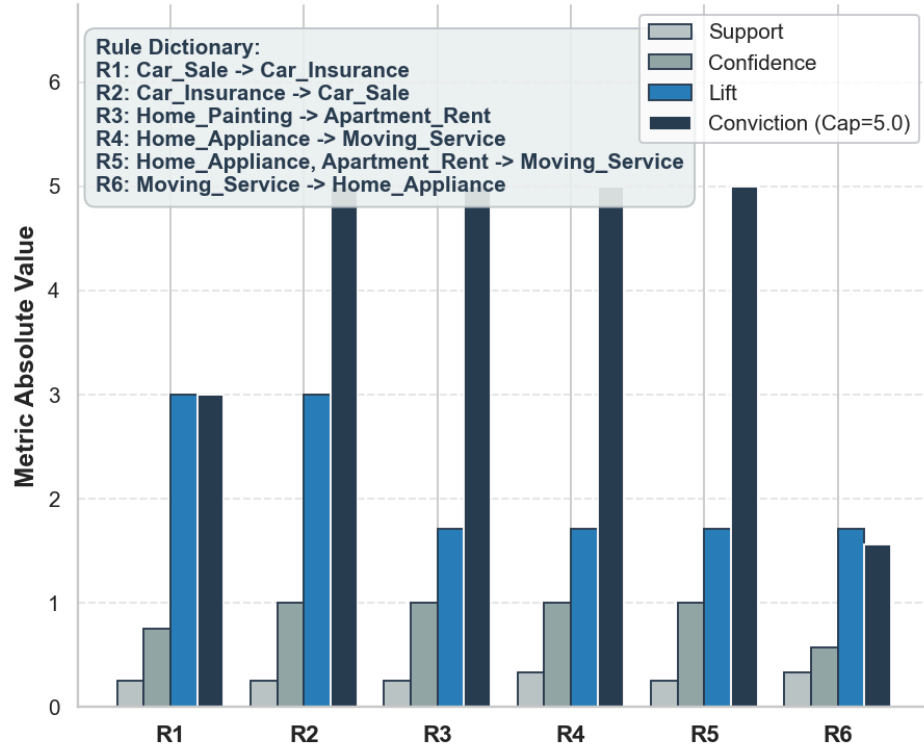
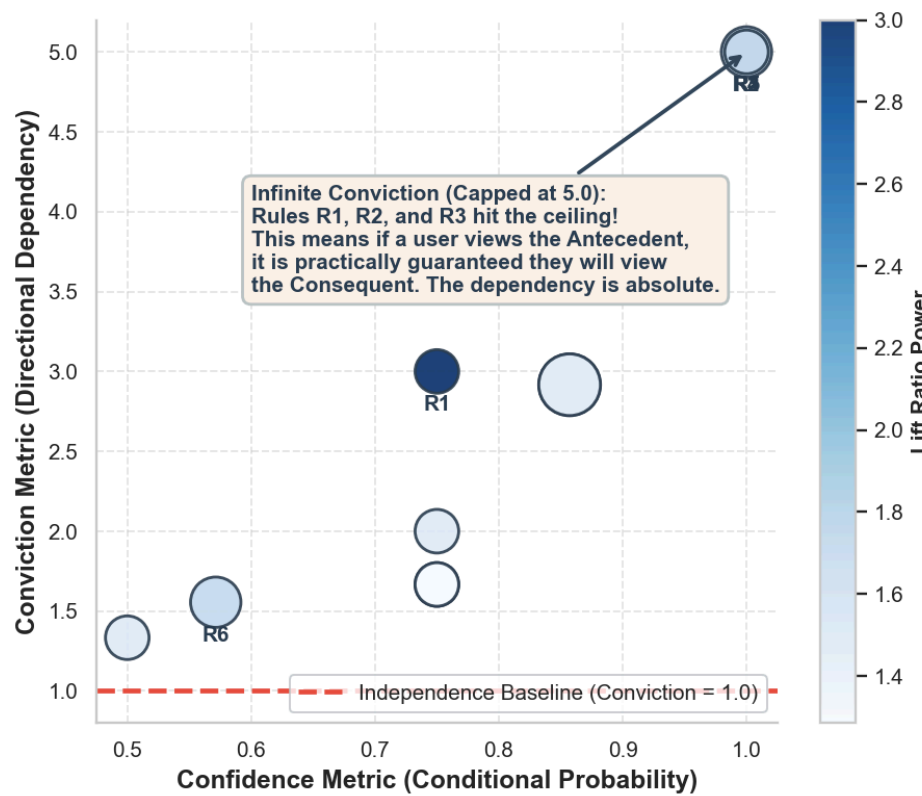


Figure 2: Confidence vs Conviction (Size = Support, Color = Lift)



نتیجه استراتژیک در مهندسی قوانین (Strategic Implication):

با بررسی گراف ۱ متوجه می‌شویم که مقایسهٔ تک‌بعدی قوانین کاملاً اشتباه است. قاعدهٔ **R1 (Home_Appliance -> Moving_Service)** دارای بالاترین مقدار Lift و Conviction (رسیده به سقف بی‌نهایت ۵.۰) است؛

این یعنی *استنتاج منطقی یک‌طرفهٔ* این قاعده بسیار قدرتمند است: کاربرانی که آگهی «لوازم خانگی» را می‌بینند، به‌طور قطع به دنبال «خدمات باربری» خواهند بود (چون جابه‌جایی لوازم خانگی نیاز به وانت دارد). این در حالی است که کانویکشنِ برعکسِ آن (باربری → لوازم خانگی) ممکن است بسیار پایین باشد (چون کسی که وانت می‌خواهد شاید در حال اسباب‌کشیِ گِل خانه باشد، نه فقط یک یخچال!).

در مقابل، قواعدی با کانویکشنِ نزدیک به ۱ (مانند R4)، نشان‌دهندهٔ *استقلال آماری (Independence)* هستند و ارزش تجاریِ کمتری برای قرارگیری در بخش «پیشنهادهای ویژهٔ دیوار» دارند.

توضیح مثال کد پایتون

در این مثال:

- داده‌های نمونه دیوار: تعداد ۱۲ نشست واقعی کاربران شامل بازدید از آگهی‌های رهن آپارتمان، خدمات اسباب‌کشی، نقاشی ساختمان، خرید خودرو و بیمه خودرو شبیه‌سازی شد.
- محاسبه هم‌زمان معیارهای ۴ گانه: با تابع `association_rules`، تمامی معیارهای پشتیبانی، اطمینان، لیفت و کانویکشن برای الگوهای بازدید هم‌زمان آگهی‌ها استخراج شدند.
- پیش‌پردازش کانویکشن: مقادیر بی‌نهایت (Infinity) که در صورت اطمینان ۱۰۰٪ رخ می‌دهند با عدد ۵ جایگزین شدند تا رسم نمودارها بدون خطا انجام شود.
- تفسیر خروجی و نمودارهای Matplotlib:
 - نمودار سمت چپ (میله‌ای ۴ تایی): مقایسه معیارهای چهارگانه را برای برترین قواعد آگهی‌های دیوار نشان می‌دهد؛ مشخص است که قاعده `Apartment_Rent -> Moving_Service` بالاترین میزان لیفت و کانویکشن را به دست آورده است.
 - نمودار سمت راست (Scatter Plot): جایگاه قواعد را در فضای چندبعدی نشان می‌دهد که در آن ارتفاع نقطه نمایانگر کانویکشن، افق نمایانگر اطمینان، رنگ نشان‌دهنده لیفت و اندازه نقطه نشان‌دهنده پشتیبانی است.

تمرین فصل چهارم – Pattern Mining: Basic Concepts and Methods

- سوال ۱ یک فروشگاه می‌بیند مشتریانی که A می‌خرند، اغلب B را هم می‌خرند و مدیر می‌خواهد آنها را کنار هم قرار دهد. چه چیزهایی را قبل از این تصمیم بررسی می‌کنید؟ (ممکن است B تقریباً توسط همه خریداری شود؛ در این حالت «اغلب بودن» اطلاعات زیادی درباره رابطه A و B نمی‌دهد!)
- سوال ۲ سیستم هزاران pattern پیدا کرده، اما تیم کسب‌وکار فقط به چند مورد علاقه‌مند است. چگونه فضای خروجی را کوچک‌تر و مفیدتر می‌کنید؟ (از ابتدا معیارهایی برای جالب بودن pattern تعیین کنید و مواردی را که با هدف کسب‌وکار بی‌ارتباط‌اند کنار بگذارید.)

3. سوال ۳ تعداد آیتم‌های موجود آن‌قدر زیاد است که بررسی همه ترکیب‌های ممکن بسیار پرهزینه خواهد بود. چه ایده‌ای برای جلوگیری از رشد بی‌رویه فضای جست‌وجو دارید؟ (یک ایده مهم این است که وقتی یک ترکیب کوچک دیگر امیدی به جالب‌بودن ندارد، بخشی از ترکیب‌های بزرگ‌تر وابسته به آن را هم بررسی نکنید!)
4. سوال ۴ دو Rule هر دو «قوی» هستند، اما مدیر فقط یکی را مفید می‌داند. چطور توضیح می‌دهید که strength به‌تنهایی برای interesting بودن کافی نیست؟ (به این فکر کنید که یک Rule ممکن است به دلیل محبوبیت عمومی آیتم‌ها قوی دیده شود، بدون اینکه رابطه تازه‌ای کشف کرده باشد.)
5. سوال ۵ مدیر به جای «چه چیزهایی با هم خریداری می‌شوند» می‌پرسد: «معمولاً کاربر بعد از مشاهده A سراغ چه چیزی می‌رود؟» چه چیزی در مسئله تغییر کرده و چه نوع داده‌ای باید حفظ شود؟ (در اینجا ترتیب رخدادها مهم شده است؛ بنابراین timestamp یا sequence را از دست ندهید!)
6. سوال ۶ اگر فقط اجازه داشته باشید یک pattern را برای مدیر ارائه دهید، کدام یک را ترجیح می‌دهید؟ بسیار رایج ولی بدیهی، کمتر رایج ولی جدید، یا نادر ولی با ارزش تجاری بالا؟ معیار انتخاب خود را تعریف کنید. (یک ترکیب ساده از «تازگی + قابلیت اقدام + اهمیت کسب‌وکار» می‌تواند نقطه شروع خوبی باشد.)
7. سوال ۷ در یک مصاحبه فنی از شما می‌خواهند یک کاربرد خلاقانه برای Association Analysis پیشنهاد کنید که مربوط به فروشگاه و خرید محصول نباشد. یک کاربرد طراحی کنید و بگویید چه patternی قرار است کشف شود. (به جای «چه چیزی با چه چیزی می‌آید»، به رخدادهای کاربران، اسناد، خطاهای نرم‌افزاری یا رفتارهای زمانی فکر کنید.)